

```

1 import os
2 import sys
3 import sqlite3
4 import hashlib
5 from PyQt6.QtWidgets import QApplication, QStackedWidget
6
7 from ui.login_ui import LoginUI
8 from ui.register_ui import RegisterUI
9 from ui.main_window_ui import MainWindowUI
10 from modules.login_module import LoginModule
11 from modules.register_module import RegisterModule
12 from core.database import init_db, ensure_default_admin
13 from core.hot_reload import HotReload
14
15
16 class AuthContainer(QStackedWidget):
17     def __init__(self):
18         super().__init__()
19         self.setWindowTitle("图片内容智能审核与违规检测系统-认证中心")
20         self.setFixedSize(450, 640)
21
22         self.login_ui = LoginUI()
23         self.register_ui = RegisterUI()
24
25         self.login_logic = LoginModule(self.login_ui)
26         self.register_logic = RegisterModule(self.register_ui)
27
28         self.addWidget(self.login_ui)
29         self.addWidget(self.register_ui)
30
31         self.login_ui.register_clicked.connect(self._goto_register)
32         self.register_ui.login_clicked.connect(self._goto_login)
33
34         self.register_logic.register_success.connect(self._goto_login)
35         self.login_logic.login_success.connect(self._on_login_verified)
36
37     def _goto_register(self):
38         self.register_ui.refresh_captcha()
39         self.setCurrentIndex(1)
40
41     def _goto_login(self):
42         self.login_ui.captcha_widget.update_captcha()
43         self.login_ui.captcha_input.clear()
44         self.setCurrentIndex(0)
45
46     def _on_login_verified(self, username):
47         role = "auditor"
48         try:
49             conn = sqlite3.connect("data/app.db")
50             cursor = conn.cursor()
51             cursor.execute("SELECT role FROM users WHERE username=?",
(username,))
52             row = cursor.fetchone()
53             if row:
54                 role = row[0]

```

```

55         conn.close()
56     except Exception as e:
57         print(f"角色查询失败: {e}")
58
59     self.main_window = MainWindowUI(username, role)
60     self.main_window.show()
61     self.close()
62
63
64 def main():
65
66     from PyQt6.QtCore import Qt
67     QApplication.setHighDpiScaleFactorRoundingPolicy(
68         Qt.HighDpiScaleFactorRoundingPolicy.PassThrough
69     )
70     app = QApplication(sys.argv)
71
72     reloader = HotReload(watch_dir="modules", enabled=True)
73     reloader.start()
74
75
76     init_db()
77
78     ensure_default_admin()
79
80     auth = AuthContainer()
81     auth.show()
82     reloader.stop()
83     sys.exit(app.exec())
84
85
86
87 if __name__ == "__main__":
88     main()
89 import random
90
91
92 class ContentDetector:
93     """内容违规检测类（模拟预测）"""
94
95     def __init__(self):
96
97         self.classes = ["正常", "色情", "暴恐", "违禁"]
98
99     def predict(self, image_array):
100         """
101         对传入的图片数组进行内容审核
102         :param image_array: 图片 Numpy 数组（例如 preprocess 返回的值）
103         :return: 包含类别和置信度的字典
104         """
105
106         predicted_class = random.choice(self.classes)
107         confidence = round(random.uniform(0.60, 0.99), 2)
108
109         return {

```

```

110         "class": predicted_class,
111         "confidence": confidence,
112         "details": {"note": "此结果为系统模拟返回值，用于前后端联调测试。"}
113     } 114 import sqlite3
115 import os
116 import hashlib
117
118 DB_PATH = os.path.join("data", "app.db")
119
120
121 def init_db():
122     """初始化 SQLite 数据库并创建必要的表（如果不存在的话）"""
123
124     os.makedirs("data", exist_ok=True)
125
126
127     conn = sqlite3.connect(DB_PATH)
128     cursor = conn.cursor()
129
130
131     cursor.execute('''
132         CREATE TABLE IF NOT EXISTS users (
133             id INTEGER PRIMARY KEY AUTOINCREMENT,
134             username TEXT UNIQUE NOT NULL,
135             password_hash TEXT NOT NULL,
136             email TEXT,
137             role TEXT DEFAULT 'auditor',
138             locked_until DATETIME
139         )
140     ''')
141
142
143     cursor.execute('''
144         CREATE TABLE IF NOT EXISTS audit_records (
145             id INTEGER PRIMARY KEY AUTOINCREMENT,
146             file_path TEXT,
147             result TEXT,
148             confidence REAL,
149             auditor TEXT,
150             audit_time TEXT,
151             status TEXT,
152             recognized_text TEXT,
153             is_deleted INTEGER DEFAULT 0
154         )
155     ''')
156
157
158     cursor.execute('''
159         CREATE TABLE IF NOT EXISTS sensitive_words (
160             id INTEGER PRIMARY KEY AUTOINCREMENT,
161             word TEXT UNIQUE NOT NULL,
162             category TEXT
163         )
164     ''')

```

```

165
166
167     cursor.execute('''
168         CREATE TABLE IF NOT EXISTS login_attempts (
169             id INTEGER PRIMARY KEY AUTOINCREMENT,
170             username TEXT NOT NULL,
171             attempt_time DATETIME DEFAULT CURRENT_TIMESTAMP,
172             success BOOLEAN NOT NULL
173         )
174     ''')
175
176     conn.commit()
177     conn.close()
178     print("[数据库] 初始化成功，数据表已就绪。")
179
180
181 def ensure_default_admin():
182     """确保数据库中存在默认 admin 账号，密码 admin123"""
183     conn = sqlite3.connect("data/app.db")
184     cursor = conn.cursor()
185     cursor.execute("SELECT COUNT(*) FROM users WHERE username='admin'")
186     if cursor.fetchone()[0] == 0:
187         admin_hash = hashlib.sha256("admin123".encode('utf-
188 8')).hexdigest()
189         cursor.execute(
190             "INSERT INTO users (username, password_hash, email, role)
191             VALUES (?, ?, ?, ?)",
192             ("admin", admin_hash, "admin@iavds.com", "admin")
193         )
194         conn.commit()
195         print("默认管理员账号已创建: admin / admin123")
196     conn.close()
197
198 import random
199
200 def compute_hash(image_path: str) -> str:
201     """
202     计算图片的哈希值（模拟感知哈希，这里用 MD5 占位）
203     :param image_path: 图片路径
204     :return: 哈希字符串
205     """
206     hash_md5 = hashlib.md5()
207     hash_md5.update(image_path.encode('utf-8'))
208     return hash_md5.hexdigest()
209
210 def check_duplicate(hash_value: str) -> bool:
211     """
212     检查该哈希值是否已经在违规库中存在
213     :param hash_value: 图片的哈希字符串
214     :return: True(存在重复/违规) 或 False(未重复)
215     """
216     return random.random() < 0.1
217
218 import os
219 import sys

```

```

219 import importlib
220 from watchdog.observers import Observer
221 from watchdog.events import FileSystemEventHandler
222
223
224 class ModuleReloadHandler(FileSystemEventHandler):
225     """处理文件修改事件的回调类"""
226
227     def on_modified(self, event):
228
229         if event.src_path.endswith('.py'):
230             filename = os.path.basename(event.src_path)
231             module_name = filename[:-3]
232
233
234             full_module_name = f"modules.{module_name}"
235
236
237             if full_module_name in sys.modules:
238                 try:
239                     importlib.reload(sys.modules[full_module_name])
240                     print(f"[热更新] 成功重载业务模块: {full_module_name}")
241                 except Exception as e:
242                     print(f"[热更新] 重载失败 {full_module_name}: {e}")
243
244
245 class HotReload:
246     """热更新管理器"""
247
248     def __init__(self, watch_dir="modules", enabled=True):
249         self.watch_dir = watch_dir
250         self.enabled = enabled
251         self.observer = None
252
253     def start(self):
254         if not self.enabled:
255             return
256
257
258         os.makedirs(self.watch_dir, exist_ok=True)
259
260         self.observer = Observer()
261         event_handler = ModuleReloadHandler()
262
263         self.observer.schedule(event_handler, self.watch_dir,
recursive=False)
264         self.observer.start()
265         print(f"[热更新] 已启动, 正在监听 {self.watch_dir}/ 目录下的修改...")
266
267     def stop(self):
268         if self.observer:
269             self.observer.stop()
270             self.observer.join()
271 import cv2
272 import numpy as np

```

```

273 import os
274
275
276 def preprocess(image_path: str):
277     """
278     图像预处理函数
279     :param image_path: 图片的绝对或相对路径
280     :return: (processed_image_array, meta_dict) 返回处理后的 Numpy 数组和图
片元数据
281     """
282
283     if not os.path.exists(image_path):
284         dummy_img = np.zeros((224, 224, 3), dtype=np.uint8)
285         return dummy_img, {"width": 224, "height": 224, "format":
"unknown", "error": "file not found"}
286
287     img = cv2.imdecode(np.fromfile(image_path, dtype=np.uint8),
cv2.IMREAD_COLOR)
288     if img is None:
289         dummy_img = np.zeros((224, 224, 3), dtype=np.uint8)
290         return dummy_img, {"width": 224, "height": 224, "format":
"unknown", "error": "decode failed"}
291
292     height, width, _ = img.shape
293
294     processed_img = cv2.resize(img, (224, 224))
295
296
297     fmt = image_path.split('.')[-1].lower() if '.' in image_path else
"unknown"
298     meta_dict = {"width": width, "height": height, "format": fmt}
299
300     return processed_img, meta_dict
301 import easyocr
302 import traceback
303
304
305 class OCRDetector:
306     """OCR 光学字符识别类（真实提取版 - 基于深度学习 EasyOCR）"""
307
308     def __init__(self):
309         print("[系统加载] 正在初始化 OCR 深度学习模型（中英双语）...")
310         try:
311
312
313             self.reader = easyocr.Reader(['ch_sim', 'en'], gpu=True)
314             print("[系统加载] OCR 模型加载完毕！")
315         except Exception as e:
316             print(f"[错误] OCR 模型加载失败：{e}")
317             self.reader = None
318
319     def detect_text(self, image_array, global_sensitive_words=None):
320         """
321         真实提取图片中的文本并进行敏感词初筛
322         :param image_array: 图片 Numpy 数组（由 cv2.imread 或 imdecode 生成）

```

```

323         :param global_sensitive_words: 敏感词列表, 建议从你系统的
SensitiveWordModule 数据库里传过来
324         :return: (text_list, sensitive_words_list)
325         """
326         text_list = []
327         sensitive_words_list = []
328
329         if self.reader is None:
330             return text_list, sensitive_words_list
331
332
333         if global_sensitive_words is None:
334             global_sensitive_words = ["违规", "测试敏感词", "发票", "代考",
"赌博"]
335
336         try:
337
338
339             extracted_texts = self.reader.readtext(image_array, detail=0)
340
341
342             text_list = [text.strip() for text in extracted_texts if
text.strip()]
343
344
345             for text in text_list:
346                 for word in global_sensitive_words:
347
348                     if word in text and word not in sensitive_words_list:
349                         sensitive_words_list.append(word)
350
351         except Exception as e:
352             print(f"OCR 识别过程发生崩溃:")
353             traceback.print_exc()
354
355         return text_list, sensitive_words_list
356
357     def detect_and_highlight(self, image_array,
global_sensitive_words=None):
358         """
359         提取文字、过滤背景乱码、并对敏感词进行 HTML 高亮染色
360         :param image_array: 图片 Numpy 矩阵
361         :param global_sensitive_words: 敏感词库列表
362         :return: (纯文字列表, 命中的敏感词列表, 供前端渲染的 HTML 高亮文本)
363         """
364         text_list = []
365         sensitive_words_list = []
366         html_output = ""
367
368         if self.reader is None:
369             return text_list, sensitive_words_list, "<b
style='color:red;'>OCR 引擎未就绪</b>"
370
371         if global_sensitive_words is None:

```

```

372         global_sensitive_words = ["违规", "赌博", "发票", "代考"]
373
374     try:
375
376
377         results = self.reader.readtext(image_array, detail=1)
378
379         for bbox, text, confidence in results:
380             text = text.strip()
381             if not text:
382                 continue
383
384
385
386             if confidence < 0.45:
387                 print(f"[系统清洗] 过滤疑似背景噪点乱码: '{text}' (置信
度: {confidence:.2f})")
388                 continue
389
390
391                 text_list.append(text)
392
393
394                 highlighted_line = text
395                 is_line_sensitive = False
396
397                 for word in global_sensitive_words:
398                     if word in highlighted_line:
399                         is_line_sensitive = True
400                         if word not in sensitive_words_list:
401                             sensitive_words_list.append(word)
402
403
404                             highlighted_line = highlighted_line.replace(
405                                 word,
406                                 f'<span style="color: #EF4444; font-weight:
bold; background-color: #FEE2E2; padding: 2px 4px; radius:
3px;">{word}</span>'
407                             )
408
409
410                 if is_line_sensitive:
411                     html_output += f'<p style="line-height: 150%; margin:
4px 0; color: #1E293B;">△ {highlighted_line}</p >'
412                 else:
413                     html_output += f'<p style="line-height: 150%; margin:
4px 0; color: #475569;"># {highlighted_line}</p >'
414
415                 if not html_output:
416                     html_output = "<p style='color: #94A3B8;'>未从图片中识别到
有效文本。</p >"
417
418     except Exception as e:
419         html_output = f"<p style='color: red;'>OCR 识别发生异常:

```



```

{str(e)}</p >"
420         traceback.print_exc()
421
422     return text_list, sensitive_words_list, html_output
423 import configparser
424 import os
425
426 CONFIG_PATH = os.path.join("data", "config.ini")
427
428
429 def get_default_config() -> configparser.ConfigParser:
430     """生成系统的默认配置"""
431     config = configparser.ConfigParser()
432     config['LOGIN'] = {
433         'remember_username': '',
434         'encrypted_password': '',
435         'max_attempts': '5',
436         'lock_minutes': '15'
437     }
438     return config
439
440
441 def read_config() -> configparser.ConfigParser:
442     """读取 config.ini 文件，如果不存在则自动创建并写入默认配置"""
443     os.makedirs("data", exist_ok=True)
444     config = get_default_config()
445
446     if os.path.exists(CONFIG_PATH):
447         config.read(CONFIG_PATH, encoding='utf-8')
448     else:
449         save_config(config)
450
451     return config
452
453
454 def save_config(config: configparser.ConfigParser):
455     """将配置对象保存到 config.ini 文件中"""
456     os.makedirs("data", exist_ok=True)
457     with open(CONFIG_PATH, 'w', encoding='utf-8') as f:
458         config.write(f)
459 import time
460 import random
461
462 class AboutSystemModule:
463     """关于系统的业务逻辑层，处理版本信息与更新检查"""
464
465     def __init__(self):
466         self.info = {
467             "name": "图片内容智能审核与违规检测系统",
468             "version": "v1.0 (Build 20260524)",
469             "developer": "Coding Assistant Team",
470             "tech_stack": "Python 3.10, PyQt6, SQLite, Matplotlib",
471             "license": "MIT Open Source License"
472         }
473

```

```

474     def get_system_info(self):
475         return self.info
476
477     def check_for_updates(self):
478         """模拟向服务器请求更新：返回是否需要更新及版本信息"""
479
480         time.sleep(0.5)
481
482
483         has_update = random.choice([True, False])
484         if has_update:
485             return {
486                 "update_available": True,
487                 "latest_version": "v1.3.0",
488                 "download_url":
489                 "https://example.com/downloads/iavds_v1.3.0.zip",
490                 "changelog": "1. 优化了 AI 识别精度\n2. 修复了已知崩溃问题"
491             }
492         return {"update_available": False}
493
494 import os
495 import time
496 from PyQt6.QtCore import QObject
497
498 class AppealHandlingModule(QObject):
499     """申诉处理业务控制层"""
500
501     def __init__(self, ui_instance, current_user="Admin_Auditor_01"):
502         super().__init__()
503         self.ui = ui_instance
504         self.db_path = "data/app.db"
505         self.current_user = current_user
506
507         self.ensure_tables_exist()
508
509
510         self.ui.query_requested.connect(self.load_appeals)
511         self.ui.review_started.connect(self.mark_as_reviewing)
512         self.ui.decision_submitted.connect(self.process_decision)
513
514     def ensure_tables_exist(self):
515         """确保申诉表存在，并灌入模拟数据供演示"""
516         os.makedirs(os.path.dirname(self.db_path), exist_ok=True)
517         try:
518             conn = sqlite3.connect(self.db_path)
519             cursor = conn.cursor()
520
521             cursor.execute('''
522                 CREATE TABLE IF NOT EXISTS appeals (
523                     id INTEGER PRIMARY KEY AUTOINCREMENT,
524                     record_id INTEGER,
525                     submitter TEXT,
526                     submit_time TEXT,
527                     reason TEXT,

```

```

528             status TEXT DEFAULT '待处理',
529             reviewer TEXT,
530             review_time TEXT,
531             review_comment TEXT
532         )
533     '''
534
535
536     cursor.execute("SELECT COUNT(*) FROM appeals")
537     if cursor.fetchone()[0] == 0:
538         mock_appeals = [
539             (1, "User_9527", "2026-05-23 10:00:00", "这是一张风景
照，识别出的敏感词是路牌名字，存在误判请核实！",
540             "待处理"),
541             (2, "ContentCreator_X", "2026-05-24 09:15:00", "我发布
的图片并没有违规暴露，请求人工介入复审。",
542             "复审中")
543         ]
544     cursor.executemany('''
545         INSERT INTO appeals (record_id, submitter,
submit_time, reason, status)
546         VALUES (?, ?, ?, ?, ?)
547     ''', mock_appeals)
548     conn.commit()
549     conn.close()
550 except Exception as e:
551     self.ui.show_toast("数据库错误", str(e), True)
552
553 def load_appeals(self, status_filter):
554     """核心查询：联合查询申诉表与审核记录表"""
555     try:
556         conn = sqlite3.connect(self.db_path)
557         cursor = conn.cursor()
558
559
560         query = '''
561             SELECT a.id, a.record_id, a.submitter, a.submit_time,
a.status, a.reason,
562             r.file_path, r.result, r.confidence,
r.recognized_text
563             FROM appeals a
564             LEFT JOIN audit_records r ON a.record_id = r.id
565         '''
566         params = []
567
568         if status_filter != "全部":
569             query += " WHERE a.status = ?"
570             params.append(status_filter)
571
572         query += " ORDER BY a.submit_time DESC"
573
574         cursor.execute(query, params)
575         records = cursor.fetchall()
576         conn.close()

```

```

577
578         self.ui.render_table(records)
579     except Exception as e:
580         self.ui.show_toast("查询失败", str(e), True)
581
582     def mark_as_reviewing(self, appeal_id):
583         """状态流转: 将工单标记为本人复审中"""
584         try:
585             conn = sqlite3.connect(self.db_path)
586             cursor = conn.cursor()
587             cursor.execute("UPDATE appeals SET status = '复审中', reviewer
= ? WHERE id = ?",
588                             (self.current_user, appeal_id))
589             conn.commit()
590             conn.close()
591             self.load_appeals(self.ui.status_combo.currentText())
592         except Exception as e:
593             print(f"状态更新失败: {e}")
594
595     def process_decision(self, appeal_id, record_id, decision, comment):
596         new_status = "已撤销" if decision == "撤销原判" else "已维持"
597         current_time = time.strftime("%Y-%m-%d %H:%M:%S")
598         conn = None
599         try:
600             conn = sqlite3.connect(self.db_path)
601             cursor = conn.cursor()
602             cursor.execute("BEGIN TRANSACTION")
603
604
605             cursor.execute('''
606                 UPDATE appeals
607                 SET status = ?, review_comment = ?, review_time = ?,
reviewer = ?
608                 WHERE id = ?
609             ''', (new_status, comment, current_time, self.current_user,
appeal_id))
610
611
612             if decision == "撤销原判":
613                 cursor.execute('''
614                     UPDATE audit_records
615                     SET result = '正常(人工申诉撤销)', status = '已完成'
616                     WHERE id = ?
617                 ''', (record_id,))
618
619             conn.commit()
620
621
622             cursor.execute("SELECT submitter FROM appeals WHERE id = ?",
(appeal_id,))
623             row = cursor.fetchone()
624             submitter = row[0] if row else "未知用户"
625
626             self.send_site_notification(submitter, appeal_id, new_status,

```

```

comment)
627         self.ui.show_toast("处理成功", f"工单 APL-{appeal_id:04d} 仲裁
完毕! \n 结果: {new_status}")
628         self.load_appeals(self.ui.status_combo.currentText())
629
630     except Exception as e:
631         if conn:
632             conn.rollback()
633             import traceback
634             traceback.print_exc()
635             self.ui.show_toast("处理失败", f"事务提交异常:\n{e}", True)
636     finally:
637         if conn:
638             conn.close()
639
640     def send_site_notification(self, user_id, appeal_id, final_status,
comment):
641         """跨模块模拟: 记录日志并发送站内信"""
642         log_msg = f"[{time.strftime('%H:%M:%S')}] 系统通知已发送给 {user_id}
-> 您的申诉工单(#{appeal_id}) 处理结果: {final_status}。审核反馈: {comment}"
643         print("-" * 50)
644         print(log_msg)
645         print("-" * 50)
646     import sqlite3
647     import os
648     import time
649     from PyQt6.QtCore import QObject
650     from datetime import datetime
651
652     now = datetime.now()
653
654
655     class AuditHistoryModule(QObject):
656         """资产审计历史数据控制枢纽(纯事务逻辑, 无 widget 污染)"""
657
658     def __init__(self, ui_instance):
659         super().__init__()
660         self.ui = ui_instance
661         self.db_path = "data/app.db"
662         self.page_size = 12
663
664         self.ensure_table_exists()
665
666
667         self.ui.query_requested.connect(self.load_history_by_conditions)
668         self.ui.delete_requested.connect(self.execute_soft_delete)
669         self.ui.appeal_api_call.connect(self.forward_to_appeal_module)
670         self.ui.report_export_call.connect(self.export_single_report_file)
671
672     def ensure_table_exists(self):
673         """保证审计记录历史核心表的安全底座"""
674         try:
675             os.makedirs(os.path.dirname(self.db_path), exist_ok=True)
676             conn = sqlite3.connect(self.db_path)
677             cursor = conn.cursor()

```

```

678
679
680         cursor.execute('''
681             CREATE TABLE IF NOT EXISTS audit_records (
682                 id INTEGER PRIMARY KEY AUTOINCREMENT,
683                 file_path TEXT,
684                 audit_time TEXT,
685                 status TEXT,
686                 result TEXT,
687                 confidence REAL,
688                 recognized_text TEXT,
689                 auditor TEXT,
690                 is_deleted INTEGER DEFAULT 0
691             )
692         ''')
693
694
695         cursor.execute("SELECT COUNT(*) FROM audit_records")
696         if cursor.fetchone()[0] == 0:
697             mock_data = [
698                 ("data/samples/pic1.jpg", "2026-05-10 14:22:11", "已
完成", "正常", 0.98, "测试样本合法文字",
699                 "System_Auditor", 0),
700                 ("data/samples/illegal_demo.png", "2026-05-12
09:15:34", "已驳回", "违规风险", 0.89,
701                 "含有敏感政治标语", "Auditor_02", 0),
702                 ("data/samples/blur_face.jpeg", "2026-05-15 18:40:02",
"待复审", "未确定", 0.52, "人脸模糊无法穿透",
703                 "Auditor_05", 0),
704             ]
705             cursor.executemany('''
706                 INSERT INTO audit_records (file_path, audit_time,
status, result, confidence, recognized_text, auditor, is_deleted)
707                 VALUES (?, ?, ?, ?, ?, ?, ?, ?)
708             ''', mock_data)
709             conn.commit()
710             conn.close()
711         except Exception as e:
712             print(f"初始化历史记录数据库失败: {e}")
713
714     def load_history_by_conditions(self, filter_payload: dict):
715         """【核心逻辑】支持复杂多条件组合查询与逻辑状态剔除的多功能引擎"""
716         try:
717             conn = sqlite3.connect(self.db_path)
718             cursor = conn.cursor()
719
720
721             sql_query = "SELECT id, file_path, audit_time, status, result,
confidence, recognized_text, auditor, is_deleted FROM audit_records WHERE
is_deleted = 0"
722             params = []
723
724
725             sql_query += " AND date(audit_time) >= date(?) AND

```

```

date(audit_time) <= date(?)"
726         params.extend([filter_payload["start_date"],
filter_payload["end_date"]])
727
728
729         if filter_payload["status"] != "全部":
730             sql_query += " AND status = ?"
731             params.append(filter_payload["status"])
732
733
734         if filter_payload["result"] != "全部":
735             sql_query += " AND result LIKE ?"
736             params.append(f"%{filter_payload['result']}%")
737
738
739         if filter_payload["keyword"]:
740             sql_query += " AND (file_path LIKE ? OR auditor LIKE ? OR
recognized_text LIKE ?)"
741             kw = f"%{filter_payload['keyword']}%"
742             params.extend([kw, kw, kw])
743
744
745             count_sql = f"SELECT COUNT(*) FROM ({sql_query})"
746             cursor.execute(count_sql, params)
747             total_records = cursor.fetchone()[0]
748             total_pages = (total_records + self.page_size - 1) //
self.page_size
749
750
751             req_page = filter_payload["page"]
752             active_page = max(1, min(req_page, total_pages if
total_pages > 0 else 1))
753
754
755             sql_query += " ORDER BY audit_time DESC LIMIT ? OFFSET ?"
756             params.extend([self.page_size, (active_page - 1) *
self.page_size])
757
758             cursor.execute(sql_query, params)
759             records = cursor.fetchall()
760             conn.close()
761
762
763             self.ui.render_history_grid(records, total_pages, active_page)
764
765         except Exception as e:
766             self.ui.show_toast("故障提示", f"检索审计轨迹数据库发生中
断:\n{str(e)}", is_error=True)
767
768     def execute_soft_delete(self, record_id: int):
769         """【安全追溯】对单条目标对象实施逻辑删除，不物理粉碎底层行记录"""
770         try:
771             conn = sqlite3.connect(self.db_path)
772             cursor = conn.cursor()

```

```

773
774         cursor.execute("UPDATE audit_records SET is_deleted = 1 WHERE
id = ?", (record_id,))
775         conn.commit()
776         conn.close()
777
778         self.ui.show_toast("操作成功", f"记录 #{record_id} 已成功从历史
视图中隐藏，数据依然留底可供追溯。")
779
780         self.ui.emit_combined_query()
781     except Exception as e:
782         self.ui.show_toast("软清理失败", f"逻辑数据变更发生异
常:\n{str(e)}", is_error=True)
783
784     def forward_to_appeal_module(self, record_id: int):
785         """外部申诉接口通信联动代理（真正将申诉工单落地入库）"""
786         try:
787             conn = sqlite3.connect(self.db_path)
788             cursor = conn.cursor()
789
790             # 🚩 1. 防呆机制：检查该条记录是否已经存在“待处理”或“复审中”的申
诉，防止重复提交
791             cursor.execute(
792                 "SELECT COUNT(*) FROM appeals WHERE record_id = ? AND
status IN ('待处理', '复审中')",
793                 (record_id,))
794             )
795             if cursor.fetchone()[0] > 0:
796                 conn.close()
797                 self.ui.show_toast("请勿重复申诉", f"记录 #{record_id} 已在
申诉处理队列中，请勿重复提交！", is_error=True)
798                 return
799
800             # 🚩 2. 获取当前的系统本地时间
801             current_time = time.strftime("%Y-%m-%d %H:%M:%S")
802
803             # 🚩 3. 构造真实的申诉单基础数据并写入 appeals 表
804             # 对应 appeal_handling.py 里的表结构: record_id, submitter,
submit_time, reason, status
805             default_reason = "前端用户通过历史轨迹面板发起：对系统判定结论存
在异议，申请人工仲裁复核。"
806
807             cursor.execute('''
808                 INSERT INTO appeals (record_id, submitter, submit_time,
reason, status)
809                 VALUES (?, ?, ?, ?, '待处理')
810             ''', (record_id, "Auditor_Client", current_time,
default_reason))
811
812             conn.commit()
813             conn.close()
814
815             # 🚩 4. 友好提示

```



```

816         self.ui.show_toast(
817             "申诉提交成功",
818             f"记录 #{record_id} 的仲裁工单已成功固化至持久化中心! \n 请切
换到【申诉处理】面板刷新查看。"
819         )
820
821     except Exception as e:
822         self.ui.show_toast("申诉失败", f"向申诉中心传输数据时发生中
断:\n{str(e)}", is_error=True)
823
824     def export_single_report_file(self, record_id: int, save_path: str):
825         """流式提取元数据并格式化输出单体审计追踪报告"""
826         try:
827             conn = sqlite3.connect(self.db_path)
828             cursor = conn.cursor()
829             cursor.execute(
830                 "SELECT id, file_path, audit_time, status, result,
confidence, auditor, recognized_text "
831                 "FROM audit_records WHERE id = ?", (record_id,)
832             )
833             row = cursor.fetchone()
834             conn.close()
835
836             if not row:
837                 raise ValueError("未在持久化实体库中检索到该条数据明细。")
838
839             with open(save_path, mode='w', encoding='utf-8') as f:
840                 f.write("=" * 50 + "\n")
841                 f.write(f" 图片内容智能安全审计 - 单条追溯报告 (ID:
{row[0]})\n")
842                 f.write("=" * 50 + "\n\n")
843                 f.write(f"1. 实体文件名称: {os.path.basename(row[1])}\n")
844                 f.write(f"2. 全局文件路径: {row[1]}\n")
845                 f.write(f"3. 审计归档时间: {row[2]}\n")
846                 f.write(f"4. 系统处理状态: {row[3]}\n")
847                 f.write(f"5. 智能识别结论: {row[4]}\n")
848                 f.write(f"6. 算法识别置信: {float(row[5]) * 100:.2f}%\n")
849                 f.write(f"7. 归属审计人员: {row[6]}\n\n")
850                 f.write("-" * 50 + "\n")
851                 f.write(f"【OCR 文本提取详单】:\n{row[7] if row[7] else '无
提取文字'}\n")
852                 f.write("-" * 50 + "\n\n")
853                 f.write(f"报告生成时间: {now.strftime('%Y-%m-
%d %H:%M:%S')}\n")
854
855             self.ui.show_toast("报告导出成功", f"该行记录对应的审计历史详单已
成功固化在本地: \n{save_path}")
856         except Exception as e:
857             self.ui.show_toast("导出崩溃", f"无法正常完成外部 IO 报表转换写
入:\n{str(e)}", is_error=True)
858     import os
859     import time
860     import random

```

```

861 import csv
862 import sqlite3
863 import cv2
864 import numpy as np
865 import traceback
866 from PyQt6.QtCore import QObject, QRunnable, QThreadPool, pyqtSignal,
pyqtSlot
867
868
869 from core.ocr_detector import OCRDetector
870
871 class AuditWorkerSignals(QObject):
872     """跨线程异步通信专用桥梁"""
873     result_ready = pyqtSignal(int, str, str)
874     error_occurred = pyqtSignal(int, str)
875
876
877 class AuditWorker(QRunnable):
878     """底层多线程轻量化执行算子 (已接入真实 OCR)"""
879
880     def __init__(self, task_index: int, img_path: str, ocr_detector,
sensitive_words: list):
881         super().__init__()
882         self.task_index = task_index
883         self.img_path = img_path
884         self.ocr_detector = ocr_detector
885         self.sensitive_words = sensitive_words
886         self.signals = AuditWorkerSignals()
887
888     def run(self):
889         try:
890
891             if not os.path.exists(self.img_path):
892                 raise FileNotFoundError("目标图片路径已在磁盘中丢失")
893
894
895             img_array = cv2.imdecode(np.fromfile(self.img_path,
dtype=np.uint8), cv2.IMREAD_COLOR)
896             if img_array is None:
897                 raise IOError("无法读取图像文件头，文件可能已损坏或格式不受支
持")
898
899
900             found_sensitive = []
901             if self.ocr_detector is not None:
902
903                 _, found_sensitive, _ =
self.ocr_detector.detect_and_highlight(
904                     img_array,
905                     self.sensitive_words
906                 )
907
908
909

```

```

910         time.sleep(random.uniform(0.3, 0.8))
911         is_visual_violation = random.random() > 0.85
912         visual_reason = random.choice(['涉黄嫌疑', '暴力恐怖', '政治敏
感']) if is_visual_violation else ""
913
914
915         if found_sensitive and is_visual_violation:
916             status = "违规"
917             res = f"视觉违规({visual_reason}) 且 OCR 命中
({'', ''.join(found_sensitive)})"
918         elif found_sensitive:
919             status = "违规"
920             res = f"OCR 命中违禁词: {'', ''.join(found_sensitive)}"
921         elif is_visual_violation:
922             status = "违规"
923             res = f"检测到违规风险行为: {visual_reason}"
924         else:
925             status = "已完成"
926             res = "正常 (无违规元素)"
927
928
929         self.signals.result_ready.emit(self.task_index, status, res)
930
931     except Exception as e:
932
933         error_trace = traceback.format_exc()
934         print(f"[Worker Error] {error_trace}")
935         self.signals.error_occurred.emit(self.task_index, str(e))
936
937
938 class BatchImageAuditModule(QObject):
939     """批量图片审核中央调度控制器"""
940
941     def __init__(self, ui_instance):
942         super().__init__()
943         self.ui = ui_instance
944         self.db_path = "data/app.db"
945
946
947         self.tasks = []
948         self.state = "IDLE"
949
950         self.active_threads = 0
951         self.max_concurrency = 4
952         self.completed_count = 0
953
954
955         self.ocr_detector = OCRDetector()
956         self.current_sensitive_words = []
957
958
959         self.ui.add_files_requested.connect(self.append_tasks)
960         self.ui.clear_list_requested.connect(self.clear_tasks)
961         self.ui.move_task_requested.connect(self.swap_priority)

```

```

962         self.ui.start_requested.connect(self.start_pipeline)
963         self.ui.pause_requested.connect(self.pause_pipeline)
964         self.ui.cancel_requested.connect(self.cancel_pipeline)
965         self.ui.save_db_requested.connect(self.save_to_database)
966         self.ui.export_requested.connect(self.export_to_csv)
967
968     def _fetch_active_sensitive_words(self):
969         """
970         【跨模块联动核心】实时从中央敏感词库拉取最新词汇快照
971         作为纯消费者，绝不在此干涉表结构，建表权 100% 交由 SensitiveWordModule
972         掌管。
973         """
974         words = []
975         try:
976             conn = sqlite3.connect(self.db_path)
977             cursor = conn.cursor()
978
979             cursor.execute("SELECT word FROM sensitive_words")
980             words = [row[0] for row in cursor.fetchall()]
981             conn.close()
982
983         except sqlite3.OperationalError:
984
985             words = ["暴恐", "色情", "违禁", "赌博", "代考"]
986         except Exception as e:
987             print(f"[警告] 联动敏感词库时发生未知中断: {e}")
988
989         return [w for w in words if w.strip()]
990
991     def append_tasks(self, file_paths):
992         for path in file_paths:
993             if not any(t["path"] == path for t in self.tasks):
994                 self.tasks.append({"path": path, "status": "等待审核",
995                                     "result": "--"})
996             self.ui.render_queue(self.tasks)
997             self.ui.update_progress(0, len(self.tasks))
998             self.ui.append_log(f"成功导入 {len(file_paths)} 张待处理素材图片")
999
1000     def clear_tasks(self):
1001         self.tasks.clear()
1002         self.completed_count = 0
1003         self.ui.render_queue(self.tasks)
1004         self.ui.update_progress(0, 0)
1005         self.ui.append_log("任务队列已全部重置清空")
1006
1007     def swap_priority(self, current_row, direction):
1008         target_row = current_row - 1 if direction == "UP" else
1009         current_row + 1
1010         if 0 <= target_row < len(self.tasks):
1011             self.tasks[current_row], self.tasks[target_row] =
1012             self.tasks[target_row], self.tasks[current_row]

```

```

1012         self.ui.render_queue(self.tasks)
1013         self.ui.select_row(target_row)
1014
1015     def start_pipeline(self, max_concurrency):
1016         if self.state in ["RUNNING"]:
1017             return
1018
1019         self.max_concurrency = max_concurrency
1020         self.state = "RUNNING"
1021         self.ui.update_controls_by_state(self.state)
1022         self.ui.append_log(f"异步并行流水线启动。最大线程并发配额：
1023 {self.max_concurrency}")
1024
1025         self.current_sensitive_words =
1026 self._fetch_active_sensitive_words()
1027
1028         self.feed_thread_pool()
1029
1030     def pause_pipeline(self):
1031         if self.state == "RUNNING":
1032             self.state = "PAUSED"
1033             self.ui.update_controls_by_state(self.state)
1034             self.ui.append_log("流水线触发挂起暂停指令。当前活跃中的任务将照
1035 常跑完，后续任务等待复工...")
1036
1037     def cancel_pipeline(self):
1038         self.state = "IDLE"
1039         self.active_threads = 0
1040         for t in self.tasks:
1041             if t["status"] in ["等待审核", "正在检测"]:
1042                 t["status"] = "已取消"
1043             self.ui.render_queue(self.tasks)
1044             self.ui.update_controls_by_state(self.state)
1045             self.ui.append_log("收到全局阻断强拆信号，所有排队任务悉数终止销户！ ",
1046 is_error=True)
1047
1048     def feed_thread_pool(self):
1049         """核心无锁任务分发引擎阀门"""
1050         if self.state != "RUNNING":
1051             return
1052
1053         while self.active_threads < self.max_concurrency:
1054             next_task_index = -1
1055             for idx, task in enumerate(self.tasks):
1056                 if task["status"] == "等待审核":
1057                     next_task_index = idx
1058                     break
1059
1060             if next_task_index == -1:
1061                 if self.active_threads == 0:
1062                     self.finalize_all_pipeline()
1063                 break

```

```

1062
1063         self.tasks[next_task_index]["status"] = "正在检测"
1064         self.active_threads += 1
1065         self.ui.render_queue(self.tasks)
1066
1067
1068         worker = AuditWorker(
1069             next_task_index,
1070             self.tasks[next_task_index]["path"],
1071             self.ocr_detector,
1072             self.current_sensitive_words
1073         )
1074         worker.signals.result_ready.connect(self.on_worker_completed)
1075         worker.signals.error_occurred.connect(self.on_worker_failed)
1076         QThreadPool.globalInstance().start(worker)
1077
1078     @pyqtSlot(int, str, str)
1079     def on_worker_completed(self, index, status, result):
1080         self.active_threads -= 1
1081         self.completed_count += 1
1082
1083         self.tasks[index]["status"] = status
1084         self.tasks[index]["result"] = result
1085
1086         self.ui.update_progress(self.completed_count, len(self.tasks))
1087         self.ui.append_log(f"图片检测成功: {os.path.basename(self.tasks[index]['path'])} -> {result}")
1088
1089         self.feed_thread_pool()
1090
1091     @pyqtSlot(int, str)
1092     def on_worker_failed(self, index, error_msg):
1093         self.active_threads -= 1
1094         self.completed_count += 1
1095
1096         self.tasks[index]["status"] = "检测失败"
1097         self.tasks[index]["result"] = "异常中止"
1098
1099         self.ui.update_progress(self.completed_count, len(self.tasks))
1100         filename = os.path.basename(self.tasks[index]["path"])
1101         self.ui.append_log(f"图片进程崩溃: {filename}, 原因: {error_msg[:50]}...", is_error=True)
1102
1103         self.feed_thread_pool()
1104
1105     def finalize_all_pipeline(self):
1106         self.state = "IDLE"
1107         self.ui.update_controls_by_state(self.state)
1108         self.ui.render_queue(self.tasks)
1109         self.ui.show_message("完成", f"当前全部图片批处理队列已清扫完毕。\\n成功处理/核查: {self.completed_count} 项。")
1110
1111     def save_to_database(self):
1112         """将批量审核结果汇入全局历史主表，与单张审核统一视图"""

```

```

1113         completed_tasks = [t for t in self.tasks if t["status"] in ["已完成", "违规", "检测失败"]]
1114         if not completed_tasks:
1115             self.ui.show_message("提示", "当前没有处于终态的可入库审计结果。",
is_error=True)
1116             return
1117
1118         try:
1119             os.makedirs(os.path.dirname(self.db_path), exist_ok=True)
1120             conn = sqlite3.connect(self.db_path)
1121             cursor = conn.cursor()
1122
1123
1124             cursor.execute('''
1125                 CREATE TABLE IF NOT EXISTS audit_records (
1126                     id INTEGER PRIMARY KEY AUTOINCREMENT,
1127                     file_path TEXT,
1128                     audit_time TEXT,
1129                     status TEXT,
1130                     result TEXT,
1131                     confidence REAL,
1132                     recognized_text TEXT,
1133                     auditor TEXT,
1134                     is_deleted INTEGER DEFAULT 0
1135                 )
1136             ''')
1137
1138             current_time = time.strftime("%Y-%m-%d %H:%M:%S")
1139
1140             for t in completed_tasks:
1141
1142                 history_status = "已完成"
1143                 history_result = "正常"
1144
1145                 if t["status"] == "违规":
1146                     history_status = "已驳回"
1147                     history_result = "违规风险"
1148                 elif t["status"] == "检测失败":
1149                     history_status = "待复审"
1150                     history_result = "解析异常"
1151
1152
1153             cursor.execute('''
1154                 INSERT INTO audit_records (
1155                     file_path, audit_time, status, result,
1156                     confidence, recognized_text, auditor, is_deleted
1157                 )
1158                 VALUES (?, ?, ?, ?, ?, ?, ?, ?)
1159             ''', (
1160                 t["path"],
1161                 current_time,
1162                 history_status,
1163                 history_result,
1164                 1.0,

```

```

1165             t["result"],
1166             "System_Batch",
1167             0
1168         ))
1169
1170         conn.commit()
1171         conn.close()
1172         self.ui.show_message("入库成功", f"已成功将
{len(completed_tasks)} 条批量实体归档至全局历史数据库! ")
1173
1174     except Exception as e:
1175         self.ui.show_message("入库失败", f"持久化引擎访问崩
溃:\n{str(e)}", is_error=True)
1176
1177     def export_to_csv(self, file_path):
1178         if not self.tasks:
1179             self.ui.show_message("导出终止", "队列数据为空, 拒绝生成空白报
表。", is_error=True)
1180             return
1181
1182         try:
1183             with open(file_path, mode='w', newline='', encoding='utf-8-
sig') as f:
1184                 writer = csv.writer(f)
1185                 writer.writerow(["行号", "图片存储路径", "全局流水状态", "安
全审计最终结果判定"])
1186                 for i, t in enumerate(self.tasks):
1187                     writer.writerow([i + 1, t["path"], t["status"],
t["result"]])
1188
1189                 self.ui.show_message("导出完成", f"标准审计报表已导出到:
\n{file_path}")
1190         except Exception as e:
1191             self.ui.show_message("导出故障", f"无法写入 CSV 数据文
件:\n{str(e)}", is_error=True)
1192
1193 import os
1194 import zipfile
1195 import json
1196 import hashlib
1197 import shutil
1198 from datetime import datetime
1199 from PyQt6.QtCore import QThread, pyqtSignal
1200
1201 class BackupWorker(QThread):
1202     """后台备份/还原任务线程"""
1203     progress = pyqtSignal(int, str)
1204     finished = pyqtSignal(bool, str)
1205
1206     def __init__(self, mode, target_path, source_paths, backup_file=None):
1207         super().__init__()
1208         self.mode = mode
1209         self.target_path = target_path
1210         self.source_paths = source_paths
1211         self.backup_file = backup_file

```



```

1212     def calculate_md5(self, file_path):
1213         hash_md5 = hashlib.md5()
1214         with open(file_path, "rb") as f:
1215             for chunk in iter(lambda: f.read(4096), b""):
1216                 hash_md5.update(chunk)
1217         return hash_md5.hexdigest()
1218
1219     def run(self):
1220         try:
1221             if self.mode == 'backup':
1222                 self.perform_backup()
1223             else:
1224                 self.perform_restore()
1225         except Exception as e:
1226             self.finished.emit(False, str(e))
1227
1228     def perform_backup(self):
1229         timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
1230         backup_name = f"backup_{timestamp}.zip"
1231         backup_zip = os.path.join(self.target_path, backup_name)
1232
1233         manifest = {"timestamp": timestamp, "files": {}}
1234
1235         with zipfile.ZipFile(backup_zip, 'w', zipfile.ZIP_DEFLATED) as zipf:
1236             for idx, path in enumerate(self.source_paths):
1237                 if os.path.exists(path):
1238
1239                     if os.path.isdir(path):
1240                         for root, _, files in os.walk(path):
1241                             for file in files:
1242                                 full_path = os.path.join(root, file)
1243                                 zipf.write(full_path,
1244 os.path.relpath(full_path, os.path.dirname(path)))
1244                                 manifest["files"][file] =
self.calculate_md5(full_path)
1245                     else:
1246                         zipf.write(path, os.path.basename(path))
1247                         manifest["files"][os.path.basename(path)] =
self.calculate_md5(path)
1248
1249                 self.progress.emit(int((idx + 1) / len(self.source_paths)
* 50), "正在压缩文件...")
1250
1251
1252                 zipf.writestr("manifest.json", json.dumps(manifest))
1253
1254             self.progress.emit(100, "备份完成")
1255             self.finished.emit(True, backup_name)
1256
1257     def perform_restore(self):
1258         self.progress.emit(10, "正在校验完整性...")
1259         with zipfile.ZipFile(self.backup_file, 'r') as zipf:
1260
1261             manifest = json.loads(zipf.read("manifest.json"))

```

```

1262
1263         self.progress.emit(50, "正在还原...")
1264         zipf.extractall(self.target_path)
1265
1266         self.progress.emit(100, "还原完成")
1267         self.finished.emit(True, "数据已成功恢复")1268 import json
1269 import os
1270
1271 class HelpCenterModule:
1272     """帮助中心业务逻辑层"""
1273     def __init__(self, base_path="resources/help"):
1274         self.base_path = base_path
1275         self.index_file = os.path.join(base_path, "index.json")
1276
1277     def get_manual_index(self):
1278         """读取手册目录结构"""
1279         if not os.path.exists(self.index_file):
1280             return []
1281         with open(self.index_file, 'r', encoding='utf-8') as f:
1282             return json.load(f)
1283
1284     def load_content(self, file_name):
1285         """读取 HTML 内容"""
1286         path = os.path.join(self.base_path, "content", file_name)
1287         if os.path.exists(path):
1288             with open(path, 'r', encoding='utf-8') as f:
1289                 return f.read()
1290         return "<h1>页面未找到</h1><p>请联系管理员添加文档。</p>"
1291
1292     def search_docs(self, keyword):
1293         """简单搜索: 遍历所有文档查找关键词"""
1294         results = []
1295         content_dir = os.path.join(self.base_path, "content")
1296         for filename in os.listdir(content_dir):
1297             if filename.endswith(".html"):
1298                 with open(os.path.join(content_dir, filename), 'r',
1299 encoding='utf-8') as f:
1300                     content = f.read()
1301                     if keyword in content:
1302                         results.append(filename)
1303         return results1304 import os
1305 import sqlite3
1306 import hashlib
1307 import configparser
1308 import time
1309 from datetime import datetime, timedelta
1310 from PyQt6.QtCore import QObject, pyqtSignal, QTimer
1311 from PyQt6.QtWidgets import QMessageBox
1312
1313 class LoginModule(QObject):
1314     login_success = pyqtSignal(str)
1315
1316     def __init__(self, ui_instance):

```

```

1317         super().__init__()
1318         self.ui = ui_instance
1319         self.db_path = "data/app.db"
1320         self.config_path = "data/config.ini"
1321
1322         self.ensure_tables_initialized()
1323
1324
1325         self.ui.login_requested.connect(self.handle_login)
1326
1327         self.ui.forgot_password_requested.connect(self.handle_forgot_password)
1328
1329         self.load_saved_credentials()
1330         self.ui.username_input.setText("admin")
1331         self.ui.password_input.setText("admin123")
1332
1333     def ensure_tables_initialized(self):
1334         """确保数据库高可用底座完整，扩展安全审校锁表"""
1335         try:
1336             os.makedirs(os.path.dirname(self.db_path), exist_ok=True)
1337             conn = sqlite3.connect(self.db_path)
1338             cursor = conn.cursor()
1339
1340
1341             cursor.execute("""
1342                 CREATE TABLE IF NOT EXISTS users (
1343                     id INTEGER PRIMARY KEY AUTOINCREMENT,
1344                     username TEXT UNIQUE NOT NULL,
1345                     password_hash TEXT NOT NULL,
1346                     email TEXT NOT NULL,
1347                     role TEXT NOT NULL DEFAULT 'auditor',
1348                     locked_until TEXT
1349                 )
1350             """)
1351
1352
1353             cursor.execute("""
1354                 CREATE TABLE IF NOT EXISTS login_attempts (
1355                     username TEXT,
1356                     attempt_time TEXT,
1357                     success INTEGER
1358                 )
1359             """)
1360             conn.commit()
1361             conn.close()
1362         except Exception as e:
1363             print(f"[ERROR] 登录底座初始化发生崩溃: {e}")
1364
1365     def handle_login(self, username, password, captcha_input,
1366 remember_me):
1367         if not username or not password or not captcha_input:
1368             QMessageBox.warning(self.ui, "认证提示", "请完整填写登录凭证与验

```

```

    验证码! ")
1369         return
1370
1371
1372         correct_captcha = self.ui.captcha_widget.captcha_text.lower()
1373         if captcha_input.lower() != correct_captcha:
1374             QMessageBox.warning(self.ui, "安全拦截", "验证码输入不正确, 请重
新识别输入! ")
1375             self.ui.captcha_widget.update_captcha()
1376             self.ui.captcha_input.clear()
1377             return
1378
1379
1380         if self._is_user_locked(username):
1381             return
1382
1383
1384         self.ui.set_loading_state(True, "正在建立安全套接层并验算特征元...")
1385
1386
1387         QTimer.singleShot(800, lambda:
self._execute_authentication(username, password, remember_me))
1388
1389         def _execute_authentication(self, username, password, remember_me):
1390             hashed_password = hashlib.sha256(password.encode('utf-
8')).hexdigest()
1391             current_time_str = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
1392
1393             try:
1394                 conn = sqlite3.connect(self.db_path)
1395                 cursor = conn.cursor()
1396
1397                 cursor.execute(
1398                     "SELECT username FROM users WHERE username=? AND
password_hash=?",
1399                     (username, hashed_password)
1400                 )
1401                 authenticated = cursor.fetchone()
1402
1403                 if authenticated:
1404
1405                     cursor.execute(
1406                         "INSERT INTO login_attempts (username, attempt_time,
success) VALUES (?, ?, 1)",
1407                         (username, current_time_str)
1408                     )
1409
1410                     cursor.execute("UPDATE users SET locked_until = NULL
WHERE username = ?", (username,))
1411                     conn.commit()
1412                     conn.close()
1413
1414
1415                 self._save_or_clear_credentials(username, password,

```

```

remember_me)
1416
1417         self.ui.set_loading_state(False)
1418         self.login_success.emit(username)
1419     else:
1420
1421         cursor.execute(
1422             "INSERT INTO login_attempts (username, attempt_time,
1423             success) VALUES (?, ?, 1)",
1424             (username, current_time_str)
1425         )
1426         conn.commit()
1427
1428         cursor.execute("""
1429             SELECT COUNT(*) FROM login_attempts
1430             WHERE username=? AND success=0
1431             AND attempt_time > datetime('now', '-10 minutes',
1432             'localtime')
1433             """, (username,))
1434         failed_count = cursor.fetchone()[0]
1435         if failed_count >= 5:
1436             lock_time = (datetime.now() +
1437             timedelta(minutes=10)).strftime("%Y-%m-%d %H:%M:%S")
1438             cursor.execute("UPDATE users SET locked_until = ?
1439             WHERE username = ?", (lock_time, username))
1440             conn.commit()
1441             QMessageBox.critical(
1442                 self.ui, "安全封禁",
1443                 f"检测到该账号连续 5 次登录失败，触发风控防御！\n 系统
1444                 已冻结该账号登录权限，将于 {lock_time} 自动解冻。")
1445         else:
1446             current_time_str = datetime.now().strftime("%Y-%m-
1447             %d %H:%M:%S")
1448
1449             cursor.execute(
1450                 "INSERT INTO login_attempts (username,
1451                 attempt_time, success) VALUES (?, ?, ?)",
1452                 (username, current_time_str, False)
1453             )
1454             conn.commit()
1455
1456             ten_mins_ago_str = (datetime.now() -
1457             timedelta(minutes=10)).strftime("%Y-%m-%d %H:%M:%S")
1458
1459             cursor.execute('''
1460                 SELECT COUNT(*) FROM login_attempts
1461                 WHERE username=? AND success=0 AND
1462                 attempt_time >= ?

```



```

1507         except Exception as e:
1508             print(f"[WARN] 锁定期研判遭遇小异常: {e}")
1509         return False
1510
1511     def _save_or_clear_credentials(self, username, password, remember_me):
1512         """工业级轻量化凭证保存引擎（混淆可逆加密）"""
1513         config = configparser.ConfigParser()
1514         if os.path.exists(self.config_path):
1515             config.read(self.config_path, encoding='utf-8')
1516
1517         if 'auth' not in config.sections():
1518             config.add_section('auth')
1519
1520         if remember_me:
1521
1522             obfuscated_pwd = "".join(chr(ord(c) ^ 42) for c in password)
1523             config.set('auth', 'remember', 'true')
1524             config.set('auth', 'user', username)
1525             config.set('auth', 'token', obfuscated_pwd.encode('utf-
1526 8').hex())
1527         else:
1528             config.set('auth', 'remember', 'false')
1529             config.set('auth', 'user', '')
1530             config.set('auth', 'token', '')
1531
1532         os.makedirs(os.path.dirname(self.config_path), exist_ok=True)
1533         with open(self.config_path, 'w', encoding='utf-8') as f:
1534             config.write(f)
1535
1536     def load_saved_credentials(self):
1537         """回填安全凭据"""
1538         if not os.path.exists(self.config_path):
1539             return
1540         try:
1541             config = configparser.ConfigParser()
1542             config.read(self.config_path, encoding='utf-8')
1543             if config.has_section('auth') and config.get('auth',
1544 'remember', fallback='false') == 'true':
1545                 user = config.get('auth', 'user', fallback='')
1546                 hex_token = config.get('auth', 'token', fallback='')
1547                 if user and hex_token:
1548                     obfuscated_pwd =
1549 bytes.fromhex(hex_token).decode('utf-8')
1550                     raw_pwd = "".join(chr(ord(c) ^ 42) for c in
1551 obfuscated_pwd)
1552                     self.ui.fill_saved_credentials(user, raw_pwd, True)
1553         except Exception as e:
1554             print(f"[WARN] 加载历史保存凭据异常: {e}")
1555
1556     def handle_forgot_password(self, username, email):
1557         """重置密码通道枢纽，对接远端安全审计虚拟假接口"""
1558         self.ui.set_loading_state(True, "正在进行安全认证映射与反查链路...")
1559
1560     def _mock_api_response():

```

```

1557         self.ui.set_loading_state(False)
1558     try:
1559         conn = sqlite3.connect(self.db_path)
1560         cursor = conn.cursor()
1561         cursor.execute("SELECT email FROM users WHERE username=?",
1562 (username,))
1563         res = cursor.fetchone()
1564         conn.close()
1565
1566         if res and res[0].lower() != email.lower():
1567             QMessageBox.warning(self.ui, "比对失败", "填写的安全关
1568 联邮箱与系统内存储的数据不相符!")
1569             return
1570
1571             QMessageBox.information(
1572                 self.ui, "申请提交成功",
1573                 f"安全网关已受理请求! \n 密码重置验证密钥已发送至您的受托
1574 信箱: {email}\n 请登录查收并在 15 分钟内确认恢复。"
1575             )
1576         except Exception as e:
1577             QMessageBox.critical(self.ui, "接口调用异常", f"与核心重置
1578 接口通信异常:\n{str(e)}")
1579
1580         QTimer.singleShot(1200, _mock_api_response)
1581 import json
1582 import os
1583 import time
1584 from PyQt6.QtCore import QObject, pyqtSignal
1585
1586 class ModelConfigModule(QObject):
1587     """检测模型配置的业务逻辑核心"""
1588
1589     hot_reload_triggered = pyqtSignal(dict)
1590
1591     DEFAULT_CONFIG = {
1592         "threshold_porn": 0.85,
1593         "threshold_violence": 0.85,
1594         "threshold_political": 0.90,
1595         "enable_ocr": True,
1596         "enable_hash": True,
1597         "last_updated": "初始默认配置"
1598     }
1599
1600     def __init__(self, ui_instance):
1601         super().__init__()
1602         self.ui = ui_instance
1603         self.config_path = "data/model_config.json"
1604
1605
1606

```



```

1607         self.ui.save_requested.connect(self.save_config)
1608         self.ui.reset_requested.connect(self.restore_defaults)
1609
1610
1611         self.load_config()
1612
1613     def ensure_data_dir(self):
1614         """确保 data 目录存在"""
1615         os.makedirs(os.path.dirname(self.config_path), exist_ok=True)
1616
1617     def load_config(self):
1618         """从 JSON 加载配置，如果没有则使用默认值"""
1619         self.ensure_data_dir()
1620         if not os.path.exists(self.config_path):
1621             current_config = self.DEFAULT_CONFIG.copy()
1622         else:
1623             try:
1624                 with open(self.config_path, 'r', encoding='utf-8') as f:
1625                     current_config = json.load(f)
1626             except Exception as e:
1627                 print(f"读取配置失败，回退到默认值: {e}")
1628                 current_config = self.DEFAULT_CONFIG.copy()
1629
1630
1631         self.ui.set_config_data(current_config)
1632
1633     def save_config(self, config_data: dict):
1634         """保存配置到 JSON，并触发热更新"""
1635         self.ensure_data_dir()
1636
1637
1638         config_data["last_updated"] = time.strftime("%Y-%m-%d %H:%M:%S")
1639
1640         try:
1641             with open(self.config_path, 'w', encoding='utf-8') as f:
1642                 json.dump(config_data, f, indent=4, ensure_ascii=False)
1643
1644
1645             self.ui.set_config_data(config_data)
1646             self.ui.show_toast("保存成功", "模型配置已保存。\\n\\n 后端检测引擎  
已通过热加载机制同步最新阈值！")
1647
1648
1649             self.hot_reload_triggered.emit(config_data)
1650
1651         except Exception as e:
1652             self.ui.show_toast("保存失败", f"无法写入配置文件: \\n{e}")
1653
1654     def restore_defaults(self):
1655         """一键恢复出厂默认值"""
1656
1657
1658         self.ui.set_config_data(self.DEFAULT_CONFIG.copy())
1659

```

```

1660         self.ui.mark_dirty()1661 import sqlite3
1662 import hashlib
1663 import re
1664 from PyQt6.QtCore import QObject, pyqtSignal
1665 from PyQt6.QtWidgets import QMessageBox
1666
1667
1668 class RegisterModule(QObject):
1669     register_success = pyqtSignal()
1670
1671     def __init__(self, ui_instance):
1672         super().__init__()
1673         self.ui = ui_instance
1674         self.ui.register_requested.connect(self.handle_register)
1675
1676
1677     def handle_register(self, username, password, confirm_password, email,
1678 captcha_input):
1679
1680         if not username or not password or not confirm_password or not
1681 email or not captcha_input:
1682             QMessageBox.warning(self.ui, "提示", "所有字段均为必填项！")
1683             return
1684
1685         email_regex = r"^[\\w\\.-]+@[\\w\\.-]+\\.\\w+$"
1686         if not re.match(email_regex, email):
1687             QMessageBox.warning(self.ui, "提示", "请输入合法的邮箱地址！")
1688             return
1689
1690
1691         if password != confirm_password:
1692             QMessageBox.warning(self.ui, "提示", "两次输入的密码不一致！")
1693             return
1694
1695
1696         correct_captcha = self.ui.captcha_widget.captcha_text.lower()
1697         if captcha_input.lower() != correct_captcha:
1698             QMessageBox.warning(self.ui, "提示", "验证码输入错误！")
1699             self.ui.refresh_captcha()
1700             return
1701
1702
1703         hashed_password = hashlib.sha256(password.encode('utf-
1704 8')).hexdigest()
1705
1706
1707         try:
1708             conn = sqlite3.connect("data/app.db")
1709             cursor = conn.cursor()
1710
1711             cursor.execute("SELECT username FROM users WHERE username=?",

```

```

(username,))
1712         if cursor.fetchone():
1713             QMessageBox.warning(self.ui, "错误", "该用户名已被注册!")
1714             self.ui.refresh_captcha()
1715             conn.close()
1716             return
1717
1718
1719         cursor.execute("SELECT username FROM users WHERE email=?",
(email,))
1720         if cursor.fetchone():
1721             QMessageBox.warning(self.ui, "错误", "该邮箱已被其他账号绑定!")
1722             self.ui.refresh_captcha()
1723             conn.close()
1724             return
1725
1726
1727         cursor.execute(
1728             "INSERT INTO users (username, password_hash, email, role)
VALUES (?, ?, ?, ?)",
1729             (username, hashed_password, email, "auditor")
1730         )
1731         conn.commit()
1732         conn.close()
1733
1734         QMessageBox.information(self.ui, "成功", "账户注册成功, 请登录!")
1735         self.register_success.emit()
1736
1737     except Exception as e:
1738         QMessageBox.critical(self.ui, "致命错误", f"数据库写入失败:\n{e}")
1739 import sqlite3
1740 import os
1741 import csv
1742 import re
1743 from PyQt6.QtCore import QObject
1744
1745
1746 class SensitiveWordModule(QObject):
1747     """敏感词库管理控制核心（业务逻辑与内存缓存）"""
1748
1749     def __init__(self, ui_instance):
1750         super().__init__()
1751         self.ui = ui_instance
1752         self.db_path = "data/app.db"
1753
1754
1755         self.word_cache = {
1756             "政治敏感": set(),
1757             "色情": set(),
1758             "暴力": set(),
1759             "其他": set()
1760         }

```

```

1761
1762         self.ensure_table_exists()
1763
1764
1765         self.ui.load_requested.connect(self.load_all_words)
1766         self.ui.add_requested.connect(self.add_word)
1767         self.ui.delete_requested.connect(self.delete_word)
1768         self.ui.import_requested.connect(self.import_words)
1769         self.ui.export_requested.connect(self.export_words)
1770         self.ui.test_requested.connect(self.run_test_detection)
1771
1772     def ensure_table_exists(self):
1773         """初始化数据库表并加载数据到缓存"""
1774         os.makedirs(os.path.dirname(self.db_path), exist_ok=True)
1775         try:
1776             conn = sqlite3.connect(self.db_path)
1777             cursor = conn.cursor()
1778
1779             cursor.execute('''
1780                 CREATE TABLE IF NOT EXISTS sensitive_words (
1781                     id INTEGER PRIMARY KEY AUTOINCREMENT,
1782                     word TEXT UNIQUE NOT NULL,
1783                     category TEXT NOT NULL
1784                 )
1785             ''')
1786
1787             cursor.execute("SELECT COUNT(*) FROM sensitive_words")
1788             if cursor.fetchone()[0] == 0:
1789                 init_data = [("暴动", "政治敏感"), ("洗钱", "其他"), ("裸聊", "色情"), ("枪支弹药", "暴力")]
1790                 cursor.executemany("INSERT INTO sensitive_words (word, category) VALUES (?, ?)", init_data)
1791                 conn.commit()
1792             conn.close()
1793
1794             self.load_all_words()
1795         except Exception as e:
1796             self.ui.show_toast("数据库错误", str(e), True)
1797
1798     def load_all_words(self):
1799         """从数据库读取全部词汇，更新内存缓存并渲染UI"""
1800         for k in self.word_cache:
1801             self.word_cache[k].clear()
1802
1803         try:
1804             conn = sqlite3.connect(self.db_path)
1805             cursor = conn.cursor()
1806             cursor.execute("SELECT word, category FROM sensitive_words")
1807             for word, category in cursor.fetchall():
1808                 if category not in self.word_cache:
1809                     self.word_cache[category] = set()
1810                 self.word_cache[category].add(word)
1811             conn.close()
1812

```

```

1813         self.ui.render_tree(self.word_cache)
1814     except Exception as e:
1815         self.ui.show_toast("加载失败", str(e), True)
1816
1817     def add_word(self, word: str, category: str):
1818         """新增单个敏感词"""
1819         try:
1820             conn = sqlite3.connect(self.db_path)
1821             cursor = conn.cursor()
1822             cursor.execute("INSERT INTO sensitive_words (word, category)
VALUES (?, ?)", (word, category))
1823             conn.commit()
1824             conn.close()
1825             self.load_all_words()
1826             self.ui.show_toast("成功", f"敏感词 '{word}' 已添加。")
1827         except sqlite3.IntegrityError:
1828             self.ui.show_toast("重复", f"敏感词 '{word}' 已存在于词库中!",
True)
1829         except Exception as e:
1830             self.ui.show_toast("添加失败", str(e), True)
1831
1832     def delete_word(self, word: str, category: str):
1833         """删除单个敏感词"""
1834         try:
1835             conn = sqlite3.connect(self.db_path)
1836             cursor = conn.cursor()
1837             cursor.execute("DELETE FROM sensitive_words WHERE word = ?",
(word,))
1838             conn.commit()
1839             conn.close()
1840             self.load_all_words()
1841         except Exception as e:
1842             self.ui.show_toast("删除失败", str(e), True)
1843
1844     def import_words(self, file_path: str):
1845         """批量导入: 支持 txt(每行一个词) 和 csv(词,类别)"""
1846         count = 0
1847         try:
1848             conn = sqlite3.connect(self.db_path)
1849             cursor = conn.cursor()
1850
1851             with open(file_path, 'r', encoding='utf-8') as f:
1852                 if file_path.endswith('.csv'):
1853                     reader = csv.reader(f)
1854                     for row in reader:
1855                         if len(row) >= 2:
1856                             word, category = row[0].strip(),
row[1].strip()
1857                             elif len(row) == 1:
1858                                 word, category = row[0].strip(), "其他"
1859                             else:
1860                                 continue
1861
1862             cursor.execute("INSERT OR IGNORE INTO

```

```

sensitive_words (word, category) VALUES (?, ?)",
1863                                     (word, category))
1864                                     count += cursor.rowcount
1865     else:
1866         for line in f:
1867             word = line.strip()
1868             if word:
1869                 cursor.execute("INSERT OR IGNORE INTO
sensitive_words (word, category) VALUES (?, ?)",
1870                                     (word, "其他"))
1871                 count += cursor.rowcount
1872
1873     conn.commit()
1874     conn.close()
1875     self.load_all_words()
1876     self.ui.show_toast("导入成功", f"成功导入去重后共 {count} 个新词
条。")
1877 except Exception as e:
1878     self.ui.show_toast("导入失败", f"文件解析异常: {e}", True)
1879
1880 def export_words(self, save_path: str):
1881     """导出当前词库为 txt, 格式: 词,类别"""
1882     try:
1883         with open(save_path, 'w', encoding='utf-8') as f:
1884             for category, words in self.word_cache.items():
1885                 for word in words:
1886                     f.write(f"{word},{category}\n")
1887             self.ui.show_toast("导出成功", f"词库已成功导出
至:\n{save_path}")
1888     except Exception as e:
1889         self.ui.show_toast("导出失败", str(e), True)
1890
1891 def run_test_detection(self, text: str):
1892     """
1893     测试区域高亮检测算法:
1894     遍历缓存寻找命中词汇, 采用富文本 HTML 标签进行替换。
1895     按词长降序排列, 防止“枪支”被“枪支弹药”部分覆盖的问题。
1896     """
1897
1898     all_words_map = {}
1899     for cat, words in self.word_cache.items():
1900         for w in words:
1901             all_words_map[w] = cat
1902
1903     sorted_words = sorted(all_words_map.keys(), key=len, reverse=True)
1904
1905     color_map = {
1906         "政治敏感": "#f56c6c", # 红色
1907         "色情": "#e6a23c", # 橙色
1908         "暴力": "#903333", # 深红
1909         "其他": "#909399" # 灰色
1910     }
1911

```

```

1912
1913
1914         html_text = text.replace('\n', '<br>')
1915
1916
1917         for word in sorted_words:
1918             if word in html_text:
1919                 cat = all_words_map[word]
1920                 color = color_map.get(cat, "#000000")
1921
1922                 replacement = f'<span style="background-color: {color};
color: white; border-radius: 3px; padding: 0 2px;"><b>{word}</b>
<i>({cat})</i></span>'
1923
1924                 html_text = re.sub(re.escape(word), replacement,
html_text)
1925
1926
1927         self.ui.render_test_result(html_text)
1928
1929         def get_current_cache(self) -> dict:
1930             """对外接口：供系统中其他模块（如图片 OCR 识别模块）直接获取内存敏感词库
进行校验"""
1931             return self.word_cache
1932 import os
1933 import sqlite3
1934 import datetime
1935 import traceback
1936
1937 from openpyxl.styles.builtins import accent_4
1938
1939
1940 from core.image_processor import preprocess
1941 from core.content_detector import ContentDetector
1942 from core.ocr_detector import OCRDetector
1943
1944
1945 from reportlab.lib.pagesizes import A4
1946 from reportlab.platypus import SimpleDocTemplate, Paragraph, Spacer,
Image, Table, TableStyle
1947 from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle
1948 from reportlab.lib import colors
1949 from reportlab.pdfbase import pdfmetrics
1950 from reportlab.pdfbase.ttfonts import TTFont
1951
1952 class SingleImageAuditModule:
1953     """图片单张审核核心控制类：完全解耦 UI，处理运算流、持久化与报告导出"""
1954     def __init__(self, ui_instance):
1955         self.ui = ui_instance
1956         self.db_path = "data/app.db"
1957
1958
1959         self.ui.image_requested.connect(self.process_audit_pipeline)
1960         self.ui.save_requested.connect(self.save_audit_to_db)
1961         self.ui.pdf_export_requested.connect(self.export_to_pdf_report)
1962

```

```

1963
1964         self.content_detector = ContentDetector()
1965         self.ocr_detector = OCRDetector()
1966
1967
1968         self.runtime_cache = {}
1969
1970     def process_audit_pipeline(self, image_path: str):
1971         """串联执行图像处理流水线"""
1972         try:
1973
1974             self.ui.update_workflow_status("审核中", "#e6a23c")
1975
1976
1977             processed_arr, meta = preprocess(image_path)
1978
1979
1980             predict_res = self.content_detector.predict(processed_arr)
1981             self.ui.render_detection_metrics(predict_res)
1982
1983
1984             raw_texts, _ = self.ocr_detector.detect_text(processed_arr)
1985
1986
1987             sensitive_words = self._fetch_active_sensitive_words()
1988             found_sensitive = [w for w in sensitive_words if any(w in
line for line in raw_texts)]
1989             self.ui.render_ocr_text(raw_texts, found_sensitive)
1990
1991
1992             ai_class = predict_res.get("class", "正常")
1993             score = predict_res.get("confidence", 1.0)
1994
1995             if ai_class == "正常" and score >= 0.85:
1996                 auto_decision = "通过"
1997             elif ai_class != "正常" and score >= 0.75:
1998                 auto_decision = "驳回"
1999             else:
2000                 auto_decision = "人工复核"
2001
2002             self.ui.update_decision_box(auto_decision)
2003
2004
2005             if any("枪支" in line or "弹药" in line for line in raw_texts):
2006                 predict_res["class"] = "暴恐"
2007                 predict_res["confidence"] = 0.92
2008
2009             self.runtime_cache = {
2010                 "image_path": image_path,
2011                 "result": auto_decision,
2012                 "confidence": score,
2013                 "texts": raw_texts,
2014                 "detected_class": ai_class
2015             }

```



```

2016
2017         except Exception as err:
2018             self._log_exception("图片智能流水线分析中断", err)
2019             self.ui.update_workflow_status("异常中断", "#f56c6c")
2020             self.ui.show_message_box("检测故障", f"流水线处理图片时遭遇底层
硬件/算法库拒绝:\n{str(err)}", is_error=True)
2021
2022     def _fetch_active_sensitive_words(self):
2023         """
2024         【跨模块联动核心】实时从中央敏感词库拉取最新词汇快照
2025         作为纯消费者，绝不在此干涉表结构，建表权 100%交由 SensitiveWordModule
掌管。
2026         """
2027         words = []
2028         try:
2029             conn = sqlite3.connect(self.db_path)
2030             cursor = conn.cursor()
2031
2032
2033             cursor.execute("SELECT word FROM sensitive_words")
2034             words = [row[0] for row in cursor.fetchall()]
2035             conn.close()
2036
2037         except sqlite3.OperationalError:
2038             words = ["暴恐", "色情", "违禁", "赌博", "代考"]
2039         except Exception as e:
2040             print(f"[警告] 联动敏感词库时发生未可知中断: {e}")
2041
2042
2043         return [w for w in words if w.strip()]
2044
2045     def save_audit_to_db(self, image_path, result, confidence, auditor,
opinion):
2046         """持久化审核单据至数据库，完成生命周期最终演变"""
2047         try:
2048             conn = sqlite3.connect(self.db_path)
2049             cursor = conn.cursor()
2050
2051             now_str = datetime.datetime.now().strftime("%Y-%m-
%d %H:%M:%S")
2052             final_status = "已通过" if result == "通过" else "已驳回" if
result == "驳回" else "待人工复核"
2053
2054             cursor.execute('''
2055                 INSERT INTO audit_records (file_path, result,
confidence, auditor, audit_time, status)
2056                 VALUES (?, ?, ?, ?, ?, ?)
2057                 ''', (image_path, f"{result} ({opinion})",
confidence, auditor, now_str, final_status))
2058
2059             conn.commit()
2060             conn.close()
2061
2062

```

```

2063         color_map = {"已通过": "#67c23a", "已驳回": "#f56c6c", "待人工
复核": "#e6a23c"}
2064         self.ui.update_workflow_status(final_status,
color_map.get(final_status, "#909399"))
2065
2066
2067         if self.runtime_cache:
2068             self.runtime_cache["auditor"] = auditor
2069             self.runtime_cache["audit_time"] = now_str
2070             self.runtime_cache["final_status"] = final_status
2071             self.runtime_cache["opinion"] = opinion
2072
2073         self.ui.show_message_box("提交成功", "本张图片审核凭证与处理记录
已安全写入系统账册。")
2074
2075     except Exception as err:
2076         self._log_exception("审核凭证入库阻断", err)
2077         self.ui.show_message_box("存储失败", f"无法将当前的审核凭证安全
写入本地数据库:\n{str(err)}", is_error=True)
2078
2079     def export_to_pdf_report(self):
2080         """根据当前缓存的内容实时构建 ReportLab 高清 PDF 数据报告"""
2081         if not self.runtime_cache:
2082             self.ui.show_message_box("导出受阻", "当前画布无活跃的审核任务快
照, 无法生成报告!", is_error=True)
2083             return
2084
2085         try:
2086
2087             confidence = self.runtime_cache.get('confidence', 0.0)
2088             if isinstance(confidence, str):
2089                 try:
2090                     confidence = float(confidence)
2091                 except ValueError:
2092                     confidence = 0.0
2093             self.runtime_cache['confidence'] = confidence
2094
2095
2096             img_path = self.runtime_cache.get('image_path', '')
2097             if not img_path or not os.path.exists(img_path):
2098                 self.ui.show_message_box("导出受阻", "原始图片文件不存在, 无
法嵌入报告!", is_error=True)
2099                 return
2100
2101             # 报告默认落地项目根目录
2102             pdf_filename =
f"audit_report_{datetime.datetime.now().strftime('%Y%m%d_%H%M%S')}.pdf"
2103             from reportlab.lib.pagesizes import A4
2104             doc = SimpleDocTemplate(pdf_filename, pagesize=A4,
leftMargin=40, rightMargin=40,
2105                                     topMargin=40, bottomMargin=40)
2106
2107
2108             # 尝试加载中文字体, 失败则回退

```

```

2109         try:
2110             from reportlab.pdfbase import pdfmetrics
2111             from reportlab.pdfbase.ttfonts import TTFont
2112             pdfmetrics.registerFont(TTFont('SimSun',
2113 'C:/Windows/Fonts/simsun.ttc'))
2114             font_name = 'SimSun'
2115         except Exception:
2116             font_name = 'Helvetica'
2117
2118         styles = getSampleStyleSheet()
2119         title_style = ParagraphStyle('TitleStyle',
2120 parent=styles['Heading1'],
2121                                     fontName=font_name, fontSize=22,
2122 alignment=1, spaceAfter=20)
2123         body_style = ParagraphStyle('BodyStyle',
2124 parent=styles['Normal'],
2125                                     fontName=font_name, fontSize=11,
2126 leading=16)
2127
2128         story = []
2129         story.append(Paragraph("图片内容智能审查与合规检测报告",
2130 title_style))
2131         story.append(Spacer(1, 10))
2132
2133         # 构建表格数据（确保所有值都是字符串）
2134         data = [
2135             [Paragraph("<b>审核项目</b>", body_style), Paragraph("<b>
2136 系统存证参数明细</b>", body_style)],
2137             [Paragraph("素材物理路径", body_style),
2138              Paragraph(str(self.runtime_cache.get('image_path', '未知
2139 '))), body_style)],
2140             [Paragraph("AI 判别核心类目", body_style),
2141              Paragraph(str(self.runtime_cache.get('detected_class', '
2142 未知'))), body_style)],
2143             [Paragraph("最高归属置信度", body_style),
2144              Paragraph(f"{confidence:.2f}", body_style)],
2145             [Paragraph("初审裁决结论", body_style),
2146              Paragraph(str(self.runtime_cache.get('final_status', '过
2147 程未终审'))), body_style)],
2148             [Paragraph("审查专家意见", body_style),
2149              Paragraph(str(self.runtime_cache.get('opinion', '无明确意
2150 见'))), body_style)],
2151             [Paragraph("流水线流水号 / 时间", body_style),
2152              Paragraph(str(self.runtime_cache.get('audit_time', '未标
2153 记'))), body_style]]
2154         ]
2155
2156         t = Table(data, colWidths=[130, 380])
2157         t.setStyle(TableStyle([
2158             ('BACKGROUND', (0, 0), (1, 0),
2159 colors.HexColor("#409eff")),
2160             ('TEXTCOLOR', (0, 0), (1, 0), colors.whitesmoke),
2161             ('ALIGN', (0, 0), (-1, -1), 'LEFT'),

```

```

2148         ('BOTTOMPADDING', (0, 0), (-1, -1), 8),
2149         ('TOPPADDING', (0, 0), (-1, -1), 8),
2150         ('GRID', (0, 0), (-1, -1), 0.5,
colors.HexColor("#dcdfe6"))),
2151     ]))
2152     story.append(t)
2153     story.append(Spacer(1, 20))
2154
2155     # 插入图片
2156     story.append(Paragraph("<b>审查原图快照存档: </b>", body_style))
2157     story.append(Spacer(1, 8))
2158     try:
2159         from reportlab.platypus import Image
2160         img = Image(img_path, width=240, height=180)
2161         story.append(img)
2162     except Exception as img_err:
2163         story.append(Paragraph(f"[提示] 无法嵌入图片:
{str(img_err)}", body_style))
2164
2165     story.append(Spacer(1, 20))
2166
2167     # OCR 文本
2168     story.append(Paragraph("<b>图片内文本视觉提炼 (OCR): </b>",
body_style))
2169     story.append(Spacer(1, 8))
2170     ocr_texts = self.runtime_cache.get('texts', [])
2171     ocr_text_block = "".join(ocr_texts) if ocr_texts else "未提取
到可疑文本数据。"
2172     story.append(Paragraph(ocr_text_block, body_style))
2173
2174     # 生成 PDF
2175     doc.build(story)
2176     self.ui.show_message_box("导出成功", f"系统合规报告已编制完成并
交付至本地: \n{pdf_filename}")
2177
2178     except Exception as err:
2179         import traceback
2180         traceback.print_exc()
2181         self.ui.show_message_box("导出失败", f"在编译、排版或写出 PDF 文
本流时遭遇内核致命拒绝:\n{str(err)}",
2182                                     is_error=True)
2183
2184     def _log_exception(self, context, err):
2185         """内部轻量日志系统捕捉回溯细节"""
2186         print(f"[{datetime.datetime.now().strftime('%H:%M:%S')}] 致命红灯
- 位置: {context}")
2187         traceback.print_some_detailed_info = traceback.print_exc()2188
import sqlite3
2189 import os
2190 import random
2191 from datetime import datetime, timedelta
2192 from openpyxl import Workbook
2193 from openpyxl.styles import Font, Alignment

```

```

2194 from PyQt6.QtCore import QObject
2195
2196
2197 class StatisticsModule(QObject):
2198     """报表与统计业务逻辑核心"""
2199
2200     def __init__(self, ui_instance):
2201         super().__init__()
2202         self.ui = ui_instance
2203         self.db_path = "data/app.db"
2204
2205         self.ensure_tables_and_mock_data()
2206
2207
2208         self.ui.query_requested.connect(self.fetch_data)
2209         self.ui.export_requested.connect(self.export_excel)
2210
2211     def ensure_tables_and_mock_data(self):
2212         """确保表结构正确，并注入随机演示数据"""
2213         os.makedirs(os.path.dirname(self.db_path), exist_ok=True)
2214         conn = sqlite3.connect(self.db_path)
2215         cursor = conn.cursor()
2216
2217
2218         cursor.execute('''
2219             CREATE TABLE IF NOT EXISTS audit_records (
2220                 id INTEGER PRIMARY KEY AUTOINCREMENT,
2221                 file_path TEXT,
2222                 result TEXT,
2223                 confidence REAL,
2224                 auditor TEXT,
2225                 audit_time TEXT,
2226                 status TEXT,
2227                 recognized_text TEXT,
2228                 is_deleted INTEGER DEFAULT 0
2229             )
2230         ''')
2231
2232
2233         cursor.execute("SELECT COUNT(*) FROM audit_records")
2234         if cursor.fetchone()[0] == 0:
2235             print("[统计模块] 正在生成过去 10 天的模拟审核记录...")
2236             categories = ['色情', '暴力', '政治敏感', '广告', '正常']
2237             auditors = ['System_AI', 'System_AI', 'Admin_Lee',
2238 'Auditor_Wang']
2239             mock_data = []
2240             today = datetime.now()
2241             for i in range(200):
2242                 rand_days = random.randint(0, 10)
2243                 rec_date = (today -
timedelta(days=rand_days)).strftime("%Y-%m-%d %H:%M:%S")
2244                 cat = random.choices(categories, weights=[15, 10, 5, 20,
50])[0]
2245                 res = '正常' if cat == '正常' else '违规'

```

```

2245         aud = random.choice(auditors)
2246
2247         mock_data.append((
2248             f"/img/test_{i}.jpg",
2249             rec_date,
2250             '已完成',
2251             res,
2252             random.uniform(0.6, 0.99),
2253             cat,
2254             aud,
2255             0
2256         ))
2257         cursor.executemany('''
2258             INSERT INTO audit_records
2259             (file_path, audit_time, status, result, confidence,
2260             recognized_text, auditor, is_deleted)
2261             VALUES (?, ?, ?, ?, ?, ?, ?, ?)
2262             ''', mock_data)
2263         conn.commit()
2264         conn.close()
2265
2266     def fetch_data(self, start_date, end_date):
2267         """执行 SQL 聚合查询并组装为报表所需的数据结构"""
2268         end_date_inclusive = f"{end_date} 23:59:59"
2269         start_date_inclusive = f"{start_date} 00:00:00"
2270
2271         try:
2272             conn = sqlite3.connect(self.db_path)
2273             cursor = conn.cursor()
2274
2275             data = {"total": 0, "passed": 0, "trend": {}, "categories":
2276             {}, "workload": {}}
2277
2278             cursor.execute('''
2279                 SELECT COUNT(*), SUM(CASE WHEN result='正常' THEN 1 ELSE
2280                 0 END)
2281                 FROM audit_records
2282                 WHERE audit_time BETWEEN ? AND ? AND is_deleted=0
2283                 ''', (start_date_inclusive, end_date_inclusive))
2284             row = cursor.fetchone()
2285             data["total"] = row[0] or 0
2286             data["passed"] = row[1] or 0
2287
2288             cursor.execute('''
2289                 SELECT substr(audit_time, 1, 10) as dt, COUNT(*)
2290                 FROM audit_records
2291                 WHERE audit_time BETWEEN ? AND ? AND is_deleted=0
2292                 GROUP BY dt ORDER BY dt
2293                 ''', (start_date_inclusive, end_date_inclusive))
2294             for dt, cnt in cursor.fetchall():
2295                 data["trend"][dt] = cnt
2296

```

```

2297
2298         cursor.execute('''
2299             SELECT recognized_text, COUNT(*)
2300             FROM audit_records
2301             WHERE result != '正常' AND audit_time BETWEEN ? AND ? AND
is_deleted=0
2302             GROUP BY recognized_text
2303             ''', (start_date_inclusive, end_date_inclusive))
2304         for cat, cnt in cursor.fetchall():
2305             data["categories"][cat] = cnt
2306
2307
2308         cursor.execute('''
2309             SELECT auditor, COUNT(*)
2310             FROM audit_records
2311             WHERE audit_time BETWEEN ? AND ? AND is_deleted=0
2312             GROUP BY auditor ORDER BY COUNT(*) DESC
2313             ''', (start_date_inclusive, end_date_inclusive))
2314         for auditor, cnt in cursor.fetchall():
2315             data["workload"][auditor] = cnt
2316
2317         conn.close()
2318
2319
2320         self.ui.update_dashboard(data)
2321
2322     except Exception as e:
2323         import traceback
2324         traceback.print_exc()
2325         self.ui.show_toast("数据查询失败", str(e), True)
2326
2327     def export_excel(self, start_date, end_date, save_path):
2328         """利用 openpyxl 导出结构化报表"""
2329         try:
2330             conn = sqlite3.connect(self.db_path)
2331             cursor = conn.cursor()
2332
2333             wb = Workbook()
2334             ws = wb.active
2335             ws.title = "统计汇总"
2336
2337             title_font = Font(name='微软雅黑', size=14, bold=True)
2338             head_font = Font(name='微软雅黑', bold=True)
2339             align = Alignment(horizontal='center', vertical='center')
2340
2341             ws.merge_cells('A1:C1')
2342             ws['A1'] = f"内容审核数据统计报表 ({start_date} 至 {end_date})"
2343             ws['A1'].font = title_font
2344             ws['A1'].alignment = align
2345
2346             ws.append([""])
2347             ws.append(["统计维度", "数值", "说明"])
2348             for cell in ws[3]:
2349                 cell.font = head_font

```

```

2350
2351
2352         cursor.execute('''
2353             SELECT recognized_text, COUNT(*) FROM audit_records
2354             WHERE audit_time BETWEEN ? AND ? AND is_deleted=0
2355             GROUP BY recognized_text
2356             ''', (f"{start_date} 00:00:00", f"{end_date} 23:59:59"))
2357
2358         for row in cursor.fetchall():
2359             ws.append([f"类别: {row[0]}", row[1], "件"])
2360
2361         ws.column_dimensions['A'].width = 20
2362         ws.column_dimensions['B'].width = 15
2363         ws.column_dimensions['C'].width = 15
2364
2365         wb.save(save_path)
2366         conn.close()
2367         self.ui.show_toast("导出成功", f"报表已成功导出至:
\n{save_path}")
2368
2369         except Exception as e:
2370             import traceback
2371             traceback.print_exc()
2372             self.ui.show_toast("导出失败", str(e), True)
2373 import
sqlite3
2374 import hashlib
2375 import datetime
2376 from PyQt6.QtCore import QObject
2377
2378
2379 class UserManagementModule(QObject):
2380     """用户特权与元数据持久化控制层（完全摒弃 Widget 部件，实现逻辑隔离）"""
2381
2382     def __init__(self, ui_instance, operator_username: str, operator_role:
str):
2383         super().__init__()
2384         self.ui = ui_instance
2385         self.op_name = operator_username
2386         self.op_role = operator_role
2387         self.db_path = "data/app.db"
2388         self.page_size = 10
2389
2390
2391         self.ui.refresh_requested.connect(self.load_filtered_users)
2392         self.ui.create_submitted.connect(self.execute_create)
2393         self.ui.edit_submitted.connect(self.execute_update)
2394         self.ui.delete_action_triggered.connect(self.execute_delete)
2395         self.ui.pwd_reset_triggered.connect(self.execute_password_reset)
2396         self.ui.unlock_triggered.connect(self.execute_unlock)
2397
2398     def verify_permission_recursive(self, operator_role: str, target_role:
str, call_depth: int = 0) -> bool:
2399         """
2400         【核心算法】基于单向继承链的深度有限递归权限校验算法

```



```

2401     规则模型: admin -> auditor -> user
2402     """
2403     hierarchy = {"admin": "auditor", "auditor": "user", "user": None}
2404
2405
2406     if operator_role not in hierarchy or (target_role not in
2407 hierarchy and target_role is not None):
2408         return False
2409
2410     if call_depth == 0 and operator_role == target_role:
2411         return operator_role == "admin"
2412
2413     parent_node = hierarchy.get(operator_role)
2414     if not parent_node:
2415         return False
2416     if parent_node == target_role:
2417         return True
2418
2419
2420     return self.verify_permission_recursive(parent_node, target_role,
2421 call_depth + 1)
2422
2423     def load_filtered_users(self, search_keyword: str, role_filter: str,
2424 page_num: int):
2425         """带有严格行级数据隔离特性的分页拉取机制"""
2426         try:
2427             conn = sqlite3.connect(self.db_path)
2428             cursor = conn.cursor()
2429
2430             base_query = "SELECT id, username, email, role, locked_until
2431 FROM users WHERE 1=1"
2432             params = []
2433
2434             if self.op_role == "admin":
2435                 pass
2436             elif self.op_role == "auditor":
2437
2438                 base_query += " AND (role = 'user' OR username = ?)"
2439                 params.append(self.op_name)
2440             else:
2441
2442                 base_query += " AND username = ?"
2443                 params.append(self.op_name)
2444
2445             if search_keyword:
2446                 base_query += " AND username LIKE ?"
2447                 params.append(f"%{search_keyword}%")
2448             if role_filter != "全部角色":
2449                 base_query += " AND role = ?"
2450                 params.append(role_filter)
2451

```

```

2452         count_query = f"SELECT COUNT(*) FROM ({base_query})"
2453         cursor.execute(count_query, params)
2454         total_records = cursor.fetchone()[0]
2455         total_pages = (total_records + self.page_size - 1) //
self.page_size
2456
2457
2458         page_num = max(1, min(page_num, total_pages if total_pages >
0 else 1))
2459
2460
2461         base_query += " ORDER BY id DESC LIMIT ? OFFSET ?"
2462         extended_params = params + [self.page_size, (page_num - 1) *
self.page_size]
2463
2464         cursor.execute(base_query, extended_params)
2465         raw_rows = cursor.fetchall()
2466         conn.close()
2467
2468
2469         self.ui.render_user_grid(raw_rows, total_pages, page_num)
2470
2471     except Exception as e:
2472         self.ui.show_alert("数据加载故障", f"无法安全访问持久化用户实体
库:\n{str(e)}", is_error=True)
2473
2474     def execute_create(self, form_data: dict):
2475         """添加新用户"""
2476
2477         if not self.verify_permission_recursive(self.op_role,
form_data["role"]):
2478             self.ui.show_alert("安全拦截", "您的系统授权等级不足, 无法创建该
等级的目标实体账户!", is_error=True)
2479             return
2480
2481         if not form_data["username"] or not form_data["password"]:
2482             self.ui.show_alert("提示", "用户名与初始密码字段不能为空。",
is_error=True)
2483             return
2484
2485         try:
2486             conn = sqlite3.connect(self.db_path)
2487             cursor = conn.cursor()
2488
2489             pwd_hash = hashlib.sha256(form_data["password"].encode('utf-
8')).hexdigest()
2490             cursor.execute('''
2491                 INSERT INTO users (username, password_hash, email, role)
2492                 VALUES (?, ?, ?, ?)
2493             ''', (form_data["username"], pwd_hash, form_data["email"],
form_data["role"]))
2494
2495             conn.commit()
2496             conn.close()

```

```

2497         self.ui.show_alert("操作成功", f"新账号
'{form_data['username']}' 已经成功注册。")
2498         self.load_filtered_users("", "全部角色", 1)
2499     except sqlite3.IntegrityError:
2500         self.ui.show_alert("冲突冲突", "该用户名在系统中已存在, 请更换账
户名!", is_error=True)
2501     except Exception as e:
2502         self.ui.show_alert("注册错误", f"数据库底层写入异常:\n{str(e)}",
is_error=True)
2503
2504     def execute_update(self, target_id: int, form_data: dict):
2505         """编辑更新已有用户资料"""
2506         if not self.verify_permission_recursive(self.op_role,
form_data["role"]):
2507             self.ui.show_alert("安全违规", "无权修改为该目的系统角色, 越权请
求已驳回!", is_error=True)
2508             return
2509
2510         try:
2511             conn = sqlite3.connect(self.db_path)
2512             cursor = conn.cursor()
2513
2514
2515             cursor.execute("SELECT role FROM users WHERE id = ?",
(target_id,))
2516             old_role_row = cursor.fetchone()
2517             if old_role_row and not
self.verify_permission_recursive(self.op_role, old_role_row[0]):
2518                 self.ui.show_alert("提权拦截", "目标对象权限资产高于您, 已被
底层防线拦截.", is_error=True)
2519                 conn.close()
2520                 return
2521
2522             if form_data["password"]:
2523
2524                 new_hash =
hashlib.sha256(form_data["password"].encode('utf-8')).hexdigest()
2525                 cursor.execute("UPDATE users SET password_hash = ?, email
= ?, role = ? WHERE id = ?",
2526                                (new_hash, form_data["email"],
form_data["role"], target_id))
2527             else:
2528
2529                 cursor.execute("UPDATE users SET email = ?, role = ?
WHERE id = ?",
2530                                (form_data["email"], form_data["role"],
target_id))
2531
2532             conn.commit()
2533             conn.close()
2534             self.ui.show_alert("更新成功", "用户信息要素变更已同步入库。")
2535             self.load_filtered_users("", "全部角色", self.ui.current_page)
2536         except Exception as e:

```

```

2537         self.ui.show_alert("修改失败", f"执行信息更新时遭遇系统异
常:\n{str(e)}", is_error=True)
2538
2539     def execute_delete(self, target_id: int, target_role: str):
2540         """删除用户"""
2541         if not self.verify_permission_recursive(self.op_role,
target_role):
2542             self.ui.show_alert("越权拦截", "您没有物理切除该级别账号的系统操
作所有权!", is_error=True)
2543             return
2544
2545         try:
2546             conn = sqlite3.connect(self.db_path)
2547             cursor = conn.cursor()
2548             cursor.execute("DELETE FROM users WHERE id = ?", (target_id,))
2549             conn.commit()
2550             conn.close()
2551             self.ui.show_alert("清理成功", "对应的用户历史档案已从系统中彻底
移除。")
2552             self.load_filtered_users("", "全部角色", self.ui.current_page)
2553         except Exception as e:
2554             self.ui.show_alert("删除故障", f"无法正常移除对应数据
行:\n{str(e)}", is_error=True)
2555
2556     def execute_password_reset(self, target_id: int, target_role: str):
2557         """强制重置目标密码为系统缺省值 admin@123"""
2558         if not self.verify_permission_recursive(self.op_role,
target_role):
2559             self.ui.show_alert("鉴权失败", "越权凭证, 您无权强制干预此级别账
号的密钥密码!", is_error=True)
2560             return
2561
2562         try:
2563             default_hash = hashlib.sha256("admin@123".encode('utf-
8')).hexdigest()
2564             conn = sqlite3.connect(self.db_path)
2565             cursor = conn.cursor()
2566             cursor.execute("UPDATE users SET password_hash = ? WHERE id
= ?", (default_hash, target_id))
2567             conn.commit()
2568             conn.close()
2569             self.ui.show_alert("重置完成", "该账户的登录密码已成功恢复为初始
预设值: admin@123")
2570         except Exception as e:
2571             self.ui.show_alert("重置失败", f"重置过程中遭遇内部阻
断:\n{str(e)}", is_error=True)
2572
2573     def execute_unlock(self, target_id: int, target_role: str):
2574         """解除账户暴力破解造成的锁闭状态"""
2575         if not self.verify_permission_recursive(self.op_role,
target_role):
2576             self.ui.show_alert("鉴权拒绝", "安全级别不足, 无法强行解除目标对
象的物理锁定保护!", is_error=True)

```

```

2577         return
2578
2579     try:
2580         conn = sqlite3.connect(self.db_path)
2581         cursor = conn.cursor()
2582         cursor.execute("UPDATE users SET locked_until = NULL WHERE id
= ?", (target_id,))
2583         conn.commit()
2584         conn.close()
2585         self.ui.show_alert("状态解锁", "该用户的登录爆破限制已解除, 当前
状态恢复为正常活动。")
2586         self.load_filtered_users("", "全部角色", self.ui.current_page)
2587     except Exception as e:
2588         self.ui.show_alert("解锁异常", f"执行解冻逻辑时发生错
误:\n{str(e)}", is_error=True)
2589
2590 from .login_module import LoginModule
2591 from .statistics_report import StatisticsModule
2592 from .about_system import AboutSystemModule
2593 from .user_management import UserManagementModule
2594 from .sensitive_word_management import SensitiveWordModule
2595 from .single_image_audit import SingleImageAuditModule
2596 from .batch_image_audit import BatchImageAuditModule
2597 from .audit_history import AuditHistoryModule
2598 from .appeal_handling import AppealHandlingModule
2599 from .model_config import ModelConfigModule
2600 from .register_module import RegisterModule
2601
2602
2603 __all__ = [
2604     "LoginModule",
2605     "StatisticsModule",
2606     "SystemSettingsModule",
2607     "AboutSystemModule",
2608     "UserManagementModule",
2609     "SensitiveWordModule",
2610     "SingleImageAuditModule",
2611     "BatchImageAuditModule",
2612     "AuditHistoryModule",
2613     "AppealHandlingModule",
2614     "ModelConfigModule",
2615     "RegisterModule"
2616 ]
2617 from PyQt6.QtWidgets import (QDialog, QVBoxLayout, QLabel,
QPushButton,
2618                             QMessageBox, QFrame)
2619 from PyQt6.QtCore import Qt
2620 from PyQt6.QtGui import QFont
2621
2622
2623 class AboutSystemUI(QDialog):
2624     """关于系统视图层, 负责信息展示与交互"""
2625
2626     def __init__(self, logic):
2627         super().__init__()
2628         self.logic = logic

```

```

2629         self.setWindowTitle("关于系统")
2630         self.setFixedSize(400, 450)
2631         self.setup_ui()
2632
2633     def setup_ui(self):
2634         layout = QVBoxLayout(self)
2635         layout.setContentsMargins(30, 30, 30, 30)
2636         layout.setSpacing(15)
2637
2638
2639         logo_label = QLabel("🐍")
2640         logo_label.setStyleSheet("font-size: 60px;")
2641         logo_label.setAlignment(Qt.AlignmentFlag.AlignCenter)
2642         layout.addWidget(logo_label)
2643
2644
2645         info = self.logic.get_system_info()
2646         name_label = QLabel(f"<b>{info['name']}</b>")
2647         name_label.setStyleSheet("font-size: 18px;")
2648         name_label.setAlignment(Qt.AlignmentFlag.AlignCenter)
2649         layout.addWidget(name_label)
2650
2651
2652         details = (
2653             f"版本: {info['version']}<br>"
2654             f"开发: {info['developer']}<br>"
2655             f"技术栈: {info['tech_stack']}<br>"
2656             f"许可: {info['license']}"
2657         )
2658         details_label = QLabel(details)
2659         details_label.setStyleSheet("color: #606266; line-height: 1.5;")
2660         details_label.setAlignment(Qt.AlignmentFlag.AlignCenter)
2661         layout.addWidget(details_label)
2662
2663         layout.addStretch()
2664
2665
2666         self.update_btn = QPushButton("检查更新")
2667         self.update_btn.setStyleSheet("padding: 8px; background-color:
#409eff; color: white; border-radius: 4px;")
2668         self.update_btn.clicked.connect(self.on_check_update)
2669         layout.addWidget(self.update_btn)
2670
2671     def on_check_update(self):
2672         """触发更新检查逻辑"""
2673         self.update_btn.setEnabled(False)
2674         self.update_btn.setText("正在检查...")
2675
2676         result = self.logic.check_for_updates()
2677
2678         self.update_btn.setEnabled(True)
2679         self.update_btn.setText("检查更新")
2680
2681         if result["update_available"]:

```

```

2682         QMessageBox.information(
2683             self, "检查更新",
2684             f"发现新版本 {result['latest_version']}!\n\n 更新内
容:\n{result['changelog']}\n\n 下载地址: {result['download_url']}"
2685         )
2686     else:
2687         QMessageBox.information(self, "检查更新", "当前已是最新版本。
")2688 import os
2689 from PyQt6.QtWidgets import (QWidget, QVBoxLayout, QHBoxLayout,
QPushButton,
2690                             QLabel, QTableWidget, QTableWidgetItem,
QHeaderView,
2691                             QComboBox, QDialog, QFormLayout, QTextEdit,
QMessageBox)
2692 from PyQt6.QtCore import Qt, pyqtSignal
2693 from PyQt6.QtGui import QPixmap, QColor
2694
2695
2696 class AppealReviewDialog(QDialog):
2697     """双屏对比复审弹窗"""
2698     decision_made = pyqtSignal(int, int, str, str)
2699
2700     def __init__(self, parent, appeal_data):
2701         super().__init__(parent)
2702
2703         self.appeal_data = appeal_data
2704         self.setWindowTitle(f"申诉复审工单 - #{self.appeal_data[0]}")
2705         self.setFixedSize(700, 500)
2706         self.setup_ui()
2707
2708     def setup_ui(self):
2709         main_layout = QHBoxLayout(self)
2710         main_layout.setSpacing(15)
2711
2712
2713         left_layout = QVBoxLayout()
2714         left_layout.addWidget(QLabel("<b>涉事图片预览:</b>"))
2715
2716         self.image_label = QLabel("暂无图片")
2717         self.image_label.setFixedSize(280, 280)
2718         self.image_label.setAlignment(Qt.AlignmentFlag.AlignCenter)
2719         self.image_label.setStyleSheet("border: 1px solid #dcdfe6;
background-color: #f5f7fa;")
2720
2721         img_path = self.appeal_data[6]
2722         if img_path and os.path.exists(img_path):
2723             pixmap = QPixmap(img_path).scaled(270, 270,
Qt.AspectRatioMode.KeepAspectRatio,
2724             Qt.TransformationMode.SmoothTransformation)
2725             self.image_label.setPixmap(pixmap)
2726
2727         left_layout.addWidget(self.image_label)
2728

```

```

2729
2730         confidence = self.appeal_data[8]
2731         try:
2732             conf_percent = float(confidence) * 100 if confidence is not
None else 0.0
2733         except (ValueError, TypeError):
2734             conf_percent = 0.0
2735
2736         info_form = QFormLayout()
2737         info_form.addRow("AI 判定:", QLabel(
2738             f"<b><span
style='color:#f56c6c;'>{self.appeal_data[7]}</span></b> (置信度:
{conf_percent:.1f}%)"'))
2739         info_form.addRow("敏感词:", QLabel(self.appeal_data[9] if
self.appeal_data[9] else "未提取"))
2740         left_layout.addLayout(info_form)
2741         left_layout.addStretch()
2742         main_layout.addLayout(left_layout, stretch=4)
2743
2744
2745         right_layout = QVBoxLayout()
2746         right_layout.addWidget(QLabel(f"<b>发起人:</b>
{self.appeal_data[2]}"))
2747         right_layout.addWidget(QLabel("<b>申诉理由:</b>"))
2748         reason_box = QTextEdit(self.appeal_data[4])
2749         reason_box.setReadOnly(True)
2750         reason_box.setFixedHeight(80)
2751         reason_box.setStyleSheet("background-color: #fcfcfc;")
2752         right_layout.addWidget(reason_box)
2753
2754         right_layout.addWidget(QLabel("<b>复审意见 (内部记录):</b>"))
2755         self.comment_input = QTextEdit()
2756         self.comment_input.setPlaceholderText("请填写复审确认依据...")
2757         right_layout.addWidget(self.comment_input)
2758
2759         btn_layout = QHBoxLayout()
2760         self.revoke_btn = QPushButton("✓ 撤销原判 (恢复正常)")
2761         self.maintain_btn = QPushButton("✗ 维持原判 (驳回申诉)")
2762         self.revoke_btn.setStyleSheet("background-color: #67c23a; color:
white; font-weight: bold; padding: 10px;")
2763         self.maintain_btn.setStyleSheet("background-color: #f56c6c; color:
white; font-weight: bold; padding: 10px;")
2764         btn_layout.addWidget(self.revoke_btn)
2765         btn_layout.addWidget(self.maintain_btn)
2766         right_layout.addLayout(btn_layout)
2767
2768         main_layout.addLayout(right_layout, stretch=6)
2769
2770         self.revoke_btn.clicked.connect(lambda: self.submit_decision("撤
销原判"))
2771         self.maintain_btn.clicked.connect(lambda: self.submit_decision("
维持原判"))
2772

```



```

2773     def submit_decision(self, decision):
2774         comment = self.comment_input.toPlainText().strip()
2775         if not comment:
2776             QMessageBox.warning(self, "提示", "请填写复审意见!")
2777         return
2778         reply = QMessageBox.question(self, "确认操作", f"您确定要
【{decision}】吗? 此操作不可逆。")
2779         if reply == QMessageBox.StandardButton.Yes:
2780             self.decision_made.emit(self.appeal_data[0],
self.appeal_data[1], decision, comment)
2781             self.accept()
2782
2783
2784 class AppealHandlingUI(QWidget):
2785     """申诉处理中心主界面"""
2786     query_requested = pyqtSignal(str)
2787     review_started = pyqtSignal(int)
2788     decision_submitted = pyqtSignal(int, int, str, str)
2789
2790     def __init__(self):
2791         super().__init__()
2792         self.cached_appeals = {}
2793         self.setup_ui()
2794
2795     def setup_ui(self):
2796         main_layout = QVBoxLayout(self)
2797         main_layout.setSpacing(12)
2798         main_layout.setContentsMargins(15, 15, 15, 15)
2799
2800
2801         toolbar_layout = QHBoxLayout()
2802         toolbar_layout.addWidget(QLabel("<b>工单状态筛选:</b>"))
2803         self.status_combo = QComboBox()
2804         self.status_combo.addItem("全部", "待处理", "复审中", "已撤销", "
已维持"])
2805         toolbar_layout.addWidget(self.status_combo)
2806         self.refresh_btn = QPushButton("刷新列表")
2807         toolbar_layout.addWidget(self.refresh_btn)
2808         toolbar_layout.addStretch()
2809         main_layout.addLayout(toolbar_layout)
2810
2811
2812         self.table = QTableWidget()
2813         self.table.setColumnCount(6)
2814         self.table.setHorizontalHeaderLabels(["申诉单号", "关联记录 ID", "
申诉人", "提交时间", "当前状态", "操作"])
2815
self.table.horizontalHeader().setSectionResizeMode(QHeaderView.ResizeMode.Stre
tch)
2816
self.table.setEditTriggers(QTableWidget.EditTrigger.NoEditTriggers)
2817
self.table.setSelectionBehavior(QTableWidget.SelectionBehavior.SelectRows)
2818         main_layout.addWidget(self.table)

```

```

2819
2820
2821         self.status_combo.currentTextChanged.connect(self.trigger_query)
2822         self.refresh_btn.clicked.connect(self.trigger_query)
2823
2824     def trigger_query(self):
2825         status = self.status_combo.currentText()
2826         self.query_requested.emit(status)
2827
2828     def render_table(self, records):
2829         self.table.setRowCount(0)
2830         self.cached_appeals.clear()
2831         for row_idx, rec in enumerate(records):
2832             a_id, r_id, submitter, s_time, status, reason, file_path,
result, conf, text = rec
2833             self.cached_appeals[a_id] = rec
2834             self.table.insertRow(row_idx)
2835             self.table.setItem(row_idx, 0, QTableWidgetItem(f"APL-
{a_id:04d}"))
2836             self.table.setItem(row_idx, 1, QTableWidgetItem(str(r_id)))
2837             self.table.setItem(row_idx, 2, QTableWidgetItem(submitter))
2838             self.table.setItem(row_idx, 3, QTableWidgetItem(s_time))
2839             status_item = QTableWidgetItem(status)
2840             if status == "待处理":
2841                 status_item.setForeground(QColor("#e6a23c"))
2842             elif status == "复审中":
2843                 status_item.setForeground(QColor("#409eff"))
2844             elif status in ["已撤销", "已维持"]:
2845                 status_item.setForeground(QColor("#909399"))
2846             self.table.setItem(row_idx, 4, status_item)
2847
2848             act_btn = QPushButton("去处理" if status in ["待处理", "复审中"]
else "查看明细")
2849             act_btn.setStyleSheet("color: #409eff;" if status in ["待处理",
"复审中"] else "color: #909399;")
2850             self.table.setCellWidget(row_idx, 5, act_btn)
2851             act_btn.clicked.connect(lambda checked, idx=a_id, s=status:
self.open_review_dialog(idx, s))
2852
2853     def open_review_dialog(self, appeal_id, current_status):
2854         if current_status == "待处理":
2855             self.review_started.emit(appeal_id)
2856         data = self.cached_appeals.get(appeal_id)
2857         if data:
2858             dialog = AppealReviewDialog(self, data)
2859             if current_status in ["已撤销", "已维持"]:
2860                 dialog.revoke_btn.setEnabled(False)
2861                 dialog.maintain_btn.setEnabled(False)
2862                 dialog.comment_input.setReadOnly(True)
2863                 dialog.comment_input.setText("该工单已处理完毕。")
2864             dialog.decision_made.connect(self.relay_decision)
2865             dialog.exec()
2866
2867     def relay_decision(self, appeal_id, record_id, decision, comment):

```

```

2868         self.decision_submitted.emit(appeal_id, record_id, decision,
comment)
2869
2870     def show_toast(self, title, message, is_error=False):
2871         if is_error:
2872             QMessageBox.critical(self, title, message)
2873         else:
2874             QMessageBox.information(self, title, message)
2875 import os
2876 from PyQt6.QtWidgets import (QWidget, QVBoxLayout, QHBoxLayout,
QPushButton,
2877                               QLabel, QTableWidgetItem,
QHeaderView,
2878                               QLineEdit, QComboBox, QDateEdit, QDialog,
QFormLayout,
2879                               QTextEdit, QMessageBox)
2880 from PyQt6.QtCore import Qt, pyqtSignal, QDate
2881 from PyQt6.QtGui import QPixmap, QColor
2882
2883
2884 class HistoryDetailDialog(QDialog):
2885     """历史记录全要素详情弹窗"""
2886     appeal_requested = pyqtSignal(int)
2887     export_requested = pyqtSignal(int)
2888
2889     def __init__(self, parent, record_data):
2890         super().__init__(parent)
2891         self.record_id = record_data[0]
2892         self.record_data = record_data
2893         self.setWindowTitle(f"审核详情 - 记录 ID: {self.record_id}")
2894         self.setFixedSize(550, 480)
2895         self.setup_ui()
2896
2897     def setup_ui(self):
2898         main_layout = QHBoxLayout(self)
2899         main_layout.setSpacing(15)
2900
2901
2902         left_layout = QVBoxLayout()
2903         self.image_label = QLabel("暂无图片预览")
2904         self.image_label.setFixedSize(200, 200)
2905         self.image_label.setAlignment(Qt.AlignmentFlag.AlignCenter)
2906         self.image_label.setStyleSheet("border: 1px solid #dcdfe6;
background-color: #f5f7fa; border-radius: 4px;")
2907
2908         img_path = self.record_data[1]
2909         if os.path.exists(img_path):
2910             pixmap = QPixmap(img_path).scaled(190, 190,
Qt.AspectRatioMode.KeepAspectRatio,
2911             Qt.TransformationMode.SmoothTransformation)
2912             self.image_label.setPixmap(pixmap)
2913         else:
2914             self.image_label.setText("原图已不存在\n或已被移动")
2915             self.image_label.setStyleSheet("color: #909399; border: 1px

```

```

solid #dcdfe6; background-color: #f5f7fa;")
2916
2917     left_layout.addWidget(self.image_label)
2918     left_layout.addStretch()
2919
2920
2921     self.appeal_btn = QPushButton("发起行政申诉")
2922     self.export_btn = QPushButton("导出单条报告")
2923     self.appeal_btn.setStyleSheet("background-color: #e6a23c; color:
white; font-weight: bold;")
2924     self.export_btn.setStyleSheet("background-color: #909399; color:
white; font-weight: bold;")
2925
2926     left_layout.addWidget(self.appeal_btn)
2927     left_layout.addWidget(self.export_btn)
2928     main_layout.addLayout(left_layout, stretch=4)
2929
2930
2931     form_widget = QWidget()
2932     form_layout = QFormLayout(form_widget)
2933     form_layout.setSpacing(10)
2934
2935
2936     res_label = QLabel(self.record_data[4])
2937     res_label.setContentsMargins(8, 4, 8, 4)
2938     res_label.setStyleSheet(self.get_badge_style(self.record_data[4]))
2939
2940     conf_val = float(self.record_data[5]) * 100 if self.record_data[5]
else 0.0
2941
2942     form_layout.addRow("图片路径:", QLabel(os.path.basename(img_path)))
2943     form_layout.addRow("审核状态:", QLabel(self.record_data[3]))
2944     form_layout.addRow("判定结果:", res_label)
2945     form_layout.addRow("识别置信度:", QLabel(f"{conf_val:.2f}%"))
2946     form_layout.addRow("审核人员:", QLabel(self.record_data[7]))
2947     form_layout.addRow("归档时间:", QLabel(self.record_data[2]))
2948
2949
2950     ocr_text = QTextEdit()
2951     ocr_text.setText(self.record_data[6] if self.record_data[6] else
"（未提取到文本要素）")
2952     ocr_text.setReadOnly(True)
2953     ocr_text.setFixedHeight(120)
2954     ocr_text.setStyleSheet("background-color: #f5f7fa; border-radius:
4px;")
2955     form_layout.addRow("识别文字内容:", ocr_text)
2956
2957     main_layout.addWidget(form_widget, stretch=6)
2958
2959
2960     self.appeal_btn.clicked.connect(lambda:
self.appeal_requested.emit(self.record_id))
2961     self.export_btn.clicked.connect(lambda:
self.export_requested.emit(self.record_id))

```

```

2962
2963     def get_badge_style(self, result_text):
2964         if "正常" in result_text:
2965             return "background-color: #f0f9eb; color: #67c23a; border:
1px solid #c2e7b0; font-weight: bold; border-radius: 3px;"
2966         elif "违规" in result_text:
2967             return "background-color: #fef0f0; color: #f56c6c; border:
1px solid #fde2e2; font-weight: bold; border-radius: 3px;"
2968         return "background-color: #fdf6ec; color: #e6a23c; border: 1px
solid #f5dab1; font-weight: bold; border-radius: 3px;"
2969
2970
2971 class AuditHistoryUI(QWidget):
2972     """审核历史记录中心控制台 UI"""
2973
2974     query_requested = pyqtSignal(dict)
2975     delete_requested = pyqtSignal(int)
2976     appeal_api_call = pyqtSignal(int)
2977     report_export_call = pyqtSignal(int, str)
2978
2979     def __init__(self):
2980         super().__init__()
2981         self.current_page = 1
2982         self.cached_rows = {}
2983         self.setup_ui()
2984
2985     def setup_ui(self):
2986         self.setObjectName("AuditHistoryUI")
2987         main_layout = QVBoxLayout(self)
2988         main_layout.setSpacing(12)
2989         main_layout.setContentsMargins(15, 15, 15, 15)
2990
2991
2992         filter_layout = QHBoxLayout()
2993         filter_layout.setSpacing(8)
2994
2995         filter_layout.addWidget(QLabel("时间范围:"))
2996         self.start_date = QDateEdit()
2997         self.start_date.setCalendarPopup(True)
2998         self.start_date.setDate(QDate.currentDate().addMonths(-1))
2999         self.end_date = QDateEdit()
3000         self.end_date.setCalendarPopup(True)
3001         self.end_date.setDate(QDate.currentDate())
3002         filter_layout.addWidget(self.start_date)
3003         filter_layout.addWidget(QLabel("至"))
3004         filter_layout.addWidget(self.end_date)
3005
3006         filter_layout.addWidget(QLabel("状态:"))
3007         self.status_combo = QComboBox()
3008         self.status_combo.addItem("全部", "已完成", "待复审", "已驳回"])
3009         filter_layout.addWidget(self.status_combo)
3010
3011         filter_layout.addWidget(QLabel("结论:"))
3012         self.result_combo = QComboBox()

```

```

3013         self.result_combo.addItem(["全部", "正常", "违规风险"])
3014         filter_layout.addWidget(self.result_combo)
3015
3016         self.keyword_input = QLineEdit()
3017         self.keyword_input.setPlaceholderText("搜索图片名/审核人...")
3018         self.keyword_input.setFixedWidth(160)
3019         filter_layout.addWidget(self.keyword_input)
3020
3021         self.search_btn = QPushButton("组合查询")
3022         self.search_btn.setStyleSheet("background-color: #409eff; color:
white; font-weight: bold;")
3023         filter_layout.addWidget(self.search_btn)
3024         main_layout.addLayout(filter_layout)
3025
3026
3027         self.history_table = QTableWidgetItem()
3028         self.history_table.setColumnCount(6)
3029         self.history_table.setHorizontalHeaderLabels(
3030             ["记录 ID", "图片名称", "审核归档时间", "状态", "判定结论", "快捷
操作"])
3031
3032         self.history_table.horizontalHeader().setSectionResizeMode(QHeaderView.ResizeM
ode.Stretch)
3033
3034         self.history_table.setSelectionBehavior(QTableWidgetItem.SelectionBehavior.SelectR
ows)
3035
3036         self.history_table.setEditTriggers(QTableWidgetItem.EditTrigger.NoEditTriggers)
3037
3038         main_layout.addWidget(self.history_table)
3039
3040
3041         footer_layout = QHBoxLayout()
3042         self.prev_btn = QPushButton("◀ 上一页")
3043         self.next_btn = QPushButton("下一页 ▶")
3044         self.page_label = QLabel("第 1 / 1 页")
3045         self.page_label.setStyleSheet("font-weight: bold; color:
#606266;")
3046
3047         footer_layout.addStretch()
3048         footer_layout.addWidget(self.prev_btn)
3049         footer_layout.addWidget(self.page_label)
3050         footer_layout.addWidget(self.next_btn)
3051         footer_layout.addStretch()
3052         main_layout.addLayout(footer_layout)
3053
3054
3055         self.search_btn.clicked.connect(self.emit_combined_query)
3056         self.prev_btn.clicked.connect(self.goto_prev)
3057         self.next_btn.clicked.connect(self.goto_next)
3058
3059         def emit_combined_query(self):
3060             payload = {
3061                 "start_date": self.start_date.date().toString("yyyy-MM-dd"),
3062                 "end_date": self.end_date.date().toString("yyyy-MM-dd"),

```

```

3059         "status": self.status_combo.currentText(),
3060         "result": self.result_combo.currentText(),
3061         "keyword": self.keyword_input.text().strip(),
3062         "page": self.current_page
3063     }
3064     self.query_requested.emit(payload)
3065
3066     def goto_prev(self):
3067         if self.current_page > 1:
3068             self.current_page -= 1
3069             self.emit_combined_query()
3070
3071     def goto_next(self):
3072         self.current_page += 1
3073         self.emit_combined_query()
3074
3075     def render_history_grid(self, raw_records, total_pages, active_page):
3076         self.current_page = active_page
3077         self.history_table.setRowCount(0)
3078         self.cached_rows.clear()
3079         self.page_label.setText(f"第 {active_page} / {max(1, total_pages)}
3080 页")
3081
3082         for row_idx, rec in enumerate(raw_records):
3083
3084             rec_id, file_path, audit_time, status, result, conf, text,
3085             auditor, _ = rec
3086             self.cached_rows[rec_id] = rec
3087
3088             self.history_table.insertRow(row_idx)
3089             self.history_table.setItem(row_idx, 0,
3090             QTableWidgetItem(str(rec_id)))
3091             self.history_table.setItem(row_idx, 1,
3092             QTableWidgetItem(os.path.basename(file_path)))
3093             self.history_table.setItem(row_idx, 2,
3094             QTableWidgetItem(audit_time))
3095             self.history_table.setItem(row_idx, 3,
3096             QTableWidgetItem(status))
3097
3098             res_item = QTableWidgetItem(result)
3099             if "违规" in result:
3100                 res_item.setForeground(QColor("#f56c6c"))
3101             elif "正常" in result:
3102                 res_item.setForeground(QColor("#67c23a"))
3103             self.history_table.setItem(row_idx, 4, res_item)
3104
3105             act_widget = QWidget()
3106             act_layout = QHBoxLayout(act_widget)
3107             act_layout.setContentsMargins(2, 2, 2, 2)
3108             act_layout.setSpacing(6)

```



```

3107         detail_btn = QPushButton("查看详情")
3108         del_btn = QPushButton("软删除")
3109         detail_btn.setStyleSheet("font-size: 11px; padding: 2px;
background-color: #f4f4f5;")
3110         del_btn.setStyleSheet("font-size: 11px; padding: 2px; color:
#f56c6c;")
3111
3112         act_layout.addWidget(detail_btn)
3113         act_layout.addWidget(del_btn)
3114         self.history_table.setCellWidget(row_idx, 5, act_widget)
3115
3116
3117         detail_btn.clicked.connect(lambda checked, idx=rec_id:
self.open_detail_popup(idx))
3118         del_btn.clicked.connect(lambda checked, idx=rec_id:
self.on_soft_delete_clicked(idx))
3119
3120     def open_detail_popup(self, record_id):
3121         data = self.cached_rows.get(record_id)
3122         if data:
3123             dialog = HistoryDetailDialog(self, data)
3124             dialog.appeal_requested.connect(lambda r_id:
self.appeal_api_call.emit(r_id))
3125             dialog.export_requested.connect(lambda r_id:
self.trigger_report_save_dialog(r_id))
3126             dialog.exec()
3127
3128     def trigger_report_save_dialog(self, record_id):
3129         from PyQt6.QtWidgets import QFileDialog
3130         path, _ = QFileDialog.getSaveFileName(self, "导出单条审计报告",
f"Audit_Report_{record_id}.txt",
3131                                             "Text Files (*.txt)")
3132         if path:
3133             self.report_export_call.emit(record_id, path)
3134
3135     def on_soft_delete_clicked(self, record_id):
3136         box = QMessageBox.question(self, "逻辑删除确认",
3137                                   "确认将该条记录移出历史面板吗? \n（此操作
为破坏性软删除，底层数据仍可追溯）")
3138         if box == QMessageBox.StandardButton.Yes:
3139             self.delete_requested.emit(record_id)
3140
3141     def show_toast(self, title, message, is_error=False):
3142         if is_error:
3143             QMessageBox.critical(self, title, message)
3144         else:
3145             QMessageBox.information(self, title, message)
3146 from
PyQt6.QtWidgets import (QWidget, QVBoxLayout, QHBoxLayout, QPushButton,
3147                        QLabel, QTableWidgetItem,
QHeaderView,
3148                        QProgressBar, QSpinBox, QFileDialog,
QTextEdit, QMessageBox)
3149 from PyQt6.QtCore import Qt, pyqtSignal
3150

```



```

3151
3152 class BatchImageAuditUI(QWidget):
3153     """批量图片审核控制面板 UI"""
3154
3155     add_files_requested = pyqtSignal(list)
3156     clear_list_requested = pyqtSignal()
3157     move_task_requested = pyqtSignal(int, str)
3158     start_requested = pyqtSignal(int)
3159     pause_requested = pyqtSignal()
3160     cancel_requested = pyqtSignal()
3161     save_db_requested = pyqtSignal()
3162     export_requested = pyqtSignal(str)
3163
3164     def __init__(self):
3165         super().__init__()
3166         self.setup_ui()
3167
3168     def setup_ui(self):
3169         self.setObjectName("BatchImageAuditUI")
3170         main_layout = QVBoxLayout(self)
3171         main_layout.setSpacing(12)
3172         main_layout.setContentsMargins(15, 15, 15, 15)
3173
3174         top_bar = QHBoxLayout()
3175         self.add_file_btn = QPushButton("添加图片...")
3176         self.add_folder_btn = QPushButton("添加文件夹...")
3177         self.clear_btn = QPushButton("清空队列")
3178
3179         self.up_btn = QPushButton("▲ 上移优先级")
3180         self.down_btn = QPushButton("▼ 下移优先级")
3181
3182         top_bar.addWidget(self.add_file_btn)
3183         top_bar.addWidget(self.add_folder_btn)
3184         top_bar.addWidget(self.clear_btn)
3185         top_bar.addSpacing(15)
3186         top_bar.addWidget(self.up_btn)
3187         top_bar.addWidget(self.down_btn)
3188         top_bar.addStretch()
3189
3190
3191         top_bar.addWidget(QLabel("最大并发线程:"))
3192         self.concurrency_spin = QSpinBox()
3193         self.concurrency_spin.setRange(1, 16)
3194         self.concurrency_spin.setValue(4)
3195         self.concurrency_spin.setFixedWidth(60)
3196         top_bar.addWidget(self.concurrency_spin)
3197         main_layout.addLayout(top_bar)
3198
3199
3200         workspace_layout = QHBoxLayout()
3201         workspace_layout.setSpacing(15)
3202
3203
3204

```

```

3205         self.task_table = QTableWidget()
3206         self.task_table.setColumnCount(4)
3207         self.task_table.setHorizontalHeaderLabels(["序号", "图片完整路径",
"当前状态", "检测结果"])
3208
3209 self.task_table.horizontalHeader().setSectionResizeMode(QHeaderView.ResizeMode.
Interactive)
3210         self.task_table.horizontalHeader().setSectionResizeMode(1,
QHeaderView.ResizeMode.Stretch)
3211
3212 self.task_table.setSelectionBehavior(QTableWidget.SelectionBehavior.SelectRows)
3213
3214 self.task_table.setSelectionMode(QTableWidget.SelectionMode.SingleSelection)
3215
3216 self.task_table.setEditTriggers(QTableWidget.EditTrigger.NoEditTriggers)
3217
3218 workspace_layout.addWidget(self.task_table, stretch=7)
3219
3220
3221 right_panel = QVBoxLayout()
3222 right_panel.addWidget(QLabel("异常与错误监控终端:"))
3223 self.log_terminal = QTextEdit()
3224 self.log_terminal.setReadOnly(True)
3225 self.log_terminal.setStyleSheet("background-color: #1e1e1e; color:
#f1f1f1; font-family: 'Consolas';")
3226 right_panel.addWidget(self.log_terminal)
3227 workspace_layout.addLayout(right_panel, stretch=3)
3228
3229 main_layout.addLayout(workspace_layout)
3230
3231
3232 bottom_layout = QHBoxLayout()
3233 self.progress_bar = QProgressBar()
3234 self.progress_bar.setValue(0)
3235 self.progress_bar.setTextVisible(True)
3236 bottom_layout.addWidget(self.progress_bar, stretch=6)
3237
3238
3239 self.start_btn = QPushButton("开始审核")
3240 self.pause_btn = QPushButton("暂停")
3241 self.cancel_btn = QPushButton("取消任务")
3242 self.save_btn = QPushButton("保存结果")
3243 self.export_btn = QPushButton("导出汇总表")
3244
3245
3246 self.start_btn.setStyleSheet("background-color: #409eff; color:
white; font-weight: bold;")
3247 self.pause_btn.setEnabled(False)
3248 self.cancel_btn.setEnabled(False)
3249
3250
3251 bottom_layout.addWidget(self.start_btn)
3252 bottom_layout.addWidget(self.pause_btn)
3253 bottom_layout.addWidget(self.cancel_btn)
3254 bottom_layout.addSpacing(10)
3255 bottom_layout.addWidget(self.save_btn)
3256 bottom_layout.addWidget(self.export_btn)

```

```

3250         main_layout.addLayout(bottom_layout)
3251
3252
3253         self.add_file_btn.clicked.connect(self.on_add_files)
3254         self.add_folder_btn.clicked.connect(self.on_add_folder)
3255         self.clear_btn.clicked.connect(lambda:
self.clear_list_requested.emit())
3256         self.up_btn.clicked.connect(lambda: self.on_move_priority("UP"))
3257         self.down_btn.clicked.connect(lambda:
self.on_move_priority("DOWN"))
3258         self.start_btn.clicked.connect(lambda:
self.start_requested.emit(self.concurrency_spin.value()))
3259         self.pause_btn.clicked.connect(lambda:
self.pause_requested.emit())
3260         self.cancel_btn.clicked.connect(lambda:
self.cancel_requested.emit())
3261         self.save_btn.clicked.connect(lambda:
self.save_db_requested.emit())
3262         self.export_btn.clicked.connect(self.on_export_click)
3263
3264     def on_add_files(self):
3265         files, _ = QFileDialog.getOpenFileNames(self, "选择待检测图片", "",
"Images (*.png *.jpg *.jpeg *.bmp *.webp)")
3266         if files:
3267             self.add_files_requested.emit(files)
3268
3269     def on_add_folder(self):
3270         import os
3271         folder = QFileDialog.getExistingDirectory(self, "选择图片文件夹")
3272         if folder:
3273             valid_exts = (".png", ".jpg", ".jpeg", ".bmp", ".webp")
3274             files = [os.path.join(folder, f) for f in os.listdir(folder)
if f.lower().endswith(valid_exts)]
3275             if files:
3276                 self.add_files_requested.emit(files)
3277             else:
3278                 QMessageBox.information(self, "提示", "所选文件夹内未检测到
支持的图片格式。")
3279
3280     def on_move_priority(self, direction):
3281         row = self.task_table.currentRow()
3282         if row >= 0:
3283             self.move_task_requested.emit(row, direction)
3284
3285     def on_export_click(self):
3286         path, _ = QFileDialog.getSaveFileName(self, "导出结果汇总表",
"audit_report.csv", "CSV Files (*.csv)")
3287         if path:
3288             self.export_requested.emit(path)
3289
3290
3291     def render_queue(self, task_list):
3292         """精准全量同步刷新任务视窗"""
3293         self.task_table.setRowCount(0)

```

```

3294         for idx, task in enumerate(task_list):
3295             self.task_table.insertRow(idx)
3296             self.task_table.setItem(idx, 0, QTableWidgetItem(str(idx +
1)))
3297             self.task_table.setItem(idx, 1,
QTableWidgetItem(task["path"]))
3298             self.task_table.setItem(idx, 2,
QTableWidgetItem(task["status"]))
3299             self.task_table.setItem(idx, 3,
QTableWidgetItem(task["result"]))
3300
3301
3302             if "违规" in task["result"]:
3303                 self.task_table.item(idx,
3).setForeground(Qt.GlobalColor.red)
3304             elif "正常" in task["result"]:
3305                 self.task_table.item(idx,
3).setForeground(Qt.GlobalColor.darkGreen)
3306
3307     def select_row(self, row_index):
3308         if 0 <= row_index < self.task_table.rowCount():
3309             self.task_table.setCurrentCell(row_index, 0)
3310
3311     def update_progress(self, current, total):
3312         self.progress_bar.setMaximum(total if total > 0 else 1)
3313         self.progress_bar.setValue(current)
3314
3315     def append_log(self, text, is_error=False):
3316         color_hex = "#ff6b6b" if is_error else "#67c23a"
3317         log_entry = f"<span style='color:{color_hex};'>[{is_error and
'ERROR' or 'INFO'}]</span> {text}"
3318         self.log_terminal.append(log_entry)
3319
3320     def update_controls_by_state(self, state: str):
3321         """根据后端状态机流转进行严格的前端防呆互斥"""
3322         is_running = state == "RUNNING"
3323         is_paused = state == "PAUSED"
3324         is_idle = state == "IDLE"
3325
3326         self.start_btn.setEnabled(is_idle or is_paused)
3327         self.pause_btn.setEnabled(is_running)
3328         self.cancel_btn.setEnabled(is_running or is_paused)
3329
3330         self.add_file_btn.setEnabled(is_idle)
3331         self.add_folder_btn.setEnabled(is_idle)
3332         self.clear_btn.setEnabled(is_idle)
3333         self.up_btn.setEnabled(is_idle)
3334         self.down_btn.setEnabled(is_idle)
3335         self.concurrency_spin.setEnabled(is_idle)
3336
3337     def show_message(self, title, msg, is_error=False):
3338         if is_error:
3339             QMessageBox.critical(self, title, msg)
3340         else:
3341             QMessageBox.information(self, title, msg)
3342

```

```

PyQt6.QtWidgets import (QWidget, QVBoxLayout, QTableWidgetItem, QTableWidgetItem,
3343                     QPushButton, QHBoxLayout, QProgressBar,
QFileDialog,
3344                     QMessageBox, QHeaderView)
3345 from PyQt6.QtCore import Qt
3346 import os
3347 import shutil
3348 from modules.data_backup import BackupWorker
3349
3350 class DataBackupUI(QWidget):
3351     def __init__(self, db_path, img_path):
3352         super().__init__()
3353         self.db_path = db_path
3354         self.img_path = img_path
3355         self.backup_dir = "./data/backups"
3356         self.setWindowTitle("数据备份与还原")
3357         self.resize(700, 500)
3358         self.setup_ui()
3359         self.refresh_backup_list()
3360
3361     def setup_ui(self):
3362         layout = QVBoxLayout(self)
3363
3364
3365         self.table = QTableWidgetItem(0, 3)
3366         self.table.setHorizontalHeaderLabels(["文件名", "修改时间", "大小"])
3367
3368         self.table.horizontalHeader().setSectionResizeMode(QHeaderView.ResizeMode.Stretch)
3369
3370         layout.addWidget(self.table)
3371
3372         self.progress = QProgressBar()
3373         layout.addWidget(self.progress)
3374
3375         btn_layout = QHBoxLayout()
3376         self.backup_btn = QPushButton("手动备份")
3377         self.restore_btn = QPushButton("还原选中")
3378         self.del_btn = QPushButton("删除过期")
3379
3380         for btn in [self.backup_btn, self.restore_btn, self.del_btn]:
3381             btn_layout.addWidget(btn)
3382
3383         layout.addLayout(btn_layout)
3384
3385         self.backup_btn.clicked.connect(self.start_backup)
3386         self.restore_btn.clicked.connect(self.start_restore)
3387         self.del_btn.clicked.connect(self.delete_backup)
3388
3389     def refresh_backup_list(self):
3390         """刷新备份列表（从固定备份目录读取）"""
3391         self.table.setRowCount(0)
3392         if not os.path.exists(self.backup_dir):

```

```

3393         os.makedirs(self.backup_dir, exist_ok=True)
3394         return
3395
3396     for f in os.listdir(self.backup_dir):
3397         if f.endswith(".zip"):
3398             row = self.table.rowCount()
3399             self.table.insertRow(row)
3400             self.table.setItem(row, 0, QTableWidgetItem(f))
3401             full_path = os.path.join(self.backup_dir, f)
3402             stat = os.stat(full_path)
3403             mtime = os.path.getmtime(full_path)
3404             from datetime import datetime
3405             mtime_str = datetime.fromtimestamp(mtime).strftime("%Y-
3406             self.table.setItem(row, 1, QTableWidgetItem(mtime_str))
3407             size_kb = stat.st_size / 1024
3408             self.table.setItem(row, 2,
3409             QTableWidgetItem(f"{size_kb:.1f} KB"))
3410
3411     def start_backup(self):
3412         """备份到固定目录，不再弹出文件夹选择对话框"""
3413         os.makedirs(self.backup_dir, exist_ok=True)
3414
3415         self.worker = BackupWorker('backup', self.backup_dir,
3416         [self.db_path, self.img_path])
3417         self.worker.progress.connect(self.update_progress)
3418         self.worker.finished.connect(self.on_backup_finished)
3419         self.worker.start()
3420         self.backup_btn.setEnabled(False)
3421         self.progress.setValue(0)
3422
3423     def on_backup_finished(self, success, msg):
3424         self.backup_btn.setEnabled(True)
3425         if success:
3426             QMessageBox.information(self, "备份成功", f"备份文件已保存到：
3427             \n{os.path.join(self.backup_dir, msg)}")
3428             self.refresh_backup_list()
3429         else:
3430             QMessageBox.critical(self, "备份失败", msg)
3431             self.progress.setValue(0)
3432
3433     def start_restore(self):
3434         curr = self.table.currentRow()
3435         if curr < 0:
3436             QMessageBox.warning(self, "提示", "请先选中一个备份文件")
3437             return
3438         file_name = self.table.item(curr, 0).text()
3439         backup_path = os.path.join(self.backup_dir, file_name)
3440
3441         reply = QMessageBox.question(self, "确认还原",
3442         "还原将覆盖现有的数据库和图片文件，确定
3443         要继续吗？\n 建议先手动备份当前数据。",
3444         QMessageBox.StandardButton.Yes |

```

```

QMessageBox.StandardButton.No)
3442         if reply != QMessageBox.StandardButton.Yes:
3443             return
3444
3445
3446         target_root = "./data"
3447         self.worker = BackupWorker('restore', target_root, [],
backup_file=backup_path)
3448         self.worker.progress.connect(self.update_progress)
3449         self.worker.finished.connect(self.on_restore_finished)
3450         self.worker.start()
3451         self.restore_btn.setEnabled(False)
3452
3453     def on_restore_finished(self, success, msg):
3454         self.restore_btn.setEnabled(True)
3455         if success:
3456             QMessageBox.information(self, "还原成功", "数据已恢复, 请重启程
序以完全生效。")
3457         else:
3458             QMessageBox.critical(self, "还原失败", msg)
3459         self.progress.setValue(0)
3460
3461     def delete_backup(self):
3462         curr = self.table.currentRow()
3463         if curr < 0:
3464             QMessageBox.warning(self, "提示", "请先选中一个备份文件")
3465             return
3466         file_name = self.table.item(curr, 0).text()
3467         full_path = os.path.join(self.backup_dir, file_name)
3468         reply = QMessageBox.question(self, "确认删除", f"确定要删除备份文件
{file_name} 吗? ",
3469                                     QMessageBox.StandardButton.Yes |
QMessageBox.StandardButton.No)
3470         if reply == QMessageBox.StandardButton.Yes:
3471             try:
3472                 os.remove(full_path)
3473                 self.refresh_backup_list()
3474                 QMessageBox.information(self, "删除成功", "备份文件已删除")
3475             except Exception as e:
3476                 QMessageBox.critical(self, "删除失败", str(e))
3477
3478     def update_progress(self, val, msg):
3479         self.progress.setValue(val)
3480         self.progress.setFormat(msg)3481 from PyQt6.QtWidgets import
(QWidget, QVBoxLayout, QHBoxLayout, QSplitter,
3482                                     QTreeWidget, QTreeWidgetItem, QTextBrowser,
3483                                     QLineEdit, QPushButton, QScrollArea, QLabel)
3484 from PyQt6.QtCore import Qt, QSize
3485 from modules.help_center import HelpCenterModule
3486
3487 class HelpCenterUI(QWidget):
3488     def __init__(self):
3489         super().__init__()
3490         self.logic = HelpCenterModule()

```

```

3491         self.setWindowTitle("系统帮助中心")
3492         self.setup_ui()
3493         self.load_manual_tree()
3494
3495     def setup_ui(self):
3496         main_layout = QHBoxLayout(self)
3497         splitter = QSplitter(Qt.Orientation.Horizontal)
3498
3499
3500         left_widget = QWidget()
3501         left_layout = QVBoxLayout(left_widget)
3502         self.search_input = QLineEdit()
3503         self.search_input.setPlaceholderText("搜索帮助文档...")
3504         self.search_input.returnPressed.connect(self.handle_search)
3505         self.tree = QTreeWidget()
3506         self.tree.setHeaderLabel("文档目录")
3507         self.tree.itemClicked.connect(self.on_tree_click)
3508         left_layout.addWidget(self.search_input)
3509         left_layout.addWidget(self.tree)
3510
3511
3512         self.browser = QTextBrowser()
3513         self.browser.setOpenExternalLinks(True)
3514
3515         splitter.addWidget(left_widget)
3516         splitter.addWidget(self.browser)
3517         splitter.setStretchFactor(1, 3)
3518         main_layout.addWidget(splitter)
3519
3520     def load_manual_tree(self):
3521         """递归加载 JSON 目录到树形控件"""
3522         data = self.logic.get_manual_index()
3523         self._add_tree_items(self.tree.invisibleRootItem(), data)
3524
3525     def _add_tree_items(self, parent, items):
3526         for item in items:
3527             node = QTreeWidgetItem(parent, [item["title"]])
3528             if "file" in item:
3529                 node.setData(0, Qt.ItemDataRole.UserRole, item["file"])
3530             if "children" in item:
3531                 self._add_tree_items(node, item["children"])
3532
3533     def on_tree_click(self, item):
3534         file_name = item.data(0, Qt.ItemDataRole.UserRole)
3535         if file_name:
3536             content = self.logic.load_content(file_name)
3537             self.browser.setHtml(content)
3538
3539     def handle_search(self):
3540         keyword = self.search_input.text()
3541         if not keyword: return
3542         results = self.logic.search_docs(keyword)
3543         if results:
3544             self.browser.setHtml(f"<h3>搜索结果: {keyword}</h3><p>找到相关

```



```

文档: {'', '.join(results)}</p>")
3545         else:
3546             self.browser.setText("未找到相关内容。")3547
3548 import random
3549 import string
3550 from PyQt6.QtWidgets import (QWidget, QVBoxLayout, QHBoxLayout, QLabel,
3551                               QLineEdit, QPushButton, QCheckBox, QFrame,
3552                               QGraphicsDropShadowEffect, QProgressBar,
QMessageBox, QInputDialog)
3553 from PyQt6.QtGui import QPainter, QColor, QFont, QPen, QLinearGradient,
QAction, QIcon
3554 from PyQt6.QtCore import Qt, pyqtSignal, QPoint, QTimer
3555
3556 class CaptchaWidget(QWidget):
3557     """手绘级图形验证码组件，包含多重随机噪点与混淆扭曲特征"""
3558     def __init__(self, parent=None):
3559         super().__init__(parent)
3560         self.setFixedSize(110, 42)
3561         self.captcha_text = ""
3562         self.update_captcha()
3563
3564     def update_captcha(self):
3565
3566         pool = "23456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ"
3567         self.captcha_text = "".join(random.choice(pool) for _ in range(4))
3568         self.update()
3569
3570     def mousePressEvent(self, event):
3571         if event.button() == Qt.MouseButton.LeftButton:
3572             self.update_captcha()
3573
3574     def paintEvent(self, event):
3575         painter = QPainter(self)
3576         painter.setRenderHint(QPainter.RenderHint.Antialiasing)
3577
3578
3579         gradient = QLinearGradient(0, 0, self.width(), self.height())
3580         gradient.setColorAt(0, QColor("#E2E8F0"))
3581         gradient.setColorAt(1, QColor("#F1F5F9"))
3582         painter.fillRect(self.rect(), gradient)
3583
3584
3585         for _ in range(5):
3586             pen = QPen(QColor(random.randint(120, 200),
random.randint(120, 200), random.randint(120, 200)), 1)
3587             painter.setPen(pen)
3588             painter.drawLine(
3589                 random.randint(0, self.width()), random.randint(0,
self.height()),
3590                 random.randint(0, self.width()), random.randint(0,
self.height())
3591             )
3592
3593
3594         for _ in range(120):

```

```

3595         painter.setPen(QColor(random.randint(150, 220),
random.randint(150, 220), random.randint(150, 220)))
3596         painter.drawPoint(random.randint(0, self.width()),
random.randint(0, self.height()))
3597
3598
3599         font_families = ["Arial", "Verdana", "Times New Roman", "Courier
New"]
3600         for i, char in enumerate(self.captcha_text):
3601             font = QFont(random.choice(font_families), random.randint(16,
20), QFont.Weight.Bold)
3602             painter.setFont(font)
3603
3604
3605             pen = QPen(QColor(random.randint(20, 100), random.randint(20,
100), random.randint(20, 100)), 2)
3606             painter.setPen(pen)
3607
3608
3609             x = 12 + i * 22 + random.randint(-3, 3)
3610             y = 28 + random.randint(-4, 4)
3611
3612
3613             painter.save()
3614             painter.translate(x, y)
3615             painter.rotate(random.randint(-15, 15))
3616             painter.drawText(QPoint(0, 0), char)
3617             painter.restore()
3618
3619
3620 class LoginUI(QWidget):
3621     """高颜值美观、现代渐变质感登录视图界面层"""
3622     login_requested = pyqtSignal(str, str, str, bool)
3623     register_clicked = pyqtSignal()
3624     forgot_password_requested = pyqtSignal(str, str)
3625
3626     def __init__(self, parent=None):
3627         super().__init__(parent)
3628         self.setWindowTitle("IAVDS 智能安全认证中心")
3629         self.setFixedSize(450, 640)
3630         self._password_visible = False
3631         self._init_layout()
3632
3633     def _init_layout(self):
3634
3635         main_layout = QVBoxLayout(self)
3636         main_layout.setContentsMargins(0, 0, 0, 0)
3637         main_layout.setSpacing(0)
3638
3639
3640         background_frame = QFrame(self)
3641         background_frame.setObjectName("MainFrame")
3642         background_frame.setStyleSheet("""
3643             QFrame#MainFrame {
3644                 background: qlineargradient(x1:0, y1:0, x2:1, y2:1,

```

```

stop:0 #1E293B, stop:1 #0F172A);
3645     }
3646     "")
3647     card_layout = QVBoxLayout(background_frame)
3648     card_layout.setContentsMargins(40, 45, 40, 45)
3649     card_layout.setSpacing(18)
3650
3651
3652     logo_container = QHBoxLayout()
3653     logo_container.setAlignment(Qt.AlignmentFlag.AlignCenter)
3654     self.logo_label = QLabel("IAVDS")
3655     self.logo_label.setAlignment(Qt.AlignmentFlag.AlignCenter)
3656     self.logo_label.setFixedSize(70, 70)
3657     self.logo_label.setStyleSheet("""
3658         QLabel {
3659             background: qlineargradient(x1:0, y1:0, x2:1, y2:1,
stop:0 #60A5FA, stop:1 #3B82F6);
3660             color: #FFFFFF;
3661             font-size: 18px;
3662             font-weight: 900;
3663             border-radius: 20px;
3664         }
3665     """)
3666
3667     shadow = QGraphicsDropShadowEffect()
3668     shadow.setBlurRadius(25)
3669     shadow.setColor(QColor(59, 130, 246, 160))
3670     shadow.setOffset(0, 6)
3671     self.logo_label.setGraphicsEffect(shadow)
3672     logo_container.addWidget(self.logo_label)
3673     card_layout.addLayout(logo_container)
3674
3675
3676     title_label = QLabel("图片内容智能审核系统")
3677     title_label.setAlignment(Qt.AlignmentFlag.AlignCenter)
3678     title_label.setStyleSheet("color: #F8FAFC; font-size: 19px; font-
weight: 700; letter-spacing: 1px;")
3679     sub_title = QLabel("INTELLIGENT AUDIT PLATFORM")
3680     sub_title.setAlignment(Qt.AlignmentFlag.AlignCenter)
3681     sub_title.setStyleSheet("color: #64748B; font-size: 10px; font-
weight: 600; letter-spacing: 2px;")
3682     card_layout.addWidget(title_label)
3683     card_layout.addWidget(sub_title)
3684     card_layout.addSpacing(10)
3685
3686
3687     input_style = ""
3688     QLineEdit {
3689         background-color: #334155;
3690         color: #F8FAFC;
3691         border: 1px solid #475569;
3692         border-radius: 8px;
3693         padding: 11px 14px;
3694         font-size: 13px;
3695     }

```

```

3696         QLineEdit:focus {
3697             border: 1px solid #3B82F6;
3698             background-color: #1E293B;
3699         }
3700     """
3701
3702
3703     self.username_input = QLineEdit()
3704     self.username_input.setPlaceholderText("请输入账号")
3705     self.username_input.setStyleSheet(input_style)
3706     card_layout.addWidget(self.username_input)
3707
3708
3709     self.password_input = QLineEdit()
3710     self.password_input.setPlaceholderText("请输入登录密码")
3711     self.password_input.setEchoMode(QLineEdit.EchoMode.Password)
3712     self.password_input.setStyleSheet(input_style)
3713
3714
3715     self.toggle_action = QAction(self)
3716     self.toggle_action.setText("🔒")
3717     self.password_input.addAction(self.toggle_action,
3718     QLineEdit.ActionPosition.TrailingPosition)
3719     self.toggle_action.triggered.connect(self._toggle_password_view)
3720     card_layout.addWidget(self.password_input)
3721
3722
3723     captcha_layout = QHBoxLayout()
3724     captcha_layout.setSpacing(12)
3725     self.captcha_input = QLineEdit()
3726     self.captcha_input.setPlaceholderText("请输入验证码")
3727     self.captcha_input.setStyleSheet(input_style)
3728
3729     self.captcha_widget = CaptchaWidget()
3730     captcha_layout.addWidget(self.captcha_input, 3)
3731     captcha_layout.addWidget(self.captcha_widget, 2)
3732     card_layout.addLayout(captcha_layout)
3733
3734
3735     utils_layout = QHBoxLayout()
3736     self.remember_cb = QCheckBox("记住密码")
3737     self.remember_cb.setStyleSheet("""
3738         QCheckBox { color: #94A3B8; font-size: 12px; }
3739         QCheckBox::indicator { width: 14px; height: 14px; border-
3740         radius: 3px; border: 1px solid #475569; }
3741         QCheckBox::indicator:checked { background-color: #3B82F6;
3742         border: 1px solid #3B82F6; }
3743     """)
3744
3745     self.forgot_btn = QPushButton("忘记密码? ")
3746     self.forgot_btn.setStyleSheet("QPushButton { color: #94A3B8;
3747     border: none; font-size: 12px; text-align: right; } QPushButton:hover { color:
3748     #60A5FA; }")
3749     self.forgot_btn.setCursor(Qt.CursorShape.PointingHandCursor)

```

```

3745         self.forgot_btn.clicked.connect(self._on_forgot_password)
3746
3747         utils_layout.addWidget(self.remember_cb)
3748         utils_layout.addWidget(self.forgot_btn)
3749         card_layout.addLayout(utils_layout)
3750         card_layout.addSpacing(6)
3751
3752
3753         self.progress_bar = QProgressBar()
3754         self.progress_bar.setRange(0, 100)
3755         self.progress_bar.setValue(0)
3756         self.progress_bar.setTextVisible(False)
3757         self.progress_bar.setFixedHeight(3)
3758         self.progress_bar.setStyleSheet("""
3759             QProgressBar { background: #334155; border: none; border-
radius: 1px; }
3760             QProgressBar::chunk { background: qlineargradient(x1:0, y1:0,
x2:1, y2:0, stop:0 #60A5FA, stop:1 #3B82F6); }
3761         """)
3762         self.progress_bar.hide()
3763
3764         self.status_label = QLabel("")
3765         self.status_label.setStyleSheet("color: #60A5FA; font-size:
11px;")
3766         self.status_label.setAlignment(Qt.AlignmentFlag.AlignCenter)
3767         self.status_label.hide()
3768
3769         card_layout.addWidget(self.progress_bar)
3770         card_layout.addWidget(self.status_label)
3771
3772         # 8. 确认登录大按钮
3773         self.login_btn = QPushButton("登 录 系 统")
3774         self.login_btn.setFixedHeight(44)
3775         self.login_btn.setCursor(Qt.CursorShape.PointingHandCursor)
3776         self.login_btn.setStyleSheet("""
3777             QPushButton {
3778                 background: qlineargradient(x1:0, y1:0, x2:1, y2:0,
stop:0 #3B82F6, stop:1 #2563EB);
3779                 color: white;
3780                 font-size: 14px;
3781                 font-weight: 700;
3782                 border: none;
3783                 border-radius: 8px;
3784             }
3785             QPushButton:hover {
3786                 background: qlineargradient(x1:0, y1:0, x2:1, y2:0,
stop:0 #60A5FA, stop:1 #3B82F6);
3787             }
3788             QPushButton:pressed {
3789                 background: #1D4ED8;
3790             }
3791         """)
3792         self.login_btn.clicked.connect(self._on_login_clicked)
3793         card_layout.addWidget(self.login_btn)
3794

```

```

3795         # 9. 导流前往注册区
3796         self.register_btn = QPushButton("尚未拥有系统通行权限? 申请注册")
3797         self.register_btn.setStyleSheet("QPushButton { color: #64748B;
border: none; font-size: 12px; } QPushButton:hover { color: #3B82F6; }")
3798         self.register_btn.setCursor(Qt.CursorShape.PointingHandCursor)
3799         self.register_btn.clicked.connect(self.register_clicked.emit)
3800         card_layout.addWidget(self.register_btn)
3801
3802         main_layout.addWidget(background_frame)
3803
3804     def _toggle_password_view(self):
3805         self._password_visible = not self._password_visible
3806         if self._password_visible:
3807             self.password_input.setEchoMode(QLineEdit.EchoMode.Normal)
3808             self.toggle_action.setText("🔒")
3809         else:
3810             self.password_input.setEchoMode(QLineEdit.EchoMode.Password)
3811             self.toggle_action.setText("👁")
3812
3813     def _on_login_clicked(self):
3814         username = self.username_input.text().strip()
3815         password = self.password_input.text()
3816         captcha = self.captcha_input.text().strip()
3817         remember = self.remember_cb.isChecked()
3818         self.login_requested.emit(username, password, captcha, remember)
3819
3820     def _on_forgot_password(self):
3821         username = self.username_input.text().strip()
3822         if not username:
3823             QMessageBox.warning(self, "提示", "请先在账号输入框中填写您需要
找回的用户名!")
3824         return
3825
3826         email, ok = QInputDialog.getText(self, "找回密码", "请输入绑定该账
号的安全邮箱地址:")
3827         if ok and email.strip():
3828             self.forgot_password_requested.emit(username, email.strip())
3829
3830     def set_loading_state(self, is_loading: bool, status_text: str = ""):
3831         """平滑载入控制逻辑, 防止重复提交"""
3832         if is_loading:
3833             self.login_btn.setEnabled(False)
3834             self.progress_bar.show()
3835             self.progress_bar.setValue(35)
3836             self.status_label.setText(status_text)
3837             self.status_label.show()
3838         else:
3839             self.login_btn.setEnabled(True)
3840             self.progress_bar.setValue(100)
3841             QTimer.singleShot(400, lambda: (self.progress_bar.hide(),
self.status_label.hide()))
3842
3843     def fill_saved_credentials(self, username: str, password_raw: str,
checked: bool):

```

```

3844         self.username_input.setText(username)
3845         self.password_input.setText(password_raw)
3846         self.remember_cb.setChecked(checkered)
3847
3848 import configparser
3849 import traceback
3850 from PyQt6.QtCore import Qt, QSize
3851 from PyQt6.QtWidgets import (QMainWindow, QStackedWidget, QWidget,
3852                             QHBoxLayout, QVBoxLayout, QLabel, QFrame,
3853                             QStatusBar,
3854                             QListWidget, QListWidgetItem)
3855
3856 import ui
3857 import modules
3858
3859 class MainWindowUI(QMainWindow):
3860     def __init__(self, username: str, role: str):
3861         super().__init__()
3862         self.username = username
3863         self.role = role
3864         self.modules = {}
3865         self.registry = [
3866             ("数据统计", "StatisticsReportUI", "StatisticsModule"),
3867             ("单张审核", "SingleImageAuditUI", "SingleImageAuditModule"),
3868             ("批量审核", "BatchImageAuditUI", "BatchImageAuditModule"),
3869             ("审核历史", "AuditHistoryUI", "AuditHistoryModule"),
3870             ("敏感词管理", "SensitiveWordUI", "SensitiveWordModule"),
3871             ("用户管理", "UserManagementUI", "UserManagementModule"),
3872             ("申诉处理", "AppealHandlingUI", "AppealHandlingModule"),
3873             ("模型配置", "ModelConfigUI", "ModelConfigModule"),
3874             ("数据备份", "DataBackupUI", "BackupWorker"),
3875             ("帮助中心", "HelpCenterUI", "HelpCenterModule"),
3876
3877         ]
3878         self.setWindowTitle("图片内容智能审核与违规检测系统")
3879         self.setMinimumSize(1320, 860)
3880         self.resize(1480, 920)
3881         self.setStyleSheet("QMainWindow { background-color: #F8FAFC; }")
3882         self.colors = ["#3B82F6", "#10B981", "#F59E0B", "#EF4444",
3883 "#8B5CF6", "#EC4899", "#6366F1"]
3884         self._build()
3885
3886     def _build(self):
3887         root = QWidget()
3888         self.setCentralWidget(root)
3889         outer = QVBoxLayout(root)
3890         outer.setContentsMargins(0, 0, 0, 0)
3891         outer.setSpacing(0)
3892
3893         header = QFrame()
3894         header.setFixedHeight(62)
3895         header.setStyleSheet("QFrame { background: #FFFFFF; border-bottom:

```

```

1px solid #E2E8F0; }")
3896         hb = QHBoxLayout(header)
3897         hb.setContentsMargins(24, 0, 24, 0)
3898
3899         logo = QFrame()
3900         logo.setFixedSize(36, 36)
3901         logo.setStyleSheet("QFrame { background:
qlineargradient(x1:0,y1:0,x2:1,y2:1, stop:0 #60A5FA, stop:1 #3B82F6); border-
radius: 8px; }")
3902         ll = QVBoxLayout(logo)
3903         ll.setContentsMargins(0, 0, 0, 0)
3904         lt = QLabel("审")
3905         lt.setAlignment(Qt.AlignmentFlag.AlignCenter)
3906         lt.setStyleSheet("color: white; font-size: 15px; font-weight:
800;")
3907         ll.addWidget(lt)
3908
3909         title_col = QVBoxLayout()
3910         t1 = QLabel("图片内容智能审核与违规检测系统")
3911         t1.setStyleSheet("color: #0F172A; font-size: 14px; font-weight:
700;")
3912         self.breadcrumb = QLabel("当前工作区 · 数据统计")
3913         self.breadcrumb.setStyleSheet("color: #64748B; font-size: 11px;")
3914         title_col.addWidget(t1)
3915         title_col.addWidget(self.breadcrumb)
3916
3917         hb.addWidget(logo)
3918         hb.addSpacing(12)
3919         hb.addLayout(title_col)
3920         hb.addStretch()
3921
3922         self.module_chip = QLabel("数据统计")
3923         self._set_chip_color("#3B82F6")
3924
3925         user_chip = QLabel(f" 👤 {self.username} ({self.role}) ")
3926         user_chip.setStyleSheet("QLabel { color: #0F172A; background:
#F1F5F9; border: 1px solid #E2E8F0; border-radius: 14px; padding: 6px 14px;
font-size: 12px; font-weight: 500; }")
3927         hb.addWidget(self.module_chip)
3928         hb.addWidget(user_chip)
3929         outer.addWidget(header)
3930
3931
3932         body = QHBoxLayout()
3933         body.setContentsMargins(16, 16, 16, 16)
3934         body.setSpacing(16)
3935
3936
3937         nav_panel = QFrame()
3938         nav_panel.setFixedWidth(240)
3939         nav_panel.setStyleSheet("QFrame { background: #FFFFFF; border:
1px solid #E2E8F0; border-radius: 12px; }")
3940         nl = QVBoxLayout(nav_panel)
3941         nl.setContentsMargins(12, 16, 12, 16)

```



```

3942
3943     self.sidebar = QListWidget()
3944     self.sidebar.setSpacing(6)
3945     self.sidebar.setStyleSheet("""
3946         QListWidget { border: none; background: transparent; outline:
none; }
3947         QListWidget::item { padding: 12px 16px; border-radius: 8px;
color: #0F172A; font-size: 13px; font-weight: 500; }
3948         QListWidget::item: hover { background-color: #EFF6FF; color:
#3B82F6; }
3949         QListWidget::item:selected { background-color: #3B82F6; color:
white; font-weight: bold; }
3950     """)
3951     for name, _, _ in self.registry:
3952         item = QListWidgetItem(name)
3953         item.setSizeHint(QSize(0, 45))
3954         self.sidebar.addItem(item)
3955
3956     about_item = QListWidgetItem("关于系统")
3957     about_item.setSizeHint(QSize(0, 45))
3958     self.sidebar.addItem(about_item)
3959
3960     self.sidebar.currentRowChanged.connect(self._switch_page)
3961     self.sidebar.itemClicked.connect(self._on_item_clicked)
3962
3963     nl.addWidget(self.sidebar)
3964
3965
3966     content_panel = QFrame()
3967     content_panel.setStyleSheet("QFrame { background: #FFFFFF; border:
1px solid #E2E8F0; border-radius: 12px; }")
3968     cpl = QVBoxLayout(content_panel)
3969     cpl.setContentsMargins(0, 0, 0, 0)
3970     self.stack = QStackedWidget()
3971     cpl.addWidget(self.stack)
3972
3973     body.addWidget(nav_panel)
3974     body.addWidget(content_panel, 1)
3975     outer.addLayout(body, 1)
3976
3977
3978     sb = QStatusBar()
3979     self.setStatusBar(sb)
3980     sb.setStyleSheet("QStatusBar { background: #FFFFFF; border-top:
1px solid #E2E8F0; color: #64748B; }")
3981     sb.showMessage("系统就绪 · 网络安全护航中")
3982
3983
3984     config = configparser.ConfigParser()
3985     config.read("data/config.ini", encoding='utf-8')
3986     db_path = config.get('database', 'path', fallback='data/app.db')
3987     img_path = config.get('storage', 'path', fallback='data/images')
3988
3989     for name, ui_str, mod_str in self.registry:
3990         ui_cls = getattr(ui, ui_str, None) if ui_str else None

```

```

3991         mod_cls = getattr(modules, mod_str, None) if mod_str else
None
3992
3993
3994         if ui_cls is None:
3995
3996             ui_inst = QWidget()
3997             layout = QVBoxLayout(ui_inst)
3998             layout.addWidget(QLabel(f"未找到组件 {ui_str}"))
3999             print(f"✗ 未找到 UI 类: {ui_str}")
4000             self.stack.addWidget(ui_inst)
4001             continue
4002
4003         if name == "数据备份":
4004             ui_inst = ui_cls(db_path, img_path)
4005         else:
4006             ui_inst = ui_cls()
4007
4008
4009         if mod_cls:
4010             try:
4011                 if name == "用户管理":
4012                     logic_inst = mod_cls(ui_inst, self.username,
self.role)
4013                 elif name == "申诉处理":
4014                     logic_inst = mod_cls(ui_inst,
current_user=self.username)
4015                 else:
4016                     logic_inst = mod_cls(ui_inst)
4017
4018                 self.modules[name] = logic_inst
4019                 print(f"✓ 模块 {name} 加载成功 (逻辑类: {mod_str})")
4020             except Exception as e:
4021                 print(f"✗ 模块 {name} 加载失败: {e}")
4022                 traceback.print_exc()
4023             else:
4024
4025
4026                 print(f"✓ 模块 {name} 加载成功 (无逻辑层)")
4027
4028                 self.stack.addWidget(ui_inst)
4029
4030         self.sidebar.setCurrentRow(0)
4031
4032         def _set_chip_color(self, color: str):
4033             self.module_chip.setStyleSheet(f"QLabel {{ color: white;
background: {color}; border-radius: 14px; padding: 6px 16px; font-size: 12px;
font-weight: 700; }}")
4034
4035         def _on_item_clicked(self, item):
4036             if item.text() == "关于系统":
4037                 from modules.about_system import AboutSystemModule
4038                 from ui.about_system_ui import AboutSystemUI
4039                 logic = AboutSystemModule()

```

```

4040         dialog = AboutSystemUI(logic)
4041         dialog.exec()
4042     else:
4043         row = self.sidebar.row(item)
4044         if row < len(self.registry):
4045             self._switch_page(row)
4046
4047     def _switch_page(self, index: int):
4048
4049         if index >= len(self.registry):
4050             return
4051         if 0 <= index < self.stack.count():
4052             self.stack.setCurrentIndex(index)
4053             name = self.registry[index][0]
4054             color = self.colors[index % len(self.colors)]
4055             self.module_chip.setText(name)
4056             self._set_chip_color(color)
4057             self.breadcrumb.setText(f"当前工作区 · {name}")
4058             self.statusBar().showMessage(f"已切换至模块: {name}")
4059
4060 from PyQt6.QtWidgets import (QWidget, QVBoxLayout, QHBoxLayout, QPushButton,
4061                               QLabel, QSlider, QCheckBox, QGroupBox,
4062                               QFormLayout,
4063                               QMessageBox)
4064 from PyQt6.QtCore import Qt, pyqtSignal
4065
4066 class ModelConfigUI(QWidget):
4067     """模型配置面板的视图层"""
4068
4069     save_requested = pyqtSignal(dict)
4070     reset_requested = pyqtSignal()
4071
4072     def __init__(self):
4073         super().__init__()
4074         self.is_dirty = False
4075         self.setup_ui()
4076         self.bind_dirty_trackers()
4077
4078     def setup_ui(self):
4079         self.setWindowTitle("检测模型配置")
4080         main_layout = QVBoxLayout(self)
4081         main_layout.setSpacing(15)
4082         main_layout.setContentsMargins(20, 20, 20, 20)
4083
4084         header_layout = QHBoxLayout()
4085         self.title_label = QLabel("<b>核心检测引擎参数配置</b>")
4086         self.title_label.setStyleSheet("font-size: 16px;")
4087
4088         self.last_update_label = QLabel("生效时间: 未知")
4089         self.last_update_label.setStyleSheet("color: #909399;")
4090
4091         header_layout.addWidget(self.title_label)

```

```
4093         header_layout.addStretch()
4094         header_layout.addWidget(self.last_update_label)
4095         main_layout.addLayout(header_layout)
4096
4097
4098         threshold_group = QGroupBox("算法置信度阈值 (高于此值判定为违规)")
4099         threshold_group.setStyleSheet(
4100             "QGroupBox { font-weight: bold; border: 1px solid #dcdfe6;
border-radius: 5px; margin-top: 10px; } QGroupBox::title { subcontrol-origin:
margin; left: 10px; padding: 0 3px; }")
4101         t_layout = QFormLayout(threshold_group)
4102         t_layout.setSpacing(15)
4103         t_layout.setContentsMargins(15, 25, 15, 15)
4104
4105
4106         self.porn_slider, self.porn_val_label = self.create_slider()
4107         t_layout.addRow("色情内容检测 (Porn):",
self.wrap_slider(self.porn_slider, self.porn_val_label))
4108
4109
4110         self.violence_slider, self.violence_val_label =
self.create_slider()
4111         t_layout.addRow("暴力血腥检测 (Violence):",
self.wrap_slider(self.violence_slider, self.violence_val_label))
4112
4113
4114         self.politics_slider, self.politics_val_label =
self.create_slider()
4115         t_layout.addRow("政治敏感检测 (Political):",
self.wrap_slider(self.politics_slider, self.politics_val_label))
4116
4117         main_layout.addWidget(threshold_group)
4118
4119
4120         feature_group = QGroupBox("辅助检测链路控制")
4121         feature_group.setStyleSheet(
4122             "QGroupBox { font-weight: bold; border: 1px solid #dcdfe6;
border-radius: 5px; margin-top: 10px; } QGroupBox::title { subcontrol-origin:
margin; left: 10px; padding: 0 3px; }")
4123         f_layout = QVBoxLayout(feature_group)
4124         f_layout.setSpacing(10)
4125         f_layout.setContentsMargins(15, 25, 15, 15)
4126
4127         self.ocr_checkbox = QCheckBox("启用 OCR 图文检测 (提取图片文本进行敏
感词匹配, 会增加耗时)")
4128         self.hash_checkbox = QCheckBox("启用图片哈希查重 (拦截已被标记为违规
的相似图片)")
4129
4130         f_layout.addWidget(self.ocr_checkbox)
4131         f_layout.addWidget(self.hash_checkbox)
4132         main_layout.addWidget(feature_group)
4133
4134         main_layout.addStretch()
4135
```

```

4136
4137     btn_layout = QHBoxLayout()
4138     self.reset_btn = QPushButton("恢复系统默认值")
4139     self.reset_btn.setStyleSheet("padding: 8px;")
4140
4141     self.save_btn = QPushButton("保存并热加载配置")
4142     self.save_btn.setStyleSheet("background-color: #67c23a; color:
white; font-weight: bold; padding: 8px;")
4143
4144     btn_layout.addWidget(self.reset_btn)
4145     btn_layout.addStretch()
4146     btn_layout.addWidget(self.save_btn)
4147     main_layout.addLayout(btn_layout)
4148
4149
4150     self.save_btn.clicked.connect(self.on_save_clicked)
4151     self.reset_btn.clicked.connect(self.reset_requested.emit)
4152
4153     def create_slider(self):
4154         """工厂方法: 创建一个 0-100 的滑块和一个显示百分比的标签"""
4155         slider = QSlider(Qt.Orientation.Horizontal)
4156         slider.setRange(0, 100)
4157         slider.setPageStep(5)
4158
4159         val_label = QLabel("0%")
4160         val_label.setFixedWidth(40)
4161         val_label.setAlignment(Qt.AlignmentFlag.AlignRight |
Qt.AlignmentFlag.AlignVCenter)
4162
4163         slider.valueChanged.connect(lambda v, lbl=val_label:
lbl.setText(f"{v}%"))
4164         return slider, val_label
4165
4166     def wrap_slider(self, slider, val_label):
4167         """将滑块和标签包裹在一个水平布局中"""
4168         widget = QWidget()
4169         layout = QHBoxLayout(widget)
4170         layout.setContentsMargins(0, 0, 0, 0)
4171         layout.addWidget(slider)
4172         layout.addWidget(val_label)
4173         return widget
4174
4175     def bind_dirty_trackers(self):
4176         """绑定事件以追踪用户是否修改了数据"""
4177         self.porn_slider.valueChanged.connect(self.mark_dirty)
4178         self.violence_slider.valueChanged.connect(self.mark_dirty)
4179         self.politics_slider.valueChanged.connect(self.mark_dirty)
4180         self.ocr_checkbox.toggled.connect(self.mark_dirty)
4181         self.hash_checkbox.toggled.connect(self.mark_dirty)
4182
4183     def mark_dirty(self):
4184         """标记数据为未保存状态（标题加星号）"""
4185         if not self.is_dirty:

```

```

4187         self.is_dirty = True
4188         self.setWindowTitle("检测模型配置 *未保存")
4189         self.title_label.setText("<b>核心检测引擎参数配置</b> <span
style='color:red;'>*</span>")
4190
4191     def clear_dirty(self):
4192         """清除未保存状态标记"""
4193         self.is_dirty = False
4194         self.setWindowTitle("检测模型配置")
4195         self.title_label.setText("<b>核心检测引擎参数配置</b>")
4196
4197     def set_config_data(self, data: dict):
4198         """供逻辑层调用：渲染 JSON 配置数据到 UI 控件上"""
4199
4200         self.porn_slider.blockSignals(True)
4201         self.violence_slider.blockSignals(True)
4202         self.politics_slider.blockSignals(True)
4203         self.ocr_checkbox.blockSignals(True)
4204         self.hash_checkbox.blockSignals(True)
4205
4206         self.porn_slider.setValue(int(data.get("threshold_porn", 0.85) *
100))
4207         self.porn_val_label.setText(f"{self.porn_slider.value()}%")
4208
4209         self.violence_slider.setValue(int(data.get("threshold_violence",
0.85) * 100))
4210         self.violence_val_label.setText(f"{self.violence_slider.value()}%")
4211
4212         self.politics_slider.setValue(int(data.get("threshold_political",
0.90) * 100))
4213         self.politics_val_label.setText(f"{self.politics_slider.value()}%")
4214
4215         self.ocr_checkbox.setChecked(data.get("enable_ocr", True))
4216         self.hash_checkbox.setChecked(data.get("enable_hash", True))
4217
4218         last_time = data.get("last_updated", "从未配置")
4219         self.last_update_label.setText(f"生效时间: {last_time}")
4220
4221
4222         self.porn_slider.blockSignals(False)
4223         self.violence_slider.blockSignals(False)
4224         self.politics_slider.blockSignals(False)
4225         self.ocr_checkbox.blockSignals(False)
4226         self.hash_checkbox.blockSignals(False)
4227
4228         self.clear_dirty()
4229
4230     def on_save_clicked(self):
4231         """收集当前界面的数据并提交给逻辑层"""
4232         payload = {
4233             "threshold_porn": self.porn_slider.value() / 100.0,
4234             "threshold_violence": self.violence_slider.value() / 100.0,

```

```

4235         "threshold_political": self.politics_slider.value() / 100.0,
4236         "enable_ocr": self.ocr_checkbox.isChecked(),
4237         "enable_hash": self.hash_checkbox.isChecked()
4238     }
4239     self.save_requested.emit(payload)
4240
4241     def closeEvent(self, event):
4242         """拦截窗口关闭事件，防数据丢失"""
4243         if self.is_dirty:
4244             reply = QMessageBox.warning(
4245                 self, "未保存提醒", "您有未保存的配置修改，是否放弃这些修改并
4246                 退出? ",
4247                 QMessageBox.StandardButton.Yes |
4248                 QMessageBox.StandardButton.No,
4249                 QMessageBox.StandardButton.No
4250             )
4251             if reply == QMessageBox.StandardButton.Yes:
4252                 event.accept()
4253             else:
4254                 event.ignore()
4255         else:
4256             event.accept()
4257
4258     def show_toast(self, title, message):
4259         QMessageBox.information(self, title, message)
4260
4261     from PyQt6.QtWidgets import (QWidget, QVBoxLayout, QHBoxLayout, QLabel,
4262                                  QLineEdit, QPushButton, QFrame,
4263                                  QGraphicsDropShadowEffect)
4264     from PyQt6.QtCore import Qt, pyqtSignal
4265
4266     class RegisterUI(QWidget):
4267         register_requested = pyqtSignal(str, str, str, str, str)
4268         login_clicked = pyqtSignal()
4269
4270         def __init__(self):
4271             super().__init__()
4272             self.setWindowTitle("账户注册")
4273             self.setFixedSize(450, 640)
4274             self.setStyleSheet("QWidget { background-color: #F8FAFC; }")
4275             self._build()
4276
4277         def _build(self):
4278             layout = QVBoxLayout(self)
4279             layout.setAlignment(Qt.AlignmentFlag.AlignCenter)
4280
4281             card = QFrame()
4282             card.setFixedSize(380, 520)
4283             card.setStyleSheet("""
4284                 QFrame { background-color: #FFFFFF; border: 1px solid #E2E8F0;
4285                 border-radius: 16px; }
4286                 QLineEdit { background-color: #F1F5F9; border: 1px solid
4287                 #E2E8F0; border-radius: 8px; padding: 10px 14px; color: #0F172A; font-size:
4288                 13px; }

```

```

4283         QLineEdit:focus { border: 1px solid #3B82F6; }
4284         QPushButton#primaryBtn { background-color: #3B82F6; color:
white; border: none; border-radius: 8px; font-weight: 700; font-size: 14px; }
4285         QPushButton#primaryBtn:hover { background-color: #2563EB; }
4286         """)
4287
4288         shadow = QGraphicsDropShadowEffect(self)
4289         shadow.setBlurRadius(20)
4290         shadow.setColor(Qt.GlobalColor.lightGray)
4291         shadow.setOffset(0, 4)
4292         card.setGraphicsEffect(shadow)
4293
4294         card_layout = QVBoxLayout(card)
4295         card_layout.setContentsMargins(32, 35, 32, 35)
4296         card_layout.setSpacing(14)
4297
4298         title = QLabel("注册新账号")
4299         title.setStyleSheet("color: #0F172A; font-size: 22px; font-weight:
800; border: none;")
4300         title.setAlignment(Qt.AlignmentFlag.AlignCenter)
4301         card_layout.addWidget(title)
4302         card_layout.addSpacing(10)
4303
4304         self.username_input = QLineEdit()
4305         self.username_input.setPlaceholderText("设置账号 (字母/数字)")
4306         card_layout.addWidget(self.username_input)
4307
4308
4309         self.email_input = QLineEdit()
4310         self.email_input.setPlaceholderText("绑定邮箱 (如
user@example.com)")
4311         card_layout.addWidget(self.email_input)
4312
4313         self.password_input = QLineEdit()
4314         self.password_input.setPlaceholderText("设置密码")
4315         self.password_input.setEchoMode(QLineEdit.EchoMode.Password)
4316         card_layout.addWidget(self.password_input)
4317
4318         self.confirm_input = QLineEdit()
4319         self.confirm_input.setPlaceholderText("确认密码")
4320         self.confirm_input.setEchoMode(QLineEdit.EchoMode.Password)
4321         card_layout.addWidget(self.confirm_input)
4322
4323
4324         captcha_layout = QHBoxLayout()
4325         captcha_layout.setSpacing(10)
4326         self.captcha_input = QLineEdit()
4327         self.captcha_input.setPlaceholderText("验证码")
4328
4329
4330         from ui.login_ui import CaptchaWidget
4331         self.captcha_widget = CaptchaWidget()
4332
4333         captcha_layout.addWidget(self.captcha_input, 3)

```



```

4334         captcha_layout.addWidget(self.captcha_widget, 2)
4335         card_layout.addLayout(captcha_layout)
4336         card_layout.addSpacing(5)
4337
4338
4339         register_btn = QPushButton("注 册")
4340         register_btn.setObjectName("primaryBtn")
4341         register_btn.setFixedHeight(42)
4342         register_btn.clicked.connect(self._on_register_clicked)
4343         card_layout.addWidget(register_btn)
4344
4345
4346         back_btn = QPushButton("已有账号? 返回登录")
4347         back_btn.setStyleSheet(
4348             "QPushButton { color: #64748B; border: none; font-size:
12px; } QPushButton:hover { color: #3B82F6; }")
4349         back_btn.clicked.connect(self.login_clicked.emit)
4350         card_layout.addWidget(back_btn)
4351
4352         layout.addWidget(card)
4353
4354     def _on_register_clicked(self):
4355
4356         self.register_requested.emit(
4357             self.username_input.text().strip(),
4358             self.password_input.text(),
4359             self.confirm_input.text(),
4360             self.email_input.text().strip(),
4361             self.captcha_input.text().strip()
4362         )
4363
4364     def refresh_captcha(self):
4365         self.captcha_widget.update_captcha()
4366         self.captcha_input.clear()
4367 from PyQt6.QtWidgets import (QWidget, QVBoxLayout, QHBoxLayout,
QPushButton,
4368                               QLabel, QTreeWidget, QTreeWidgetItem,
QLineEdit,
4369                               QTextEdit, QComboBox, QDialog, QFormLayout,
4370                               QMessageBox, QFileDialog, QHeaderView)
4371 from PyQt6.QtCore import Qt, pyqtSignal
4372
4373
4374 class AddEditWordDialog(QDialog):
4375     """敏感词添加/编辑弹窗"""
4376
4377     def __init__(self, parent=None, word="", category="其他"):
4378         super().__init__(parent)
4379         self.setWindowTitle("管理敏感词")
4380         self.setFixedSize(300, 150)
4381
4382         layout = QFormLayout(self)
4383         self.word_input = QLineEdit(word)
4384         self.category_combo = QComboBox()

```

```

4385         self.category_combo.addItem("政治敏感", "色情", "暴力", "其他"])
4386         self.category_combo.setCurrentText(category)
4387
4388         self.save_btn = QPushButton("保存")
4389         self.save_btn.clicked.connect(self.accept)
4390
4391         layout.addRow("敏感词:", self.word_input)
4392         layout.addRow("所属类别:", self.category_combo)
4393         layout.addRow("", self.save_btn)
4394
4395     def get_data(self):
4396         return self.word_input.text().strip(),
4397         self.category_combo.currentText()
4398
4399 class SensitiveWordUI(QWidget):
4400     """敏感词管理主界面"""
4401
4402     load_requested = pyqtSignal()
4403     add_requested = pyqtSignal(str, str)
4404     delete_requested = pyqtSignal(str, str)
4405     import_requested = pyqtSignal(str)
4406     export_requested = pyqtSignal(str)
4407     test_requested = pyqtSignal(str)
4408
4409     def __init__(self):
4410         super().__init__()
4411         self.setup_ui()
4412
4413     def setup_ui(self):
4414         main_layout = QHBoxLayout(self)
4415         main_layout.setSpacing(15)
4416         main_layout.setContentsMargins(15, 15, 15, 15)
4417
4418         left_widget = QWidget()
4419         left_layout = QVBoxLayout(left_widget)
4420
4421         header_layout = QHBoxLayout()
4422         header_layout.addWidget(QLabel("<b>敏感词库树形视图</b>"))
4423         self.search_input = QLineEdit()
4424         self.search_input.setPlaceholderText("搜索敏感词...")
4425         header_layout.addWidget(self.search_input)
4426         left_layout.addLayout(header_layout)
4427
4428         self.tree_widget = QTreeWidget()
4429         self.tree_widget.setHeaderLabels(["敏感词", "操作提示"])
4430         self.tree_widget.header().setSectionResizeMode(0,
4431         QHeaderView.ResizeMode.Stretch)
4432         left_layout.addWidget(self.tree_widget)
4433
4434         btn_layout = QHBoxLayout()

```

```

4437         self.add_btn = QPushButton("添加词条")
4438         self.del_btn = QPushButton("删除选中")
4439         self.import_btn = QPushButton("批量导入")
4440         self.export_btn = QPushButton("导出词库")
4441
4442         for btn in [self.add_btn, self.del_btn, self.import_btn,
4443 self.export_btn]:
4444             btn_layout.addWidget(btn)
4445             left_layout.addLayout(btn_layout)
4446
4447         main_layout.addWidget(left_widget, stretch=5)
4448
4449         right_widget = QWidget()
4450         right_layout = QVBoxLayout(right_widget)
4451
4452         right_layout.addWidget(QLabel("<b>效果测试区域</b> (模拟 OCR 识别出
4453 的文本)"))
4454         self.test_input = QTextEdit()
4455         self.test_input.setPlaceholderText("在此输入测试文本，点击下方按钮检
4456 测效果...")
4457         right_layout.addWidget(self.test_input, stretch=3)
4458
4459         self.test_btn = QPushButton("开始智能检测")
4460         self.test_btn.setStyleSheet("background-color: #409eff; color:
4461 white; font-weight: bold; padding: 8px;")
4462         right_layout.addWidget(self.test_btn)
4463
4464         right_layout.addWidget(QLabel("<b>检测结果 (命中敏感词将高
4465 亮):</b>"))
4466         self.test_result = QTextEdit()
4467         self.test_result.setReadOnly(True)
4468         self.test_result.setStyleSheet("background-color: #f5f7fa;")
4469         right_layout.addWidget(self.test_result, stretch=3)
4470
4471         main_layout.addWidget(right_widget, stretch=5)
4472
4473         self.add_btn.clicked.connect(self.on_add_clicked)
4474         self.del_btn.clicked.connect(self.on_delete_clicked)
4475         self.import_btn.clicked.connect(self.on_import_clicked)
4476         self.export_btn.clicked.connect(self.on_export_clicked)
4477         self.test_btn.clicked.connect(self.on_test_clicked)
4478         self.search_input.textChanged.connect(self.filter_tree)
4479
4480     def on_add_clicked(self):
4481         dialog = AddEditWordDialog(self)
4482         if dialog.exec() == QDialog.DialogCode.Accepted:
4483             word, category = dialog.get_data()
4484             if word:
4485                 self.add_requested.emit(word, category)
4486             else:
4487                 self.show_toast("提示", "敏感词不能为空", True)

```

```

4486
4487     def on_delete_clicked(self):
4488         item = self.tree_widget.currentItem()
4489         if item and item.parent():
4490             word = item.text(0)
4491             category = item.parent().text(0)
4492             reply = QMessageBox.question(self, "确认", f"确定要删除敏感词
'{word}' 吗? ")
4493             if reply == QMessageBox.StandardButton.Yes:
4494                 self.delete_requested.emit(word, category)
4495             else:
4496                 self.show_toast("提示", "请展开类别并选中具体的敏感词进行删除",
True)
4497
4498     def on_import_clicked(self):
4499         file_path, _ = QFileDialog.getOpenFileName(self, "导入词库", "",
"Text/CSV Files (*.txt *.csv)")
4500         if file_path:
4501             self.import_requested.emit(file_path)
4502
4503     def on_export_clicked(self):
4504         save_path, _ = QFileDialog.getSaveFileName(self, "导出词库",
"sensitive_words_export.txt", "Text Files (*.txt)")
4505         if save_path:
4506             self.export_requested.emit(save_path)
4507
4508     def on_test_clicked(self):
4509         text = self.test_input.toPlainText().strip()
4510         if text:
4511             self.test_requested.emit(text)
4512
4513
4514     def render_tree(self, cache_dict):
4515         """渲染树形菜单"""
4516         self.tree_widget.clear()
4517         for category, words in cache_dict.items():
4518             root_item = QTreeWidgetItem(self.tree_widget, [category, f"共
{len(words)} 词"])
4519             root_item.setExpanded(True)
4520             for word in sorted(words):
4521                 child_item = QTreeWidgetItem(root_item, [word, ""])
4522                 child_item.setForeground(0,
Qt.GlobalColor.red if category
== "暴力" or category == "政治敏感" else Qt.GlobalColor.black)
4524
4525     def render_test_result(self, html_result):
4526         """渲染高亮后的 HTML 文本"""
4527         self.test_result.setHtml(html_result)
4528
4529     def filter_tree(self, text):
4530         """纯前端搜索过滤（无需查库）"""
4531         text = text.lower()
4532         for i in range(self.tree_widget.topLevelItemCount()):
4533             root = self.tree_widget.topLevelItem(i)

```

```

4534         root_match = False
4535         for j in range(root.childCount()):
4536             child = root.child(j)
4537             match = text in child.text(0).lower()
4538             child.setHidden(not match)
4539             if match:
4540                 root_match = True
4541
4542         root.setHidden(not (root_match or text == ""))
4543
4544     def show_toast(self, title, message, is_error=False):
4545         if is_error:
4546             QMessageBox.critical(self, title, message)
4547         else:
4548             QMessageBox.information(self, title, message)
4549 import os
4550 from PyQt6.QtWidgets import (QWidget, QHBoxLayout, QVBoxLayout,
4551                               QPushButton,
4552                               QLabel, QGraphicsView, QGraphicsScene,
4553                               QGraphicsPixmapItem,
4554                               QTextBrowser, QComboBox, QPlainTextEdit,
4555                               QProgressBar,
4556                               QGroupBox, QMessageBox, QFileDialog)
4557 from PyQt6.QtCore import Qt, pyqtSignal, QRectF
4558 from PyQt6.QtGui import QPixmap, QWheelEvent, QPainter
4559
4560 class AdvancedGraphicsView(QGraphicsView):
4561     """自定义图形视图：支持鼠标滚轮缩放、鼠标右键/中键拖拽平移"""
4562
4563     def __init__(self, parent=None):
4564         super().__init__(parent)
4565         self.setRenderHint(QPainter.RenderHint.Antialiasing)
4566         self.setRenderHint(QPainter.RenderHint.SmoothPixmapTransform)
4567         self.setTransformationAnchor(QGraphicsView.ViewportAnchor.AnchorUnderMouse)
4568         self.setResizeAnchor(QGraphicsView.ViewportAnchor.AnchorUnderMouse)
4569         self.setHorizontalScrollBarPolicy(Qt.ScrollBarPolicy.ScrollBarAlwaysOff)
4570         self.setVerticalScrollBarPolicy(Qt.ScrollBarPolicy.ScrollBarAlwaysOff)
4571         self.setDragMode(QGraphicsView.DragMode.ScrollHandDrag)
4572         self._zoom = 0
4573
4574     def wheelEvent(self, event: QWheelEvent):
4575         """滚动鼠标滚轮进行画布缩放"""
4576         if event.angleDelta().y() > 0:
4577             factor = 1.25
4578             self._zoom += 1
4579         else:
4580             factor = 0.8
4581             self._zoom -= 1
4582
4583         if self._zoom > -10 and self._zoom < 20:
4584             self.scale(factor, factor)

```

```

4583         else:
4584             if factor > 1:
4585                 self._zoom -= 1
4586             else:
4587                 self._zoom += 1
4588
4589
4590 class SingleImageAuditUI(QWidget):
4591     """单张图片审核用户界面"""
4592
4593     image_requested = pyqtSignal(str)
4594     save_requested = pyqtSignal(str, str, float, str, str)
4595     pdf_export_requested = pyqtSignal()
4596
4597     def __init__(self):
4598         super().__init__()
4599         self.current_image_path = ""
4600         self.current_confidence = 1.0
4601         self.rotation_angle = 0
4602         self.setup_ui()
4603
4604     def setup_ui(self):
4605         self.setObjectName("SingleImageAuditUI")
4606         main_layout = QHBoxLayout(self)
4607         main_layout.setSpacing(15)
4608         main_layout.setContentsMargins(15, 15, 15, 15)
4609
4610
4611         left_widget = QWidget()
4612         left_layout = QVBoxLayout(left_widget)
4613         left_layout.setContentsMargins(0, 0, 0, 0)
4614
4615
4616         btn_container = QHBoxLayout()
4617         self.upload_btn = QPushButton("选择本地图片")
4618         self.rotate_left_btn = QPushButton("逆时针旋转 90°")
4619         self.rotate_right_btn = QPushButton("顺时针旋转 90°")
4620         btn_container.addWidget(self.upload_btn)
4621         btn_container.addWidget(self.rotate_left_btn)
4622         btn_container.addWidget(self.rotate_right_btn)
4623         left_layout.addLayout(btn_container)
4624
4625
4626         self.scene = QGraphicsScene(self)
4627         self.view = AdvancedGraphicsView(self.scene)
4628         self.view.setStyleSheet("border: 2px dashed #dcdfe6; background-
4629 color: #f5f7fa; border-radius: 8px;")
4629         left_layout.addWidget(self.view)
4630
4631
4632         self.setAcceptDrops(True)
4633
4634
4635         right_widget = QWidget()

```

```

4636     right_widget.setFixedWidth(380)
4637     right_layout = QVBoxLayout(right_widget)
4638     right_layout.setContentsMargins(0, 0, 0, 0)
4639
4640
4641     status_box = QGroupBox("当前审核 workflow 状态")
4642     status_box_layout = QVBoxLayout(status_box)
4643     self.status_label = QLabel("系统状态: 待审核")
4644     self.status_label.setStyleSheet("font-size: 15px; font-weight:
bold; color: #909399;")
4645     status_box_layout.addWidget(self.status_label)
4646     right_layout.addWidget(status_box)
4647
4648
4649     ai_box = QGroupBox("AI 内容智能检测分布")
4650     ai_box_layout = QVBoxLayout(ai_box)
4651
4652     self.bars = {}
4653     categories = [("正常", "#67c23a"), ("色情", "#f56c6c"), ("暴恐",
"#e6a23c"), ("违禁", "#409eff")]
4654     for name, color in categories:
4655         row = QHBoxLayout()
4656         lbl = QLabel(f"{name}:")
4657         lbl.setFixedWidth(40)
4658         bar = QProgressBar()
4659         bar.setRange(0, 100)
4660         bar.setValue(0)
4661         bar.setTextVisible(True)
4662         bar.setStyleSheet(f"QProgressBar::chunk {{ background-color:
{color}; border-radius: 2px; }}")
4663         row.addWidget(lbl)
4664         row.addWidget(bar)
4665         ai_box_layout.addLayout(row)
4666         self.bars[name] = bar
4667     right_layout.addWidget(ai_box)
4668
4669
4670     ocr_box = QGroupBox("OCR 文字识别与敏感词高亮")
4671     ocr_box_layout = QVBoxLayout(ocr_box)
4672     self.ocr_browser = QTextBrowser()
4673     self.ocr_browser.setPlaceholderText("暂无文本识别数据")
4674     ocr_box_layout.addWidget(self.ocr_browser)
4675     right_layout.addWidget(ocr_box)
4676
4677
4678     action_box = QGroupBox("人工审查裁决中心")
4679     action_box_layout = QVBoxLayout(action_box)
4680
4681     action_box_layout.addWidget(QLabel("审核判定结论: "))
4682     self.result_combo = QComboBox()
4683     self.result_combo.addItem("通过", "驳回", "人工复核"])
4684     action_box_layout.addWidget(self.result_combo)
4685
4686     action_box_layout.addWidget(QLabel("综合审核处理意见: "))

```

```

4687         self.opinion_text = QTextEdit()
4688         self.opinion_text.setPlaceholderText("请在此处输入具体批注或整改意见...")
4689         action_box_layout.addWidget(self.opinion_text)
4690
4691
4692         operations_layout = QHBoxLayout()
4693         self.save_btn = QPushButton("保存审核结果")
4694         self.save_btn.setStyleSheet(
4695             "background-color: #409eff; color: white; font-weight: bold;
height: 32px; border-radius: 4px;")
4696         self.export_btn = QPushButton("导出 PDF 报告")
4697         self.export_btn.setStyleSheet(
4698             "background-color: #67c23a; color: white; font-weight: bold;
height: 32px; border-radius: 4px;")
4699         operations_layout.addWidget(self.save_btn)
4700         operations_layout.addWidget(self.export_btn)
4701         action_box_layout.addLayout(operations_layout)
4702
4703         right_layout.addWidget(action_box)
4704
4705
4706         main_layout.addWidget(left_widget, stretch=3)
4707         main_layout.addWidget(right_widget, stretch=1)
4708
4709
4710         self.upload_btn.clicked.connect(self.on_open_file_dialog)
4711         self.rotate_left_btn.clicked.connect(lambda: self.rotate_image(-
90))
4712         self.rotate_right_btn.clicked.connect(lambda:
self.rotate_image(90))
4713         self.save_btn.clicked.connect(self.on_save_clicked)
4714         self.export_btn.clicked.connect(lambda:
self.pdf_export_requested.emit())
4715
4716
4717         def dragEnterEvent(self, event):
4718             if event.mimeData().hasUrls():
4719                 event.acceptProposedAction()
4720
4721         def dropEvent(self, event):
4722             urls = event.mimeData().urls()
4723             if urls:
4724                 file_path = urls[0].toLocalFile()
4725                 ext = os.path.splitext(file_path)[1].lower()
4726                 if ext in ['.jpg', '.jpeg', '.png', '.bmp', '.webp']:
4727                     self.load_image_into_scene(file_path)
4728             else:
4729                 QMessageBox.warning(self, "格式错误", "系统仅支持载入 JPG,
PNG, BMP, WEBP 格式图像!")
4730
4731         def on_open_file_dialog(self):
4732             file_path, _ = QFileDialog.getOpenFileName(self, "选择审核图片",
"", "Images (*.jpg *.jpeg *.png *.bmp *.webp)")

```



```

4733         if file_path:
4734             self.load_image_into_scene(file_path)
4735
4736     def load_image_into_scene(self, path):
4737         """将选定或拖入的图片渲染到场景画布上"""
4738         self.current_image_path = path
4739         self.rotation_angle = 0
4740         pixmap = QPixmap(path)
4741         if pixmap.isNull():
4742             QMessageBox.critical(self, "图像异常", "图片文件已损坏或解码失
4743 败!")
4744         return
4745
4746         self.scene.clear()
4747         self.pixmap_item = QGraphicsPixmapItem(pixmap)
4748         self.scene.addItem(self.pixmap_item)
4749         self.scene.setSceneRect(QRectF(pixmap.rect()))
4750         self.view.fitInView(self.scene.sceneRect(),
4751                               Qt.AspectRatioMode.KeepAspectRatio)
4752
4753         self.image_requested.emit(path)
4754
4755     def rotate_image(self, angle):
4756         """步进旋转图片视图对象"""
4757         if hasattr(self, 'pixmap_item'):
4758             self.rotation_angle = (self.rotation_angle + angle) % 360
4759             self.view.rotate(angle)
4760
4761     def on_save_clicked(self):
4762         if not self.current_image_path:
4763             QMessageBox.warning(self, "提示", "请先载入并审核一张图片后再保
4764 存结果。")
4765         return
4766         res = self.result_combo.currentText()
4767         opinion = self.opinion_text.toPlainText().strip()
4768
4769         self.save_requested.emit(self.current_image_path, res,
4770                                   self.current_confidence, "admin", opinion)
4771
4772     def update_workflow_status(self, text, stylesheet_color):
4773         self.status_label.setText(f"系统状态: {text}")
4774         self.status_label.setStyleSheet(f"font-size: 15px; font-weight:
4775 bold; color: {stylesheet_color};")
4776
4777     def render_detection_metrics(self, data_dict):
4778         """渲染概率进度条"""
4779         target_class = data_dict.get("class", "正常")
4780         confidence = data_dict.get("confidence", 0.0)
4781         self.current_confidence = confidence
4782
4783         for name, bar in self.bars.items():
4784             if name == target_class:

```

```

4782         bar.setValue(int(confidence * 100))
4783     else:
4784
4785         bar.setValue(int((1.0 - confidence) * 30))
4786
4787     def render_ocr_text(self, text_list, sensitive_words):
4788         """渲染富文本 OCR 内容并高亮危险词汇"""
4789         self.ocr_browser.clear()
4790         full_html = ""
4791         for line in text_list:
4792             processed_line = line
4793             for word in sensitive_words:
4794                 if word in processed_line:
4795                     processed_line = processed_line.replace(word,
4796                                                                 f"<span
style='background-color: #ffcccc; color: #ff0000; font-weight:
bold;'>{word}</span>")
4797             full_html += f"<p>{processed_line}</p>"
4798         self.ocr_browser.setHtml(full_html if full_html else "未在图片内提
炼出可识别的文本。")
4799
4800     def update_decision_box(self, decision_text):
4801         self.result_combo.setCurrentText(decision_text)
4802
4803     def show_message_box(self, title, msg, is_error=False):
4804         if is_error:
4805             QMessageBox.critical(self, title, msg)
4806         else:
4807             QMessageBox.information(self, title, msg)
4808 import
matplotlib
4809 matplotlib.use('QtAgg')
4810 import matplotlib.pyplot as plt
4811 from matplotlib.backends.backend_qtagg import FigureCanvasQTAgg as
FigureCanvas
4812 from matplotlib.backends.backend_qtagg import NavigationToolbar2QT as
NavigationToolbar
4813
4814 from PyQt6.QtWidgets import (QWidget, QVBoxLayout, QHBoxLayout,
QPushButton,
4815                               QLabel, QComboBox, QDateEdit, QTabWidget,
QGroupBox,
4816                               QGridLayout, QFileDialog, QMessageBox)
4817 from PyQt6.QtCore import Qt, pyqtSignal, QDate
4818
4819
4820 matplotlib.rcParams['font.family'] = 'sans-serif'
4821 matplotlib.rcParams['font.sans-serif'] = ['Microsoft YaHei', 'SimHei',
'Arial']
4822 matplotlib.rcParams['axes.unicode_minus'] = False
4823
4824 class StatisticsReportUI(QWidget):
4825     """统计报表视图层"""
4826
4827

```

```

4828     query_requested = pyqtSignal(str, str)
4829
4830     export_requested = pyqtSignal(str, str, str)
4831
4832     def __init__(self):
4833         super().__init__()
4834         self.setup_ui()
4835         self.bind_events()
4836
4837     def setup_ui(self):
4838         self.setWindowTitle("数据统计与报表中心")
4839         main_layout = QVBoxLayout(self)
4840         main_layout.setSpacing(15)
4841
4842
4843         control_group = QGroupBox("报表筛选条件")
4844         control_layout = QHBoxLayout(control_group)
4845
4846         control_layout.addWidget(QLabel("时间范围:"))
4847         self.time_combo = QComboBox()
4848         self.time_combo.addItems(["最近 7 天", "今日", "本周", "本月", "自定义"])
4849         control_layout.addWidget(self.time_combo)
4850
4851
4852         self.start_date = QDateEdit(QDate.currentDate().addDays(-7))
4853         self.start_date.setCalendarPopup(True)
4854         self.start_date.setVisible(False)
4855         self.end_date = QDateEdit(QDate.currentDate())
4856         self.end_date.setCalendarPopup(True)
4857         self.end_date.setVisible(False)
4858         self.date_sep_label = QLabel("至")
4859         self.date_sep_label.setVisible(False)
4860
4861         control_layout.addWidget(self.start_date)
4862         control_layout.addWidget(self.date_sep_label)
4863         control_layout.addWidget(self.end_date)
4864
4865         self.query_btn = QPushButton("生成报表")
4866         self.query_btn.setStyleSheet("background-color: #409eff; color: white; font-weight: bold;")
4867         self.export_btn = QPushButton("导出 Excel")
4868         self.export_btn.setStyleSheet("background-color: #67c23a; color: white; font-weight: bold;")
4869
4870         control_layout.addStretch()
4871         control_layout.addWidget(self.query_btn)
4872         control_layout.addWidget(self.export_btn)
4873         main_layout.addWidget(control_group)
4874
4875
4876         metrics_layout = QHBoxLayout()
4877         card_total, self.lbl_total = self.create_metric_card("审核总量",
"0", "#409eff")

```

```

4878         card_pass, self.lbl_pass = self.create_metric_card("通过率",
4879 "0.0%", "#67c23a")
4879         card_reject, self.lbl_reject = self.create_metric_card("驳回拦截率",
4880 "0.0%", "#f56c6c")
4880
4881         metrics_layout.addWidget(card_total)
4882         metrics_layout.addWidget(card_pass)
4883         metrics_layout.addWidget(card_reject)
4884         main_layout.addLayout(metrics_layout)
4885
4886
4887         self.tabs = QTabWidget()
4888
4889
4890         self.trend_tab = QWidget()
4891         self.trend_layout = QVBoxLayout(self.trend_tab)
4892         self.trend_fig, self.trend_ax = plt.subplots(figsize=(6, 4))
4893         self.trend_canvas = FigureCanvas(self.trend_fig)
4894         self.trend_layout.addWidget(NavigationToolbar(self.trend_canvas,
4895 self.trend_tab))
4895         self.trend_layout.addWidget(self.trend_canvas)
4896         self.tabs.addTab(self.trend_tab, "审核量趋势图")
4897
4898
4899         self.dist_tab = QWidget()
4900         self.dist_layout = QHBoxLayout(self.dist_tab)
4901
4902         self.dist_fig, (self.pie_ax, self.bar_ax) = plt.subplots(1, 2,
4903 figsize=(10, 4))
4903         self.dist_canvas = FigureCanvas(self.dist_fig)
4904
4905         dist_vlayout = QVBoxLayout()
4906         dist_vlayout.addWidget(NavigationToolbar(self.dist_canvas,
4907 self.dist_tab))
4907         dist_vlayout.addWidget(self.dist_canvas)
4908         self.dist_layout.addLayout(dist_vlayout)
4909         self.tabs.addTab(self.dist_tab, "类别与工作量分布")
4910
4911         main_layout.addWidget(self.tabs, stretch=1)
4912
4913     def create_metric_card(self, title, init_val, color):
4914         """创建核心指标 UI 卡片"""
4915         card = QWidget()
4916         card.setStyleSheet(f"background-color: #f2f6fc; border-radius:
4917 8px; border-left: 5px solid {color};")
4917         layout = QVBoxLayout(card)
4918         title_lbl = QLabel(title)
4919         title_lbl.setStyleSheet("color: #909399; font-size: 14px;")
4920         val_lbl = QLabel(init_val)
4921         val_lbl.setStyleSheet(f"color: {color}; font-size: 24px; font-
4922 weight: bold;")
4922         val_lbl.setAlignment(Qt.AlignmentFlag.AlignCenter)
4923         layout.addWidget(title_lbl)
4924         layout.addWidget(val_lbl)

```

```

4925         return card, val_lbl
4926
4927     def bind_events(self):
4928
4929         self.time_combo.currentTextChanged.connect(self.toggle_date_pickers)
4930         self.query_btn.clicked.connect(self.trigger_query)
4931         self.export_btn.clicked.connect(self.trigger_export)
4932
4933     def toggle_date_pickers(self, text):
4934         is_custom = (text == "自定义")
4935         self.start_date.setVisible(is_custom)
4936         self.end_date.setVisible(is_custom)
4937         self.date_sep_label.setVisible(is_custom)
4938
4939     def get_date_range(self):
4940         """解析当前选择的时间范围"""
4941         text = self.time_combo.currentText()
4942         end_dt = QDate.currentDate()
4943
4944         if text == "今日":
4945             start_dt = end_dt
4946         elif text == "本周":
4947             start_dt = end_dt.addDays(-(end_dt.dayOfWeek() - 1))
4948         elif text == "本月":
4949             start_dt = QDate(end_dt.year(), end_dt.month(), 1)
4950         elif text == "最近 7 天":
4951             start_dt = end_dt.addDays(-6)
4952         else:
4953             start_dt = self.start_date.date()
4954             end_dt = self.end_date.date()
4955
4956         return start_dt.toString("yyyy-MM-dd"), end_dt.toString("yyyy-MM-
4957         dd")
4958
4959     def trigger_query(self):
4960         start, end = self.get_date_range()
4961         if start > end:
4962             QMessageBox.warning(self, "日期错误", "起始日期不能晚于结束日
4963             期!")
4964         return
4965         self.query_requested.emit(start, end)
4966
4967     def trigger_export(self):
4968         start, end = self.get_date_range()
4969         path, _ = QFileDialog.getSaveFileName(self, "导出报表", f"审核报表
4970         _{start}_{end}.xlsx", "Excel Files (*.xlsx)")
4971         if path:
4972             self.export_requested.emit(start, end, path)
4973
4974     def update_dashboard(self, data):
4975         """供逻辑层调用，刷新整个大屏"""
4976
4977         total = data.get("total", 0)
4978         passed = data.get("passed", 0)

```

```

4975         rejected = total - passed
4976
4977         self.lbl_total.setText(str(total))
4978         self.lbl_pass.setText(f"{max(0, passed/total*100):.1f}%" if
total > 0 else "0.0%")
4979         self.lbl_reject.setText(f"{max(0, rejected/total*100):.1f}%" if
total > 0 else "0.0%")
4980
4981
4982         self.trend_ax.clear()
4983         trend_data = data.get("trend", {})
4984         if trend_data:
4985             dates = list(trend_data.keys())
4986             counts = list(trend_data.values())
4987             self.trend_ax.plot(dates, counts, marker='o', linestyle='-',
color='#409eff', linewidth=2)
4988             self.trend_ax.fill_between(dates, counts, color='#409eff',
alpha=0.1)
4989             self.trend_ax.set_title("每日审核量走势")
4990             self.trend_ax.tick_params(axis='x', rotation=45)
4991             self.trend_fig.tight_layout()
4992         self.trend_canvas.draw()
4993
4994
4995         self.pie_ax.clear()
4996         cat_data = data.get("categories", {})
4997         if cat_data:
4998             labels = list(cat_data.keys())
4999             sizes = list(cat_data.values())
5000             colors = ['#f56c6c', '#e6a23c', '#909399', '#67c23a',
'#c8c9cc']
5001             self.pie_ax.pie(sizes, labels=labels, autopct='%1.1f%%',
startangle=140, colors=colors)
5002             self.pie_ax.set_title("违规类别占比")
5003
5004
5005         self.bar_ax.clear()
5006         work_data = data.get("workload", {})
5007         if work_data:
5008             auditors = list(work_data.keys())
5009             tasks = list(work_data.values())
5010             self.bar_ax.bar(auditors, tasks, color='#67c23a', width=0.5)
5011             self.bar_ax.set_title("审核员/系统 工作量统计")
5012             for i, v in enumerate(tasks):
5013                 self.bar_ax.text(i, v + 0.5, str(v), ha='center')
5014
5015         self.dist_fig.tight_layout()
5016         self.dist_canvas.draw()
5017
5018     def show_toast(self, title, msg, is_error=False):
5019         if is_error:
5020             QMessageBox.critical(self, title, msg)
5021         else:
5022             QMessageBox.information(self, title, msg)
5023
5024 from
PyQt6.QtWidgets import (QWidget, QVBoxLayout, QHBoxLayout, QPushButton,

```

```

5024             QLabel, QTableWidgetItem,
QLineEdit,
5025             QComboBox, QHeaderView, QDialog, QFormLayout,
QMessageBox)
5026 from PyQt6.QtCore import Qt, pyqtSignal
5027
5028
5029 class UserFormDialog(QDialog):
5030     """用户添加/编辑通用表单弹窗"""
5031
5032     def __init__(self, parent=None, is_edit=False, current_data=None):
5033         super().__init__(parent)
5034         self.is_edit = is_edit
5035         self.current_data = current_data or {}
5036         self.setWindowTitle("编辑用户" if is_edit else "创建新用户")
5037         self.setFixedSize(360, 260)
5038         self.setup_form()
5039
5040     def setup_form(self):
5041         layout = QVBoxLayout(self)
5042         form_layout = QFormLayout()
5043         form_layout.setSpacing(12)
5044
5045         self.username_input = QLineEdit()
5046         if self.is_edit:
5047             self.username_input.setText(self.current_data.get("username",
5048             ""))
5049             self.username_input.setEnabled(False)
5049         form_layout.addRow("用户名:", self.username_input)
5050
5051         self.password_input = QLineEdit()
5052         self.password_input.setEchoMode(QLineEdit.EchoMode.Password)
5053         if self.is_edit:
5054             self.password_input.setPlaceholderText("留空表示不修改密码")
5055         form_layout.addRow("系统密码:", self.password_input)
5056
5057         self.email_input = QLineEdit()
5058         if self.is_edit:
5059             self.email_input.setText(self.current_data.get("email", ""))
5060         form_layout.addRow("电子邮箱:", self.email_input)
5061
5062         self.role_combo = QComboBox()
5063         self.role_combo.addItem("user", "auditor", "admin"])
5064         if self.is_edit:
5065             self.role_combo.setCurrentText(self.current_data.get("role",
5066             "user"))
5067         form_layout.addRow("分配角色:", self.role_combo)
5068
5069         layout.addLayout(form_layout)
5070         layout.addSpacing(10)
5071
5072         btn_layout = QHBoxLayout()
5073         self.submit_btn = QPushButton("确定提交")

```

```

5074         self.cancel_btn = QPushButton("取消")
5075         self.submit_btn.setStyleSheet("background-color: #409eff; color:
white; font-weight: bold;")
5076         btn_layout.addWidget(self.submit_btn)
5077         btn_layout.addWidget(self.cancel_btn)
5078         layout.addLayout(btn_layout)
5079
5080         self.submit_btn.clicked.connect(self.accept)
5081         self.cancel_btn.clicked.connect(self.reject)
5082
5083     def get_fields(self):
5084         return {
5085             "username": self.username_input.text().strip(),
5086             "password": self.password_input.text().strip(),
5087             "email": self.email_input.text().strip(),
5088             "role": self.role_combo.currentText()
5089         }
5090
5091
5092 class UserManagementUI(QWidget):
5093     """用户管理中央控制台 UI"""
5094
5095     refresh_requested = pyqtSignal(str, str, int)
5096     create_submitted = pyqtSignal(dict)
5097     edit_submitted = pyqtSignal(int, dict)
5098     delete_action_triggered = pyqtSignal(int, str)
5099     pwd_reset_triggered = pyqtSignal(int, str)
5100     unlock_triggered = pyqtSignal(int, str)
5101
5102     def __init__(self):
5103         super().__init__()
5104         self.current_page = 1
5105         self.setup_ui()
5106
5107     def setup_ui(self):
5108         self.setObjectName("UserManagementUI")
5109         main_layout = QVBoxLayout(self)
5110         main_layout.setSpacing(12)
5111         main_layout.setContentsMargins(15, 15, 15, 15)
5112
5113
5114         top_filter_layout = QHBoxLayout()
5115         top_filter_layout.setSpacing(10)
5116
5117         top_filter_layout.addWidget(QLabel("搜索账号:"))
5118         self.search_input = QLineEdit()
5119         self.search_input.setPlaceholderText("请输入要模糊检索的用户名...")
5120         self.search_input.setFixedWidth(200)
5121         top_filter_layout.addWidget(self.search_input)
5122
5123         top_filter_layout.addWidget(QLabel("角色筛选:"))
5124         self.role_filter = QComboBox()
5125         self.role_filter.addItems(["全部角色", "admin", "auditor", "user"])
5126         top_filter_layout.addWidget(self.role_filter)

```



```

5127
5128         self.query_btn = QPushButton("查询")
5129         self.query_btn.setStyleSheet("background-color: #409eff; color:
white; font-weight: bold;")
5130         top_filter_layout.addWidget(self.query_btn)
5131         top_filter_layout.addStretch()
5132
5133
5134         self.add_user_btn = QPushButton("+ 登记新用户")
5135         self.add_user_btn.setStyleSheet("background-color: #67c23a; color:
white; font-weight: bold;")
5136         top_filter_layout.addWidget(self.add_user_btn)
5137
5138         main_layout.addLayout(top_filter_layout)
5139
5140
5141         self.data_table = QTableWidgetItem()
5142         self.data_table.setColumnCount(6)
5143         self.data_table.setHorizontalHeaderLabels(
5144             ["用户 ID", "账户名", "绑定的电子邮箱", "系统授权角色", "锁定安全
状态", "快捷审查操作"])
5145
5146         self.data_table.horizontalHeader().setSectionResizeMode(QHeaderView.ResizeMode.
Stretch)
5147
5148         self.data_table.setSelectionBehavior(QTableWidgetItem.SelectionBehavior.SelectRows)
5149
5150         self.data_table.setEditTriggers(QTableWidgetItem.EditTrigger.NoEditTriggers)
5151
5152         self.data_table.setStyleSheet("QTableWidgetItem { background-color:
#ffffff; border-radius: 6px; }")
5153
5154         main_layout.addWidget(self.data_table)
5155
5156
5157         footer_layout = QHBoxLayout()
5158         self.prev_btn = QPushButton("◀ 上一页")
5159         self.next_btn = QPushButton("下一页 ▶")
5160         self.page_info_label = QLabel("第 1 / 1 页")
5161         self.page_info_label.setStyleSheet("font-weight: bold; color:
#606266;")
5162
5163         footer_layout.addStretch()
5164         footer_layout.addWidget(self.prev_btn)
5165         footer_layout.addWidget(self.page_info_label)
5166         footer_layout.addWidget(self.next_btn)
5167         footer_layout.addStretch()
5168         main_layout.addLayout(footer_layout)
5169
5170
5171         self.query_btn.clicked.connect(self.emit_refresh)
5172         self.prev_btn.clicked.connect(self.goto_prev_page)
5173         self.next_btn.clicked.connect(self.goto_next_page)
5174         self.add_user_btn.clicked.connect(self.open_add_dialog)
5175
5176     def emit_refresh(self):

```

```

5172         word = self.search_input.text().strip()
5173         role = self.role_filter.currentText()
5174         self.refresh_requested.emit(word, role, self.current_page)
5175
5176     def goto_prev_page(self):
5177         if self.current_page > 1:
5178             self.current_page -= 1
5179             self.emit_refresh()
5180
5181     def goto_next_page(self):
5182         self.current_page += 1
5183         self.emit_refresh()
5184
5185     def open_add_dialog(self):
5186         dialog = UserFormDialog(self, is_edit=False)
5187         if dialog.exec() == QDialog.DialogCode.Accepted:
5188             self.create_submitted.emit(dialog.get_fields())
5189
5190
5191     def render_user_grid(self, users_list, total_pages, active_page):
5192         """渲染重组用户网格与行内操纵按钮"""
5193         self.current_page = active_page
5194         self.data_table.setRowCount(0)
5195         self.page_info_label.setText(f"第 {active_page} / {max(1,
total_pages)}} 页")
5196
5197         for row_idx, user in enumerate(users_list):
5198             self.data_table.insertRow(row_idx)
5199             uid, name, email, role, lock_time = user
5200
5201
5202             status_text = f"锁死至: {lock_time}" if lock_time else "正常活
动"
5203
5204             self.data_table.setItem(row_idx, 0,
QTableWidgetItem(str(uid)))
5205             self.data_table.setItem(row_idx, 1, QTableWidgetItem(name))
5206             self.data_table.setItem(row_idx, 2, QTableWidgetItem(email if
email else "--"))
5207             self.data_table.setItem(row_idx, 3, QTableWidgetItem(role))
5208             self.data_table.setItem(row_idx, 4,
QTableWidgetItem(status_text))
5209
5210
5211             action_panel = QWidget()
5212             panel_layout = QHBoxLayout(action_panel)
5213             panel_layout.setContentsMargins(4, 2, 4, 2)
5214             panel_layout.setSpacing(6)
5215
5216             edit_btn = QPushButton("改")
5217             del_btn = QPushButton("删")
5218             reset_btn = QPushButton("设密")
5219             unlock_btn = QPushButton("解")
5220

```

```

5221
5222         for b in [edit_btn, del_btn, reset_btn, unlock_btn]:
5223             b.setFixedWidth(38)
5224             b.setStyleSheet("font-size: 11px; padding: 2px;")
5225
5226         panel_layout.addWidget(edit_btn)
5227         panel_layout.addWidget(del_btn)
5228         panel_layout.addWidget(reset_btn)
5229         if lock_time:
5230             panel_layout.addWidget(unlock_btn)
5231
5232         self.data_table.setCellWidget(row_idx, 5, action_panel)
5233
5234
5235         edit_btn.clicked.connect(lambda checked, u=user:
self.open_edit_dialog(u))
5236         del_btn.clicked.connect(lambda checked, i=uid, r=role:
self.delete_action_triggered.emit(i, r))
5237         reset_btn.clicked.connect(lambda checked, i=uid, r=role:
self.pwd_reset_triggered.emit(i, r))
5238         unlock_btn.clicked.connect(lambda checked, i=uid, r=role:
self.unlock_triggered.emit(i, r))
5239
5240     def open_edit_dialog(self, user_tuple):
5241         uid, name, email, role, _ = user_tuple
5242         cache_data = {"username": name, "email": email, "role": role}
5243         dialog = UserFormDialog(self, is_edit=True,
current_data=cache_data)
5244         if dialog.exec() == QDialog.DialogCode.Accepted:
5245             self.edit_submitted.emit(uid, dialog.get_fields())
5246
5247     def show_alert(self, title, message, is_error=False):
5248         if is_error:
5249             QMessageBox.critical(self, title, message)
5250         else:
5251             QMessageBox.information(self, title, message)5252
from .login_ui import LoginUI
5253 from .main_window_ui import MainWindowUI
5254 from .statistics_report_ui import StatisticsReportUI
5255 from .about_system_ui import AboutSystemUI
5256 from .user_management_ui import UserManagementUI
5257 from .sensitive_word_management_ui import SensitiveWordUI
5258 from .single_image_audit_ui import SingleImageAuditUI
5259 from .batch_image_audit_ui import BatchImageAuditUI
5260 from .audit_history_ui import AuditHistoryUI
5261 from .appeal_handling_ui import AppealHandlingUI
5262 from .data_backup_ui import DataBackupUI
5263 from .help_center_ui import HelpCenterUI
5264 from .model_config_ui import ModelConfigUI
5265 from .register_ui import RegisterUI
5266
5267
5268 __all__ = [
5269     "LoginUI",
5270     "MainWindowUI",

```

```
5271     "StatisticsReportUI",
5272     "SystemSettingsUI",
5273     "AboutSystemUI",
5274     "UserManagementUI",
5275     "SensitiveWordUI",
5276     "SingleImageAuditUI",
5277     "BatchImageAuditUI",
5278     "AuditHistoryUI",
5279     "AppealHandlingUI",
5280     "DataBackupUI",
5281     "HelpCenterUI",
5282     "ModelConfigUI",
5283     "RegisterUI"
5284 ]
```

Github-Liunznz